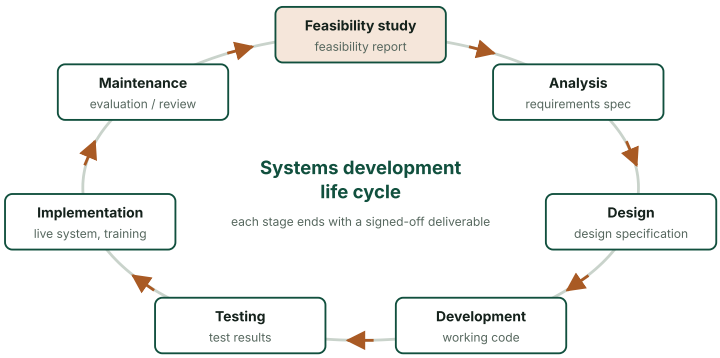
Why develop new systems? Systems, the software crisis and the life cycle

AP01 FACT FILE

Specification: students should be able to

- Describe the characteristics of a system and explain how large systems are made up of sub-systems
- Understand that a computer system consists of a user interface, processes and data, and explain the factors that affect the choice of user interface
- Explain the reasons why an organisation may need to develop a new information system
- Explain the impact of the software crisis and the emergence of software engineering
- Describe the stages of the systems development life cycle (SDLC), the purpose of each stage and why a structured approach is used

Key words: system, sub-system, computer system, user interface (UI), process, data, information, legacy system, software crisis, software engineering, user requirements, systems development life cycle (SDLC), feasibility, analysis, design, maintenance, evaluation, deliverable



The systems development life cycle: seven stages arranged in a cycle, each labelled with its main deliverable

Systems and why we develop them

A system is a collection of interdependent components that work together to achieve a purpose. Every system has three characteristics: a defined purpose, a defined need that it meets, and a number of components that rely on one another. Large systems, such as the computer system of a health trust or an airline, are usually too complex to understand as a whole, so they are broken down into smaller sub-systems, such as appointments, pharmacy and payroll, which pass data between them. In this course, systems development means creating systems that involve computer processing. Organisations develop new systems when the current one no longer serves its purpose or meets its need: the business has grown, laws have changed, competitors offer better services, technology has moved on, or the old system is unreliable. Because software development went badly wrong in its early years, a period known as the software crisis, the industry adopted a structured, engineering approach, which today is represented by the systems development life cycle. This lesson covers all of these ideas, which form the foundation for the rest of AS 1.

Data, user interface and processes

A computer system uses computing power to support a function. It must be able to receive data, process it, store both the data received and the results of processing, turn processed data into information that people can use, and pass data to and from linked sub-systems. Data are raw facts and figures without context; information is data that has been processed and given meaning, such as a low-balance alert or a monthly sales report. Any computer system can therefore be described as three elements. The data is what the system holds and works on, whether entered by users or generated by processes. The user interface is how users send data into the system and see the results. The processes are the operations that manipulate the data, such as calculating, validating, searching, sorting and updating records. Each element matters in its own right: inaccurate or incomplete data produces unreliable output; a poor interface leads users to make errors, take longer or avoid the system altogether; and poor processing produces incorrect results or wastes time. When answering questions, always give concrete examples of each element for the scenario.

Choosing a suitable user interface

The user interface is the gateway between a system and its users, so its usability often decides whether a system succeeds. The type of interface chosen depends on three things. The first is the application: how much data must be input and what output is required. A self-service kiosk at a cinema needs only a few taps, whereas a nurse updating a patient record must complete many fields, so a keyboard, mouse and form-based screen suit that task. The second is the environment: a busy, noisy shop floor, a moving vehicle or an outdoor car park each impose limits. A satnav takes a destination by touchscreen while the car is stopped, but gives directions by voice while driving so that the driver keeps their eyes on the road. The third is the characteristics of the users: trained staff can cope with a complex interface, but members of the public need something simple and intuitive, with accessibility features for users with disabilities. Interfaces also evolve with technology and demand, from manual payment to contactless cards, barcode scanning, voice assistants and automatic number-plate recognition.

Reasons for a new system

Organisations develop new systems when the current system no longer meets their needs. Internal reasons include growth in the volume of data or transactions, inefficient manual processes, duplicated data entry and the errors that follow, unreliable or unsupported legacy hardware and software, and the need for better management information to support decisions. External reasons include changes in legislation, such as data protection or tax reporting rules, pressure from competitors, rising customer expectations of online and mobile access, and the arrival of new, affordable technology. A good answer always links the reason to its effect on the organisation, for example: the paper diary cannot be used by two members of staff at once, so double bookings occur and customers are lost. Simply saying that a new system would be faster or better rarely earns credit unless you explain what the improvement means in that context. A new system has to satisfy the organisation that operates it as well as the individuals who use it, which is covered further in AP02.

The software crisis and software engineering

In the early decades of computing, software was written informally, with few agreed methods. From the mid-1960s, demand for larger and more complex systems grew rapidly, and the industry could not keep up; the term software crisis became widely used after the 1968 NATO Software Engineering Conference held in Germany. Several factors caused the crisis. Developers had little experience of similar projects on which to base estimates, so time and cost estimates were crude and usually wrong. Development tools were primitive, so software took a long time to write, and there were no tools to help detect errors early, which made faults slow and expensive to find and systems costly to maintain. Methods for capturing requirements were imprecise, so systems often failed to do what users needed, and there were no specialist testers or formal testing process. The impact was software that was late, over budget, unreliable and sometimes abandoned. The response was software engineering: treating the design, construction and upkeep of software as a disciplined, systematic engineering activity, in the way civil or mechanical engineers work, so that quality software is delivered on time, within budget and to the user's requirements.

The systems development life cycle

The SDLC is a structured framework that breaks the development of an information system into stages: feasibility study, analysis, design, development, testing, implementation, and maintenance and evaluation. Each stage has clear objectives and deliverables, such as the feasibility report, requirements specification, design specification, test results and user documentation, and these are usually signed off by the client before the next stage begins. Documentation is produced throughout, and the finished system is formally handed over to the client. Precise definition of the user requirements is critical: an error made at this stage may only be discovered after coding, when it is expensive to put right because design, code and tests must all be redone. Breaking the project into stages allows managers to estimate costs, plan and monitor progress against milestones. The main weakness of a strict sequence is inflexibility: if the client's needs change late in the project, returning to earlier stages is slow and costly, which is why approaches such as prototyping and agile methods exist. The process is called a cycle because evaluation and maintenance of one system eventually reveal the need for its replacement.

T Key terms from the mark schemes

System
A set of interdependent components that work together for a stated purpose, satisfying an identified need.
`Credited: Something with a purpose, a need and a number of components`

Sub-system
A smaller system that forms part of a larger system and performs one part of its overall function, often exchanging data with other sub-systems.

User interface (UI)
The means by which a user interacts with a computer system, entering data into it and viewing the results of processing.
`Credited: How the user interacts with the system`

Data
Raw facts and figures that have no context or meaning on their own, which are input to or stored by a system.
`Credited: Raw facts`

Process
An operation carried out by a computer system that manipulates data, for example calculating, sorting, searching or updating, converting data into information.
`Credited: A task carried out by the system`

Software crisis
The period from the mid-1960s when software projects were frequently late, over budget, unreliable and failed to meet user requirements because development methods could not keep pace with demand.

Software engineering
Using a disciplined, systematic engineering process to design, build and maintain software.

Systems development life cycle (SDLC)
A structured framework of stages (feasibility, analysis, design, development, testing, implementation, maintenance and evaluation) used to plan, build and maintain an information system.

Legacy system
An old system that is still in use but is often unreliable, unsupported or expensive to maintain.

Deliverable
A document or product, such as a feasibility report or test plan, that is produced and signed off at the end of a stage of a project.

Information system
A system of people, procedures, hardware, software and data that collects, processes, stores and distributes information to support an organisation.

Analysis
The stage in which the current system is investigated using fact-finding methods to establish the problems and the requirements of the new system.

Exam focus

State or list questions want short points, one mark each, such as naming a characteristic of a system or a function of a computer system. Describe questions need the point plus what it involves, for example: Analysis, the current system is investigated using fact-finding methods to establish the user requirements (2). Explain questions need a reason or consequence, so use because or so that. For the software crisis, mark schemes credit specific causes: little experience so inaccurate estimates, unsophisticated tools, no early error detection, systems not meeting user requirements, and no specialist testing; and they credit software engineering as a systematic or structured approach to design, development and maintenance. When asked for the data, user interface and processes of a scenario, give items specific to it, such as product ID and quantity in stock for a stock system, rather than generic words. Always apply answers to the context; do not confuse testing (checking the system works) with evaluation (judging whether it met its objectives once in use), or data with information.

? Check yourself

1. A Northern Ireland bus operator uses a real-time passenger information system at its stops. State the three characteristics that this, like every system, must have. [3]
2. A leisure centre run by a Northern Ireland council uses a computer system to manage memberships, class bookings and payments. Identify the purpose, the need and two components of this system. [4]
3. A newsagent in Omagh uses a computer system for stock control. For this system, describe one example of the data it holds, one example of its user interface and one example of a process it carries out. [6]

Answers: 1. A defined purpose, e.g. to tell passengers when the next bus will arrive (1) / meets a defined need, e.g. passengers waiting without knowing about delays (1) / a number of interdependent components, e.g. GPS units on buses, a central server, display screens at stops (1). Accept the characteristics without examples. 2. Purpose, e.g. to manage memberships and allow customers to book classes and pay (1) / Need, e.g. high volume of bookings cannot be handled manually; avoid overbooked classes (1) / Any two components, e.g. membership database (1), online booking website/app (1), card payment terminal, access-control turnstiles, staff PCs, network 3. Data (max 2): an appropriate data item, e.g. product barcode/ID, description, quantity in stock, reorder level, price (1) with description of where it comes from/why it is held (1) / User interface (max 2): e.g. barcode scanner at till / stock screen to view and edit levels / form to add new products (1) with how it is used (1) / Processes (max 2): e.g. reduce stock quantity when an item is sold, increase it when a delivery arrives, compare quantity with reorder level and produce reorder list, produce sales reports (1) with description (1)

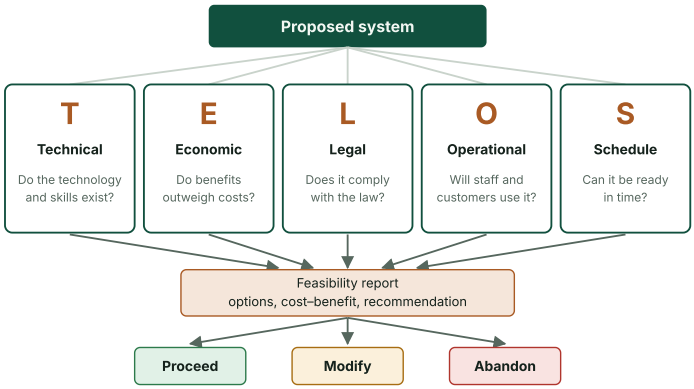
User needs, time and cost: the feasibility study and the development team

AP02 FACT FILE

Specification: students should be able to

- Explain the need for software systems that meet the needs of organisations and/or individuals
- Explain the main factors affecting systems development: the user needs, time and cost
- Describe the purpose of a feasibility study and explain technical, economic, legal, operational and schedule feasibility
- Explain the use of a cost–benefit analysis and describe the content of a feasibility report
- Describe the roles of the systems analyst, the project manager and the programmer during systems development

Key words: user needs, constraint, budget, deadline, feasibility study, TELOS, technical feasibility, economic feasibility, legal feasibility, operational feasibility, schedule feasibility, cost–benefit analysis, tangible benefit, intangible benefit, payback period, feasibility report, systems analyst, project manager



A feasibility study tests a proposed system against five TELOS criteria and produces a feasibility report recommending to proceed, modify or abandon

Meeting needs within time and cost

Every new system is developed to meet a need, and it must satisfy two groups at once. The organisation needs the system to fulfil its purpose: a ferry operator, for example, cannot take thousands of bookings a week by phone, so an online system is essential. The individuals who use the system, whether customers or staff, need it to be accessible, quick and easy to use; if customers find it confusing they will go to a competitor, and if staff find it awkward they will make mistakes or find workarounds, so the system fails. Development is also shaped by three factors: the user needs, the time available and the cost. User needs define what must be built, time sets the deadline by which it must work, and cost sets the budget. Before committing to a project, an organisation carries out a feasibility study to check that the proposed system can meet its needs within these limits, and a team of specialists, led by the systems analyst, project manager and programmer, then carries the work through. This lesson covers each of these ideas.

User needs, time and cost as constraints

The user needs are the requirements that the organisation and its users have of the system: what it must do, what data it must handle and how people will interact with it. They must be identified precisely and agreed, because a misunderstood requirement leads to the wrong system being built. Time is a constraint because most systems have a fixed date by which they must be operational, such as the start of a financial year, a tourist season or a change in the law. There is no value in designing a feature that will take a year to build if the system must go live in six months, so the scope must fit the schedule. Cost is a constraint because every project has a budget. The budget limits how many developers can be employed, what hardware and software can be bought, and how much training and testing can be paid for. The three factors interact: adding features increases time and cost, a shorter deadline needs more staff or fewer features, and a smaller budget means a smaller system. Cutting testing to save time or money is a false economy that echoes the failures of the software crisis.

Purpose of a feasibility study and TELOS

A feasibility study is a short, preliminary investigation that decides whether a proposed system should be developed. It examines the problem, looks at alternative solutions, including improving or keeping the current system, and evaluates them against five criteria often remembered as TELOS. Technical feasibility asks whether suitable hardware, software, network capacity and technical skills exist. Economic feasibility weighs costs against benefits. Legal feasibility checks compliance with legislation such as the UK GDPR and Data Protection Act 2018, the Computer Misuse Act 1990 as amended, copyright law and accessibility duties. Operational feasibility considers whether the system will fit the organisation's working practices and whether staff and customers will accept and use it. Schedule feasibility asks whether it can be delivered by a required deadline. The output is a feasibility report containing the terms of reference, the problems with the current system, the options considered, the TELOS findings with outline costs and benefits, and a justified recommendation with an outline plan. Management use it to decide whether to approve and fund the project, so it reduces the risk of committing large sums to an unworkable idea.

Cost–benefit analysis and payback

A cost–benefit analysis lists the one-off and ongoing costs of a proposed system and compares them with its benefits over a period, often three to five years. One-off costs include development, hardware, software licences, converting existing data and initial training; ongoing costs include maintenance, support, licence renewals and consumables. Tangible benefits can be given a money value, such as fewer staff hours spent on data entry or reduced stock wastage, while intangible benefits, such as improved reputation, better customer satisfaction or better decision-making, are harder to quantify but still matter. The payback period is the time taken for cumulative benefits to equal cumulative costs. To calculate it, set out the costs and benefits year by year, work out the net figure for each year and keep a running cumulative total; payback occurs when the total reaches zero. For example, a £30 000 system with running costs of £4 000 a year that saves £14 000 a year has a net saving of £10 000 a year, so it pays back in 30 000 ÷ 10 000 = 3 years. Because every figure is an estimate, the analysis should state its assumptions.

The systems analyst

The systems analyst's central job is to establish what users need from a new or changed computer system and what data processing it must carry out. The analyst acts as the link between the client, who understands the business, and the technical team, who will build the system. Early in a project the analyst carries out the feasibility study and writes the feasibility report. If the project goes ahead, the analyst investigates the current system using fact-finding methods: interviewing managers and users, observing staff at work, issuing questionnaires to large groups and inspecting existing documents such as forms, invoices and reports. The findings are analysed to establish exactly what the new system must do, and the analyst then defines the system specification, the detailed proposal for a new, modified or replacement system that the programmers will work from. Throughout, the analyst liaises with the client to make sure requirements are understood and agreed. Good analysts need strong communication skills as well as technical knowledge, because a requirement that is misunderstood at this stage becomes very expensive to correct once code has been written.

The project manager and the programmer

The project manager plans the project, co-ordinates the people working on it and keeps control of its spending. They make sure it is completed on time, within budget and to its objectives, that resources are used efficiently and that everyone involved is doing their job. Typical duties are planning the schedule, splitting the project into tasks and sub-tasks, allocating staff, hardware and software to each task, identifying risks, monitoring progress against milestones and reporting back to the client. The programmer builds and tests the system to the analyst's design. Programmers write or amend source code in a high-level language, working from module specifications, algorithms or flowcharts. They comment their code meaningfully so others can maintain it, test it using a test plan, debug any faults found, integrate the new code with existing systems, write operational documentation and work closely with the analyst and project manager so that requirements are met in the expected time. A programmer who built a system is well placed to maintain it later, and experienced programmers are valuable for maintaining and enhancing other systems too.

T Key terms from the mark schemes

Feasibility study
A preliminary investigation to decide whether a proposed system is technically, economically, legally, operationally and schedule feasible, ending in a recommendation.
Credited: TELOS

Feasibility report
The document produced by a feasibility study that describes the options considered, their feasibility and a justified recommendation for management.

Operational feasibility
Whether a proposed system will work in practice in the organisation and be accepted and used by its staff and customers.

Cost–benefit analysis
A comparison of the estimated costs of a proposed system with its expected tangible and intangible benefits over a period of time.

Payback period
The length of time it takes for the cumulative benefits of a system to equal its cumulative costs.

Intangible benefit
A benefit that is difficult to measure in money, such as improved customer satisfaction or staff morale.

Tangible benefit
A benefit that can be measured in money, such as reduced staffing or postage costs.

User needs
The requirements that an organisation and the individuals who use a system have of it: what it must do and how they need to interact with it.
Credited: User requirements

Systems analyst
The person who identifies the user and data processing requirements of a new or modified system, carrying out the feasibility study, fact-finding and producing the system specification.

Project manager
The person responsible for planning, co-ordinating and controlling a project so that it is delivered on time, within budget and meets its objectives.

Programmer
The person who writes, documents, tests and debugs the source code for the system designed by the systems analyst.

C C# example

Programmer

```
// Module: payback period (spec 4.2 from the analyst)
// Input: one-off cost and yearly net saving in pounds
// Output: number of whole years until costs are recovered
double initialCost = 30000;      // development + hardware
double netSavingPerYear = 10000; // savings minus running costs
double cumulative = -initialCost;
int year = 0;

// Add each year's net saving until the total is no longer negative
while (cumulative < 0)
{
    year++;
    cumulative += netSavingPerYear;
}
Console.WriteLine($"Payback reached in year {year}");
```

Exam focus

When asked to explain an area of feasibility, name it and then apply it to the scenario, for example: Operational, the staff are not confident with computers, so they may resist the change unless they are trained (2). Do not simply define the word economic. For calculation questions, set out costs and benefits year by year with a cumulative total; method marks are often available even if the final figure is wrong. For roles, questions typically ask for the main role (1 mark) and a number of duties (1 mark each). Mark schemes credit the analyst identifying user and data processing requirements, doing the feasibility study, liaising with the client, fact-finding and defining the system specification; the project manager planning the schedule, splitting tasks, allocating resources, identifying risks, monitoring progress, keeping to time and budget, and reporting to the client; and the programmer coding from specifications, commenting, debugging, testing with a test plan, integrating with existing systems and writing documentation. A common mistake is giving the programmer the analyst's fact-finding duties, or the project manager the coding. For needs, time and cost, explain the effect on the project, not just the factor's name.

? Check yourself

1. State the purpose of a feasibility study. [1]
2. A ferry company operating from Larne is developing an online booking system. Explain why the system must meet the needs of both the company and its individual customers. [4]
3. Explain how time and cost can each limit what is delivered when a new system is developed for a Belfast secondary school. [4]

Answers: 1. To decide whether a proposed system is possible/worthwhile/should go ahead, resulting in a recommendation to management (1) 2. Company (max 2): the volume of bookings is too large to handle by phone or in person (1), so an online system is needed to take bookings and payments efficiently and fill sailings (1) / Customers (max 2): customers must find it accessible and easy to use, e.g. clear choice of sailing, vehicle type, payment (1); if not, they may book with a competitor so the system fails its purpose (1) 3. Time (2): the system must be operational by a fixed date, e.g. the start of the school year (1), so features that cannot be built and tested by then must be dropped or delayed (1) / Cost (2): the budget is limited (1), so it restricts the number of developers, hardware purchased, licences, training or amount of testing (1)

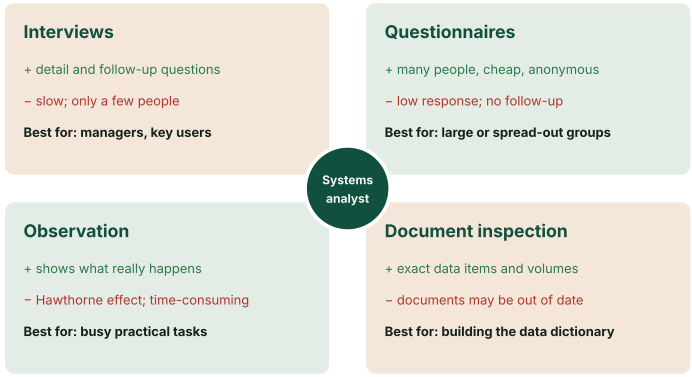
Analysis and fact-finding methods

AP03 FACT FILE

Specification: students should be able to

- Describe the purpose of the analysis (investigation) stage of systems development
- Explain the difference between direct and indirect data sources and state a benefit and drawback of each
- Describe the four main fact-finding techniques: interviews, questionnaires, observation and document sampling
- Evaluate the strengths and weaknesses of each fact-finding technique
- Select and justify suitable fact-finding techniques for a given scenario

Key words: analysis, systems analyst, fact-finding, direct data source, indirect data source, interview, structured interview, unstructured interview, questionnaire, closed question, open question, observation, Hawthorne effect, document sampling, triangulation, representative sample, quantitative data, qualitative data



The four fact-finding methods with an advantage, a disadvantage and the users each suits, around the systems analyst

Why analysis matters

Analysis, sometimes called the investigation stage, is where a systems analyst finds out exactly how an organisation's existing system works and what the new system must do. The analyst needs to understand the current system in terms of its inputs, the processing carried out on them, the outputs produced and the data that is stored. Just as importantly, the analyst identifies the strengths of the current system, which should be kept, its weaknesses and problems, which the new system must solve, and opportunities to add useful new features. Everything learned is used to agree the user requirements with the client before any design work begins. This matters because an error in understanding at this point is copied into the design, the code and the testing, and it becomes far more expensive to put right later. Because nobody can learn everything about an organisation from one conversation, the analyst gathers evidence using a range of fact-finding techniques: interviews, questionnaires, observation and document sampling. The findings are written up in a system specification that the client approves. A well-run analysis stage therefore reduces the risk of building a system that works technically but does not do what the users actually need.

Direct and indirect data sources

Data used to investigate a system or a business decision can come from direct or indirect sources. A direct data source is one created specifically for the current purpose, so the analyst collects the data first-hand without relying on anyone else, for example by writing and distributing a survey. Its great benefit is that the questions can be targeted at exactly the information needed. Its drawback is cost: time must be spent deciding what to ask, designing the questions, collecting responses and analysing them. An indirect data source already exists and was collected for a different reason, but is reused for the current purpose. Examples include government population statistics, a council's licensing records or an organisation's own sales history. Because the data has already been gathered and paid for, it is cheap and usually available straight away. However, it was not designed for the new purpose, so it may not contain precisely the data required, may be out of date or may need filtering and processing to remove irrelevant records. Although CCEA does not name this distinction in the specification outcomes, it is a useful way of thinking about evidence and it appears in textbook questions.

Interviews

Interviews are valuable because the staff who use a system every day usually know more about how it really works than anyone else. They are used to confirm the analyst's understanding, to fill gaps in it and to explore in depth an area where the interviewee is an expert. Good interviews need careful planning. The analyst decides precisely what must be found out, chooses interviewees carefully because interviewing everyone is neither possible nor necessary, and tells each person why they were selected, what the interview is for, how long it will take and how the information will be used. Some staff may fear that a new system threatens their job, so the analyst must be sensitive and reassuring. Interviews may be structured, with fixed questions that make comparison easy, or unstructured, allowing follow-up questions when something interesting emerges. Many analysts prefer to take notes and paraphrase answers back rather than record, since recording can make people guarded. The main strengths are depth of detail and flexibility; the main weaknesses are the time involved, the small number of people covered and possible bias, so answers should be cross-checked against other evidence.

Questionnaires

A questionnaire puts the same set of questions to many people and is the most practical method when the users are numerous or spread across several locations. Most are now completed online, which means respondents can answer privately, anonymously and at a time that suits them, and the software can total and chart the responses automatically. Typical input types are radio buttons, where only one option can be selected; checkboxes, where several options may be ticked; sliders or rating scales for a value on a range; and free text boxes for comments. Closed questions produce quantitative data that is quick to analyse, while open questions give qualitative data that is richer but slower to process. A well-designed questionnaire can include consistency checks, such as asking for age and later for date of birth, so that careless or untruthful responses can be filtered out. The disadvantages are significant. Response rates are often low, the analyst cannot ask a follow-up question when an answer is unclear, and once the questionnaire has been issued a confusing or leading question cannot be fixed. Writing clear, unbiased questions therefore takes time and should be tested on a small group first.

Observation

Observation means watching the system being used in its real environment, often by shadowing a representative sample of staff through their working day. It allows the analyst to record each process, the data needed to carry it out, the data it produces and how long it takes. It is especially useful because people do not always work in the way the official procedures describe. Staff often develop their own shortcuts and workarounds, some of which are more efficient than the documented method and should be kept in the new system, while others reveal genuine problems. The main weakness is the Hawthorne effect: people who know they are being watched may change their behaviour, for example by taking more care or following rules they normally skip, so the analyst sees an unrepresentative picture. This can be reduced by observing as unobtrusively as possible, watching several people and observing over a longer period. Observation is also time-consuming, may miss tasks that happen rarely, such as end-of-year processing, and is harder to use where similar work is done in slightly different ways at several sites, because local practices at one site may not apply elsewhere.

Document sampling and combining methods

Document sampling, also called document inspection, involves examining a sample of the paperwork and files the current system uses and produces, such as order forms, invoices, receipts, reports and procedure manuals. It is often a sensible first step because it shows the analyst who the key users are, which data items are recorded and in what format, how data moves between departments, the volume of data handled and the outputs the organisation relies on. It may also show how the system came about and how it has evolved. It is objective and does not interrupt staff, but it can be slow, and documentation may be incomplete or out of date. It also shows what is recorded rather than how or why people use it. Because every technique has weaknesses, analysts combine them and use triangulation, checking a fact from one source against another. For instance, a manager may claim a form is rarely used, but a sample of files may show hundreds a month. Such contradictions are worth investigating, as they frequently point to the real problems the new system must solve. The choice of methods depends on the number and location of users, the depth required, the time and budget available and the sensitivity of the topic.

T Key terms from the mark schemes

Analysis
The stage of systems development in which the existing system is investigated and the requirements for its replacement are identified and agreed with the client.
`Credited: Investigation`

Fact-finding
Gathering information about a current system, using techniques such as interviews, questionnaires, observation and document sampling, to establish how it works and what is required.

Systems analyst
The person who investigates the current system, establishes the requirements and specifies the new system.

Direct data source
A source of data collected first-hand specifically for the current purpose, such as a survey designed for the project.
`Credited: Primary data source`

Indirect data source
Data that already exists and is reused for a purpose other than the one for which it was originally collected.
`Credited: Secondary data source`

Hawthorne effect
The tendency of people to change their behaviour because they know they are being observed, which can make observation unreliable.
`Credited: Users act up when watched`

Closed question
A question with a fixed set of possible answers, producing quantitative data that is easy to analyse.

Open question
A question that lets the respondent answer in their own words, giving richer qualitative data that is harder to analyse.

Document sampling
Examining a sample of existing forms, invoices, reports and records to identify the data items, formats, volumes and outputs of the current system.
`Credited: Document inspection`

Triangulation
Checking a finding from one source of evidence against another to confirm that it is accurate and consistent.
`Credited: Cross-checking`

Exam focus

Short questions ask you to state or name techniques, give one benefit and one drawback, or explain why fact-finding is needed. For the purpose of analysis, mark schemes credit understanding the existing system's inputs, processes and outputs, identifying its strengths and weaknesses, and finding opportunities for improvement. When describing a technique, give its main features as well as an evaluation point: for observation, mention shadowing a representative sample and identifying the processes and data involved, then name the Hawthorne effect as a drawback. Avoid one-word evaluations such as 'quick' or 'cheap' unless you say compared with what and why. Scenario questions worth three or more marks expect you to choose a technique, say who it would be used with and justify it using details from the scenario, such as the number of users, their location or the type of information needed. In a 6 to 8 mark discussion, cover several techniques, give strengths and weaknesses of each and reach a reasoned conclusion about the best combination. Common mistakes are confusing direct and indirect sources, describing a questionnaire as allowing follow-up questions, and forgetting that interviews are time-consuming and only reach a few people.

? Check yourself

1. Before designing a new ticketing system for a Derry theatre, the systems analyst spends three weeks investigating how the box office currently works. State three things the analyst is trying to achieve during this investigation. [3]
2. A sports shop in Bangor is deciding whether to stock a new range of hurling equipment. Using this example, explain the difference between a direct and an indirect data source. [2]
3. The owner of a café in Portrush is considering opening a second café in Portstewart. Identify one direct and one indirect data source she could use, and state one drawback of each. [4]

Answers: 1. Any three from: to understand the existing system's inputs, processes and outputs (1) / to identify strengths of the current system to keep (1) / to identify weaknesses or problems the new system must solve (1) / to identify opportunities for improvement or new features (1) / to establish and agree the user requirements (1) / to provide the information needed for the system specification (1) 2. A direct data source is collected first-hand specifically for the current purpose, e.g. a survey of the shop's customers about hurling equipment (1) / an indirect data source already exists, was gathered for some other reason and is reused here, e.g. local GAA club membership figures or the shop's past sales records (1) 3. Direct, e.g. a survey/questionnaire of people in Portstewart, interviews with local residents or a trial pop-up stall (1); drawback: expensive/time-consuming to design, carry out and analyse (1). Indirect, e.g. NISRA population statistics, council business listings, sales data from the Portrush café (1); drawback: collected for another purpose so may not contain exactly the data needed / may need filtering / may be out of date (1)

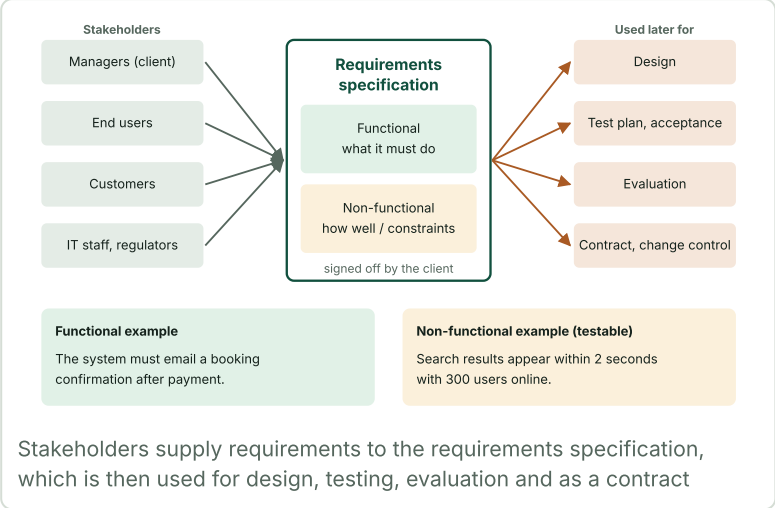
Stakeholders, requirements and the system specification

AP04 FACT FILE

Specification: students should be able to

- Identify the stakeholders in a systems development project and describe their differing needs
- Distinguish between functional and non-functional user requirements
- Identify functional and non-functional requirements for systems such as stock control, reservation, payroll and billing
- Describe the purpose and contents of the system (requirements) specification produced at the end of analysis
- Explain how the role of the specification has changed with the move towards more flexible, Agile approaches

Key words: stakeholder, client, end user, user requirements, functional requirement, non-functional requirement, quality attribute, constraint, system specification, requirements specification, performance, reliability, availability, security, usability, testable, MoSCoW, Agile



Stakeholders supply requirements to the requirements specification, which is then used for design, testing, evaluation and as a contract

Requirements: the bridge from analysis to design

Once fact-finding is complete, the systems analyst turns the evidence into a clear statement of user requirements: everything the new system must do and the standards it must meet. These requirements are agreed with the client and recorded in the system specification, which becomes the reference point for the rest of the project. Designers use it to plan the system, programmers build to it, testers check against it and the final evaluation measures the finished product against it. Requirements are divided into two kinds. Functional requirements state what the system must do, for example recording a booking, calculating a payslip or printing an invoice. Non-functional requirements state the conditions under which it must do those things, such as how quickly, how reliably, how securely and for how many users at once. A useful test is to ask whether the statement describes an action the system carries out or a quality by which that action is judged. Requirements must also reflect the needs of the stakeholders, and different stakeholders frequently want different things, so the analyst has to identify each group's needs, resolve conflicts with the client and write each requirement so that it can later be tested.

Stakeholders

A stakeholder is any person or group who has an interest in a system or is affected by it. Internal stakeholders include the client, usually senior managers who commission the system and pay for it; the end users who operate it every day; and the IT staff who will install, support and maintain it. External stakeholders include customers, suppliers, shareholders and regulators. Regulators matter because systems handling personal data must comply with the UK GDPR and the Data Protection Act 2018, overseen by the Information Commission (formerly the Information Commissioner's Office, ICO), while a payroll system must meet HMRC rules. Stakeholders' priorities often conflict. A manager might want detailed reports and tight security, while front-line users want a quick, simple screen with as few log-ins as possible, and customers want convenience and privacy. The analyst must identify these groups during fact-finding, record their requirements, and help the client decide priorities. Involving end users early has a further benefit: people who feel consulted are more likely to accept the new system, while staff who feel it has been imposed on them may resist it or use it badly, which can undermine even a technically excellent solution.

Functional requirements

A functional requirement describes something the system must do or allow its users to do. Functional requirements cover inputs, processing, outputs and stored data, and are usually written in a form such as 'The system must allow authorised staff to add and edit customer records' or 'The system must produce a monthly report of unpaid accounts'. For a stock control system, typical functional requirements are keeping track of current stock levels including items already on order, recording when each batch arrived so that stock can be rotated on a first-in, first-out basis, alerting staff when an item falls to its reorder level and producing management reports. For a payroll system they include adding and removing employees, entering hours worked, changing pay rates, calculating gross pay, deductions and net pay, and producing payslips. A billing system must calculate charges from usage and tariffs, add VAT, issue bills and record payments. A reservation system must show availability, take bookings and payments, and send confirmations. Note that backing up data can itself be a functional requirement, because the system must provide a backup facility, while how often backups happen is a non-functional condition.

Non-functional requirements

Non-functional requirements describe the conditions that the system's behaviour must satisfy rather than the behaviour itself. They are sometimes called quality attributes because they provide the criteria by which the system is judged. The main categories are performance, including response time, throughput and use of processor and memory; availability, meaning the hours or percentage of time the system can be used; reliability, meaning how rarely it fails and how accurately it works; security, covering who has access, who has priority, password rules and the frequency of backups; capacity, such as the number of simultaneous users; usability and accessibility; maintainability; and legal compliance. For example, a requirement that changes to shared data must be visible to every user within two seconds is non-functional, because it does not add a new function but sets a standard for an existing one. Good non-functional requirements are measurable. 'The website should be fast' cannot be tested, whereas 'search results must appear within two seconds with 500 users logged in' can. Weak answers in assessments often state a vague quality without any measure or link to the scenario.

Requirements depend on perspective

The same system can be described quite differently depending on whose requirements are being considered. Take an online ferry booking system. From the ferry company's point of view, functional requirements might include presenting sailings that match the customer's choices, monitoring how full each sailing is, raising fares automatically as capacity is approached and keeping a booking history for registered members. Its non-functional requirements would include restricting management screens to authorised staff, being available at all times, being backed up daily and coping with many simultaneous users without selling the same space twice. From the customer's point of view, the functional requirements are things like searching for sailings, choosing a cabin, paying by card, receiving an email confirmation and amending a booking online, while non-functional requirements include fast confirmation, keeping card details secure and working well on a mobile phone. When a question specifies a perspective, answers must match it: listing what the customer wants when the question asks for the organisation's requirements will not earn marks. Thinking about each stakeholder in turn is also a useful way to generate a complete set of requirements during analysis.

The system specification

The system specification, also called the requirements specification, is the main document produced at the end of analysis. It states precisely what the client expects the software to do and sets out a schedule for producing it. Typical contents are the scope and objectives of the project, a description of the current system and its problems, the stakeholders, the functional and non-functional requirements, the data requirements, data flow diagrams, and constraints such as budget, deadline and hardware. Historically, the specification formed the basis of a legally binding contract between the organisation buying the system and the supplier. The agreement worked both ways: the client could not later demand extra or different features, while the supplier was obliged to deliver everything specified, and changes led to penalties or revised charges. This suited an era with few software suppliers, who could plan their workforce around fixed deliverables. Today there are many more suppliers and development skills are widespread, and Agile approaches deliver software in short iterations with frequent client feedback. As a result, modern specifications tend to be less prescriptive and include agreed processes for changing requirements, while still giving both sides a clear shared understanding.

Key terms from the mark schemes

Stakeholder
A person or group with an interest in a system or who is affected by it, such as managers, end users, customers and regulators.

End user
A person who operates the system directly in their day-to-day work or as a customer.
`Credited: User`

Functional requirement
A statement of something the system must do, such as an input, process, calculation or output.
`Credited: What the system must do`

Non-functional requirement
A condition or constraint on how the system behaves, such as response time, availability, reliability, security or usability, rather than a function it performs.
`Credited: Additional conditions the system should meet`

Quality attribute
A measurable characteristic used to judge a system, such as performance, reliability or security; non-functional requirements specify these.

System specification
The document produced at the end of analysis that states precisely what the client expects the system to do, including requirements, constraints and schedule, and is agreed by the client.
`Credited: Requirements specification`

MoSCoW
A method of prioritising requirements as Must have, Should have, Could have and Won't have this time.

Constraint
A limit within which the system must be developed or operate, such as budget, deadline, hardware or legislation.

Testable requirement
A requirement stated precisely enough, usually with a measure, that it can be shown to have been met or not.
`Credited: Measurable requirement`

Exam focus

Expect a two-mark question asking you to distinguish the two kinds of requirement: one mark for explaining that a functional requirement describes an action the system has to carry out, and one for saying a non-functional requirement is a condition, constraint or quality the system must meet, such as performance or security. Scenario questions then ask for two or three examples of each for a named system such as payroll, stock control, billing or reservations. Each example must clearly belong to that system, so 'add and edit employee details' suits payroll, whereas 'be secure' on its own is too vague; 'only payroll staff can view salary details' is much better. Begin functional requirements with 'The system must allow...' or 'The system must produce...', and give non-functional requirements a category and, where possible, a measure. Read carefully whether the question asks for the organisation's or the customer's perspective. If asked to name the document produced at the end of analysis, the answer is the system (requirements) specification. When explaining its importance, mention that it records what was agreed, guides design, provides criteria for testing and evaluation, and helps avoid disputes. Avoid giving hardware items as requirements.

? Check yourself

1. Define the term stakeholder and give two examples of stakeholders in a new school management information system. [3]
2. A requirements list for a Strabane veterinary practice includes 'Record each animal's vaccinations' and 'Show any pet's record within two seconds'. Using these as examples, explain how a functional requirement differs from a non-functional requirement. [2]
3. A hardware merchant in Newry is replacing its paper-based stock control system. State two functional and two non-functional requirements of the new system. [4]

Answers: 1. Definition: a person or group with an interest in, or affected by, the system (1). Any two examples (1 for both): teachers / pupils / parents / senior leadership / office staff / IT technician / Education Authority / Department of Education (regulator). 2. A functional requirement describes an action the system must carry out / a function, input, process or output, e.g. recording vaccinations (1) / a non-functional requirement describes a condition, constraint or quality the system must meet, such as performance, e.g. displaying a record within two seconds (1) 3. Functional, any two (1 each): record/track current stock levels including items on order / alert staff when stock reaches reorder level / record goods received and sold / produce stock reports / generate purchase orders / support FIFO stock rotation. Non-functional, any two (1 each): stock levels must be accurate/up to date / available during trading hours / accessible only to authorised staff / backed up daily / support multiple simultaneous users / respond within a stated time

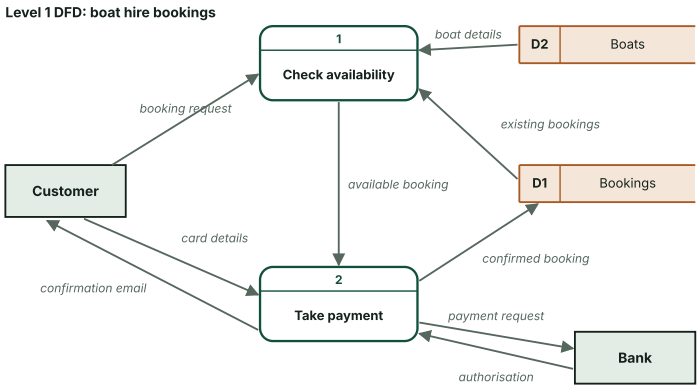
Data flow diagrams

AP05
FACT FILE

Specification: students should be able to

- Explain the purpose of a data flow diagram (DFD) and its advantages
- Identify and use the standard DFD elements: external entities (sources and sinks), processes, data stores and data flows
- Produce a context diagram (Level 0 DFD) for a simple scenario
- Produce a Level 1 DFD by exploding a context diagram, maintaining consistency between levels
- Interpret DFDs and identify and correct errors using the rules for valid DFDs

Key words: data flow diagram (DFD), context diagram, Level 0, Level 1, external entity, source, sink, process, data store, data flow, system boundary, granularity, explode, decomposition, balancing, duplicated entity



A level 1 data flow diagram for a boat hire booking system with external entities, numbered processes, data stores and labelled data flows

Modelling a system with data flows

A data flow diagram, or DFD, is a graphical model showing how data moves through a system. The idea is simple: data arrives from outside the system, processes act on it, some of it is stored for later, and new data flows back out to people or organisations. DFDs are widely used during analysis because they capture the analyst's understanding of the existing system, and of the proposed one, in a form that is easy to check. They rely on only a handful of symbols, so a client or end user with little technical knowledge can follow the diagram and point out anything that is missing or wrong. That builds an agreed, common understanding between the analyst, the client and the users before design begins. DFDs can be drawn at different levels of detail, called granularity: a coarse, high-level overview called a context diagram or Level 0 DFD, and more detailed Level 1 diagrams that show the main sub-processes and data stores. Carefully drawn DFDs can also reveal problems in the current system, such as bottlenecks where too much data passes through one process, unnecessary duplication, or data that is stored but never used. A DFD shows what happens to data, not the order of steps or the decisions made.

The elements of a DFD

A DFD is built from a small set of elements. An external entity is a person, organisation or other system outside the system boundary that supplies data to the system, when it is called a source, or receives data from it, when it is called a sink. Many entities, such as a customer who sends an order and receives an invoice, are both. In the notation used in the CCEA textbook, external entities are drawn as ellipses; if the same entity has to appear twice to avoid crossing lines, it is drawn again with a short diagonal line to show that it is a duplicate. A process transforms incoming data into outgoing data and is drawn as a rectangle with a strip at the top holding its number, such as 1.0 or 2.0, and is named with a verb phrase such as 'Calculate invoice'. A data store, drawn as an open-ended rectangle with an identifier such as D1, holds data for later use, for example a stock file or a customer database table. A data flow is an arrow, labelled with the data it carries, showing the direction of movement. Other textbooks use squares for entities and rounded rectangles for processes; either notation is acceptable if it is applied consistently.

Context diagrams

The context diagram, also known as a Level 0 DFD, is the highest-level and least detailed view of a system. Its main job is to show the boundary of the system: what lies inside it and what lies outside. The entire system is represented as one single process, often numbered 0, in the centre of the diagram. Around it are all the external entities with which the system exchanges data, connected by labelled data flows that show what each entity sends to the system and receives from it. Detailed internal matters, such as checking a customer's credit rating or updating individual files, are deliberately left out at this level. Data stores are normally omitted, or used very sparingly, because the purpose is to define scope rather than to show how the system works internally. Because it gives such a clear overview, a context diagram is particularly useful for managers who need to see what the system will cover and which parts of the organisation it will interact with. Drawing it also forces the analyst and client to agree the scope: for example, whether purchasing goods from suppliers or carrying out marketing is part of a sales system, or outside it and therefore an external entity or ignored entirely.

Level 1 DFDs and balancing

A Level 1 DFD is produced by exploding, or decomposing, the single process in the context diagram into the main sub-processes that make up the system. Each sub-process is numbered, typically 1.0, 2.0, 3.0, and given a verb-phrase name. The data stores that these processes use are added, along with the internal data flows between processes and between processes and stores. The external entities do not change, because they are outside the system and are not being explored in more detail. The crucial rule linking the two levels is consistency, often called balancing. Every data flow that enters or leaves the system on the context diagram must also appear on the Level 1 diagram, connected to whichever sub-process now handles it, and with the same label. Nothing should cross the system boundary at Level 1 that was not shown at Level 0. If a context diagram shows a two-way exchange between a customer and the system, the Level 1 diagram must still show both flows. The same idea continues downwards: a Level 1 process can be exploded into a Level 2 diagram with processes numbered 2.1, 2.2 and so on, although CCEA AS candidates only need to produce context and Level 1 diagrams.

Rules for valid DFDs

Several rules make a DFD valid, and many exam questions ask you to apply them. First, data must always pass through a process: a flow cannot go directly from one external entity to another, from one data store to another, or between an external entity and a data store. If a customer's order needs to be stored, a process such as 'Record order' must sit between the customer and the orders file. Second, every process must have at least one input and at least one output, because its job is to transform data; a process with only inputs is sometimes called a black hole, and one with only outputs is a miracle. Third, every data store should be written to by at least one process and read by at least one process somewhere in the system, otherwise it serves no purpose. Fourth, every data flow must be labelled with a noun phrase that names the data it carries, such as 'Payment details', rather than an action. Processes are named with verb phrases, and both processes and data stores are given unique identifiers. Finally, levels must balance. Checking a finished diagram against each of these rules in turn is a reliable way to find mistakes before submitting it.

Drawing a DFD from a scenario

Turning a written description into a DFD is a skill that improves with a methodical approach. Begin by reading the scenario twice. Underline the people, departments and organisations that send data into the system or receive data from it; these are your candidate external entities. Then look for verbs describing actions on data, such as receives, checks, calculates, records or sends, which suggest processes, and for nouns describing records that are kept, such as a stock file or member list, which suggest data stores. Draw the context diagram first, showing one process for the whole system, all the entities and every labelled flow between them. Next, explode that process into two to four main sub-processes, decide which stores each one reads or writes, and add the internal flows. Finally, check the Level 1 diagram against the context diagram for balancing and against each validity rule. Remember that departments inside the organisation being modelled may be processes rather than entities, depending on where the boundary is drawn, so read carefully what the question says the system covers. Using the scenario's own names for entities and data makes the diagram easier to mark and avoids ambiguity.

T Key terms from the mark schemes

Data flow diagram (DFD)
A diagram that models how data moves through a system using external entities, processes, data stores and labelled data flows.

Context diagram
A top-level (Level 0) DFD showing the whole system as a single process with its external entities and the data flows between them, defining the system boundary.
`Credited: Level 0 DFD`

Level 1 DFD
A DFD in which the single process of the context diagram is exploded into its main numbered sub-processes, with data stores and internal flows added.

External entity
A person, organisation or system outside the system boundary that is a source or destination of data.
`Credited: Source/sink; terminator`

Source
An external entity from which data enters the system.

Sink
An external entity to which the system sends data.
`Credited: Destination`

Process
An activity in a DFD that transforms input data into output data; named with a verb phrase and numbered.

Data store
A place in a DFD where data is held for later use, such as a file or database table.
`Credited: File`

Data flow
A labelled arrow in a DFD showing data moving between entities, processes and data stores.

Balancing
The rule that the data flows into and out of a process (or the whole system) must be the same on its exploded lower-level DFD.
`Credited: Consistency between levels`

Explode
To break a DFD process down into more detailed sub-processes on a lower-level diagram.
`Credited: Decompose`

Exam focus

DFD questions range from one-mark identification to drawing a full diagram. When asked for the purpose of a DFD, mark schemes credit points such as providing a visual representation of the system, showing it at different levels of detail, showing how the system interacts with external entities and identifying the processes and data stores. You may be shown a diagram with parts labelled A to D and asked to identify the external entity, data flow, process and data store, so learn the shapes. When drawing a context diagram, marks are typically awarded for a single correctly named process, each correct external entity and each correctly labelled flow in the right direction; unlabelled arrows usually lose marks. For a Level 1 diagram, show numbered processes, the data stores and flows, and make sure it balances with the context diagram. If asked about the relationship between the two levels, explain that Level 1 explodes the single process into sub-processes, adds data stores and keeps the same external flows. When identifying an error, name the rule broken and give a specific correction. Common mistakes are drawing a flowchart with decisions, connecting entities directly to stores, and omitting flow labels.

? Check yourself

1. An analyst plans to show a Newtownabbey car dealership's managers a set of data flow diagrams of their proposed servicing system. Describe two ways the diagrams will be useful. [2]
2. In a DFD for a school library's loans system, explain how a source differs from a sink, giving an example of each from the library. [4]
3. A trainee analyst at a Lisburn software firm has drawn a context diagram for a car-wash booking app and is about to draw the Level 1 DFD. State two ways in which the Level 1 diagram will differ from the context diagram. [2]

Answers: 1. Any two from: provides a visual/graphical representation of the system (1) / shows the system at different levels of detail (1) / shows how the system interacts with external entities / defines the boundary (1) / identifies the processes and data stores (1) / gives a shared understanding that users can check (1) / helps identify bottlenecks or problems in the current system (1) 2. A source lies outside the system and sends data into it (1), e.g. a pupil/borrower sending a loan request or book return (1) / a sink lies outside the system and is sent data by it (1), e.g. a pupil receiving an overdue reminder, or a librarian/teacher receiving a report (1) 3. Any two from: context diagram has a single process; Level 1 has several numbered sub-processes (1) / context diagram normally shows no data stores; Level 1 shows data stores (1) / Level 1 shows internal data flows between processes and stores; context shows only flows crossing the boundary (1) / context gives an overview for management; Level 1 gives more detail (1)

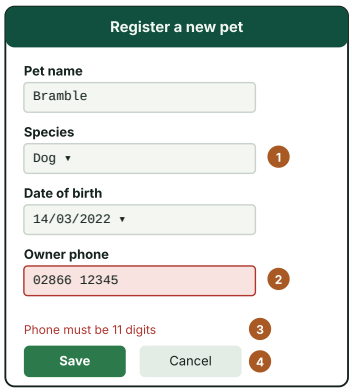
System design: storyboarding, prototyping and data design

AP06 FACT FILE

Specification: students should be able to

- Describe the purpose of systems design, development and testing and the contents of a design document
- Describe the use of storyboarding in developing the user interface
- Describe the use of prototyping in the design of the user interface and the data model
- Describe the design of inputs, outputs and user interfaces, applying principles of human–computer interaction (HCI)
- Describe the design of data structures and the use of a data dictionary
- Explain the role of validation and verification in design and describe common validation checks

Key words: design document, storyboarding, navigation, prototyping, look and feel, stub, data model, human–computer interaction (HCI), input design, output design, data dictionary, data structure, validation, verification, range check, format check, consistency check



HCI features

1 Drop-down list prevents typing errors
2 Field highlighted where the error is
3 Clear, helpful error message (feedback)
4 Buttons in the same place on every screen

Data dictionary (extract)

Field	Type	Validation
PetName	String(30)	Presence
Species	String(10)	Lookup list
DateOfBirth	Date	Range: not future
OwnerPhone	String(11)	Length, format

An annotated input screen design for registering a pet, with numbered HCI features, beside part of its data dictionary

Overview: from requirements to a design

Systems design is the creative stage that follows analysis. Because the designer has to understand exactly what the system is for, design can only begin once the user requirements have been agreed, and the finished design must meet every one of those requirements. The result is a design document. It sets out the user interface and navigation between screens, the inputs and how they will be captured, the outputs and reports, the data structures and data dictionary, the validation and verification each data item needs, algorithms for the main processes and an initial test plan. Development follows: programmers build the system from this document, usually as a set of separate modules that are tested on their own and again as they are joined together. Two techniques are used heavily during design to involve users before money is spent on coding. Storyboarding sketches each screen and the routes between them, and prototyping builds an early working model that users can try. Both aim to find problems while they are still cheap to fix.

Storyboarding

Storyboarding comes from the film industry, where a story was drawn as a sequence of sketches so that the crew could picture each scene and spot problems before filming. In system design each frame of the storyboard represents a screen or page. The designer records what each screen contains, identifies every input and output on it, and draws arrows to show navigation from one screen to another. The storyboard is then talked through with users, who can say where the route is too long, where a feature is missing or where a screen is confusing. Its strengths are that it is quick and cheap to produce, needs no technical knowledge to understand, encourages communication between designers and users early in the project and allows changes to be made before any code exists. Its weaknesses are that it is not interactive, shows only limited detail and cannot convey how fast or responsive the finished system will feel, so it cannot fully judge usability. Storyboards are therefore often followed by a prototype.

Prototyping the interface and data model

A prototype is an early, deliberately incomplete version of the software that gives users the look and feel of the proposed system. It may contain stubs, which are menu options or buttons that appear but do nothing yet, and it often has little error checking, so it may only work when given perfectly correct data. For the user interface, a prototype lets users compare menu arrangements, colour schemes and screen layouts, and lets the client and developer confirm that the system will meet the agreed requirements. For the data model, building a prototype forces the designer to decide what data must be input, what outputs are needed and what processing links them, which reveals the tables and relationships the database requires. Prototyping improves the quality of the requirements, finds problems and enhancements earlier and reduces misunderstandings because users can react to something real. However, it can be costly and time-consuming, users may be swayed by appearance rather than accuracy or security, an early version may be accepted too readily, and users may expect the finished system far sooner than is realistic.

HCI, input and output design

Input design decides what data is captured, how it is captured and how screens or forms are laid out. Good input design reduces typing and errors by using drop-down lists, option buttons, date pickers and sensible default values. Output design decides the content, layout and medium of screens, printed reports and documents, matched to each audience: a manager may need a summary chart, while a clerk needs a detailed list. Human–computer interaction principles guide these decisions. Consistency means the same layout, colours and wording on every screen, so users learn the system quickly. Feedback means confirming actions and giving clear error messages that say how to put the problem right. Error prevention restricts input and asks for confirmation before destructive actions such as deleting. A low memory load means showing choices rather than expecting users to remember codes. Accessibility covers contrast, scalable text, keyboard navigation and labels for screen readers, and avoids using colour alone to carry meaning. Finally, the interface should suit its users, offering shortcuts for experts and guidance for novices.

Data structures and the data dictionary

Designers must decide how data will be held: as database tables with keys and relationships, as files of records, or as arrays and lists inside a program. These decisions are recorded in a data dictionary, a central description of every data item in the system. For each field it typically gives the name, data type, length or size, format, a description, an example value and the validation rule, and it notes primary and foreign keys and the relationships between tables. The data dictionary keeps a development team consistent, because every programmer uses the same names, types and rules; without it, one module might store a date as text and another as a date value, causing faults when modules are joined. It also gives testers the exact limits to test and helps maintenance staff understand the data years later. A validation rule in a data dictionary should be precise, for example a range check of 1.00 to 50.00 pounds, rather than simply naming the type of check.

Validation and verification

Validation is an automatic check carried out by the software to make sure data is sensible and obeys set rules before it is accepted. A presence check ensures a field is not left blank; a type check ensures, for example, that a number has been entered; a range check tests that a value lies between limits; a length check limits the number of characters; a format check matches a pattern such as a postcode; a lookup check compares the entry with a list of allowed values; a check digit detects errors in codes such as barcodes; and a consistency or cross-field check compares fields, such as making sure a return date is after a loan date. Validation cannot prove that data is correct: a pupil whose balance is really 12.60 pounds might be entered as 16.20, which is still within range. Verification checks that data has been entered or transferred accurately, for instance by double entry of a password or email address, or by proof-reading the screen against the original document. Both are specified during design.

T Key terms from the mark schemes

Design document
The output of the design phase, setting out the user interface, inputs, outputs and reports, data structures, validation and verification, algorithms and an initial test plan for the new system.
`Credited: design specification`

Storyboarding
A design technique in which each screen of a proposed interface is sketched, showing its contents, inputs and outputs and the navigation between screens, to obtain early feedback from users.

Prototyping
Building an early, incomplete working version of a system so that users can experience its look and feel and give feedback on the interface and data requirements.
Credited: prototype

Data dictionary
A central record of every data item in a system, giving its name, data type, length, format, description, example and validation rules.

Verification
A check that data has been entered or transferred accurately, such as double entry or proof-reading against the source.

Consistency check
A validation check that compares two or more fields to make sure their values are compatible, e.g. return date after loan date.
Credited: cross-field check

Stub
A part of a prototype or program, such as a menu option or subroutine, that is present but has no working functionality yet.
Credited: placeholder

Validation
An automatic check by the system that data is reasonable and follows set rules before it is accepted; it cannot ensure the data is correct.

Human–computer interaction (HCI)
The study and design of how people interact with computer systems so that interfaces are usable, efficient and accessible.

Design specification
The deliverable of the design stage describing the inputs, outputs, interfaces, data structures, processes, security and test plan of the new system.
Credited: design document

C C# example

Prototyping the interface and the data model

```
// Prototype main menu: only option 1 is built
Console.WriteLine("1. View today's menu");
Console.WriteLine("2. Order history");
Console.WriteLine("3. Top up balance");
string choice = Console.ReadLine();
if (choice == "1")
    Console.WriteLine("Chicken curry £2.40 | Pasta bake £2.10");
else
    Console.WriteLine("Coming soon"); // stub
```

Exam focus

Short questions often ask you to list contents of a design document; give specific items such as interface design, data structures, validation and verification required, report design and a test plan. For storyboarding, describe what a storyboard shows (contents of each screen, inputs and outputs, navigation) and why it helps (early user feedback, no specialist knowledge needed). Prototyping questions are often in two parts: explain its use for the interface and for the data model separately, then give advantages and disadvantages. Each point usually earns one mark, so two advantages and two disadvantages make four marks, and they must be different ideas, not the same point reversed. For validation, name the check and state the exact rule for that field, for example a range check that the age is between 11 and 19 inclusive; a bare name such as range check may only earn half the credit. Many candidates confuse validation with verification, so learn one clear example of each. In design questions, justify each feature by saying how it helps the user, linking it to an HCI principle.

? Check yourself

1. A Craigavon software company is designing a new booking system for a driving school. State four items that should be included in its design document. [4]
2. Describe how storyboarding could be used when designing the screens of a mobile app for booking pitches at a GAA club. [3]
3. Give two advantages and two disadvantages of using storyboarding to design a user interface. [4]

Answers: 1. Any four from: design of the user interface/screens and navigation (1) / input design (1) / output and report design (1) / specification of data structures/data dictionary (1) / validation and verification required (1) / algorithms for processes (1) / security or access levels (1) / a test plan (1) 2. Each screen of the app (e.g. login, choose pitch, choose time, confirm) is sketched as a frame showing its contents (1) / the inputs and outputs on each screen are identified, e.g. date selected, available slots shown (1) / arrows show navigation between screens, and the storyboard is walked through with club members to get feedback/spot awkward routes before coding (1) 3. Advantages, two from: quick/cheap to create (1) / promotes early communication between designer and users (1) / no specialist knowledge needed to understand (1) / early feedback on functionality and navigation (1) / changes can be made early/cheaply (1). Disadvantages, two from: not interactive (1) / limited detail (1) / cannot show speed, responsiveness or usability accurately (1)

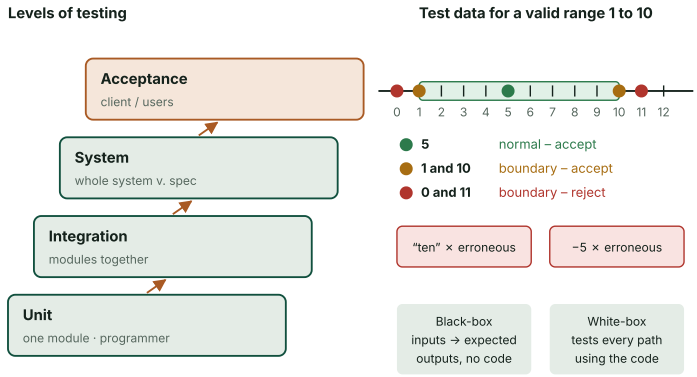
Testing and test plans

AP07 FACT FILE

Specification: students should be able to

- Explain the importance of testing throughout the development of a system in ensuring system quality
- Describe unit and integration testing, including black-box and white-box approaches
- Distinguish between system, acceptance, alpha and beta testing
- Describe the purpose of a test plan and its main components
- Distinguish between normal, boundary (extreme) and erroneous (invalid) test data and select examples for a given rule

Key words: unit testing, integration testing, integration error, system testing, alpha testing, beta testing, acceptance testing, sign-off, black-box testing, white-box testing, test plan, expected outcome, normal data, boundary data, erroneous data



Levels of testing from unit to acceptance, and normal, boundary and erroneous test data on a number line for a valid range of 1 to 10

Overview: testing throughout development

Testing is not a single event at the end of a project but an activity that runs through the whole of development. Software is built as a set of units, so each unit is tested on its own first and any errors are corrected. Units that work perfectly in isolation may still fail when they are joined, so integration testing follows. The complete system is then tested against its agreed specification, the developer carries out a final in-house alpha phase, selected external users may beta test a pre-release version, and finally the client carries out acceptance testing before signing off the system. Testing at every stage matters for quality because an error is far cheaper and easier to fix when it is found early, close to where it was made, than when it is discovered by the client or after the system has gone live. Testing is also planned rather than random: a test plan sets out every test, the data to use and the expected outcome, so that the most vulnerable areas, such as range limits and unusual paths through the code, are always checked. Even thorough testing cannot prove a program is free of errors.

Unit testing: black box and white box

Unit testing checks that each individual component works correctly in all circumstances. Done well, it gives confidence in each unit, which narrows the search when a fault appears later. Two approaches are used. In black-box testing the code is treated as if hidden inside a closed box: the tester knows only what the unit is meant to do, enters inputs and compares the actual outputs with the expected outputs taken from the specification. Testers therefore need no knowledge of the code and can be non-technical. In white-box testing, sometimes called glass-box testing, the tester can see the code and designs tests around its structure, making sure every branch, path and loop is exercised, and concentrating on the places where errors are most likely. For example, a water-billing module that uses one rate for households and two rates for businesses has three branches; black-box tests compare bills for a range of customers, while white-box tests deliberately include at least one case for each branch and cases either side of the 100 cubic metre switch-over point. Programmers often use dry runs and trace tables when white-box testing.

Integration and system testing

Integration testing takes place when units that have already passed their own tests are combined. Faults typically occur at the interfaces between units, because one unit passes data in a form the next does not expect. A classic case is one unit storing dates as day, month, year while another reads them as month, day, year: each unit works and displays sensible dates on its own, but together they misread dates such as the fourth of July as the seventh of April. Such integration errors only appear when the parts are brought together, which is why unit testing alone is never enough. When integration testing finds a fault, the unit concerned is modified, unit tested again and then re-integrated. Agreeing data formats early in a shared data dictionary reduces these problems.

System testing, sometimes called application testing, is then performed on the complete, integrated system to judge whether it complies with the agreed specification. It is essentially black-box testing applied to the whole system rather than one part, so the tester does not need technical knowledge of how it was built, and it is carried out by following a detailed test plan.

Alpha, beta and acceptance testing

Alpha testing is the final phase of in-house testing carried out by the software developer before the software leaves their control. It brings together unit, integration and system testing, follows a test plan and may use simulated or real data; small changes can still be made afterwards. Beta testing comes after alpha testing. A pre-release version is given to a group of potential users who use it in a realistic environment, on their own equipment and without the developers present, and report any faults they find. It can be thought of as external user acceptance testing and is especially common for software that will be sold to many customers or run on many different devices.

Acceptance testing is the final test, carried out once system testing is complete and bespoke software is ready for handover to the client. End users try it in their real working setting, with genuine data at realistic volumes, to gain confidence that it meets their requirements and works alongside the organisation's other systems. They report problems, and once these are resolved the users sign off the software, which formally completes the contract.

Test plans and their components

A test plan is a detailed document that a team of testers follows exactly. Its purpose is to make testing thorough and systematic, so that every requirement, every pathway through the system and every validation routine is checked, and the system's actual performance is compared with its design. For each test the plan records a test number, the part of the system being tested, what is being tested and why (for example, boundary data to check the lowest valid value is accepted), the exact test data and the expected outcome. These columns are completed by the person who designs the tests before any testing happens. The plan is then passed to the tester, who carries out each test and fills in the remaining columns: the actual outcome and PASS or FAIL. A test passes only when the actual outcome matches the expected outcome exactly. Failed tests are reported so the fault can be fixed, after which the tests are run again to make sure the fix works and has not broken anything else. A systems analyst may help by suggesting test data, particularly at the boundaries, and by checking results against the specification.

Choosing test data

Test data falls into three categories. Normal or valid data is the typical data the system expects; it is correct in type and format and should be accepted and processed. For a mark out of 100 stored as a whole number, values such as 43, 72 and 89 are normal. Boundary or extreme data is still valid but lies at the very edges of the allowed range, such as 0 and 100, because this is where programmers most often make off-by-one mistakes, for instance writing less than instead of less than or equal to. Where a range is divided into bands, every band edge is a boundary, so a grading scheme of 0–44, 45–64, 65–79 and 80–100 needs tests at 0, 44, 45, 64, 65, 79, 80 and 100.

Erroneous, invalid or exceptional data is wrong in type, format or range, such as 55.5, the letter X, the word sixty, a negative number or a value above the maximum. It should be rejected, ideally with a helpful message, rather than accepted or causing the program to crash. Testers often also try the values just outside each limit, which are invalid and should be rejected.

T Key terms from the mark schemes

Unit testing
Testing an individual module or subroutine on its own, usually in-house by the programmer, to check it works correctly in all circumstances.

Integration testing
Testing units that have already been tested once they are combined, to check that they interact correctly and pass data between them as expected.

System testing
Testing the complete, integrated system against its agreed specification; essentially black-box testing of the whole system.
Credited: application testing

Alpha testing
The final phase of in-house testing carried out by the developer, following a test plan, before the software is released to users.

Beta testing
Testing of a pre-release version by a group of potential external users in a realistic environment, without developers present, who report any faults.
Credited: external user acceptance testing

Acceptance testing
The final test, by end users in their real working environment with genuine data at realistic volumes, to confirm the system meets their requirements before they sign it off.
Credited: user acceptance testing

Black-box testing
Testing that checks outputs against expected results for given inputs without any access to or knowledge of the program code.
Credited: functional testing

White-box testing
Testing that uses knowledge of the program code to design tests for every path, branch and loop.
Credited: glass-box / structural testing

Test plan
A document listing every test to be carried out, giving the part of the system tested, what is tested, the test data and the expected outcome, with space for the actual outcome and pass/fail.

Normal data
Typical valid data, correct in type and format, that the system should accept and process.
Credited: valid data

Boundary data
Valid test data at the very edges of an allowed range, used to find errors in range limits.
Credited: extreme data

Erroneous data
Test data that is wrong in type, format or range and should be rejected by the system.
Credited: invalid / exceptional data

C C# example

Worked example: a water bill module

```
decimal Bill(decimal m3, bool business)
{
    if (!business)
        return m3 * 1.50m;          // branch 1
    else if (m3 <= 100)
        return m3 * 1.50m;          // branch 2
    else
        return m3 * 1.20m;          // branch 3
}
Console.WriteLine(Bill(10, false));  // 15.00
Console.WriteLine(Bill(100, true));  // 150.00
Console.WriteLine(Bill(101, true));  // 121.20
```

Exam focus

Questions on testing stages usually give one or two marks per stage, so state both what is tested and who tests it or where: unit testing is in-house by the developer on individual components; acceptance testing is by end users in a live environment with real data, ending in sign-off. Do not confuse system testing (the whole system, by the developer, against the specification) with acceptance testing (by the client, against their requirements). For alpha versus beta, contrast in-house developers with external potential users in a realistic environment. Test data questions are common and very precise. For a junior band of 4 to 12 whole years, boundary values are 4 and 12; normal is any integer strictly between; invalid examples include text or a decimal such as 7.5. When a range is open-ended, such as 13 or older, choose a reasonable upper limit and say so. Always give a specific expected outcome, such as an error message stating the valid range, never ‘it works’. A typical slip is to list 0 or 101 as boundary data that should be accepted when they lie outside the valid range.

? Check yourself

1. Explain why testing should take place throughout the development of a system rather than only at the end. [3]
2. Describe what happens during unit testing and integration testing. [4]
3. Explain why a system made of units that have all passed unit testing may still fail integration testing. Use an example. [3]

Answers: 1. Any three from: errors are cheaper/easier to fix when found early, near where they were made (1) / each unit can be checked before being combined, narrowing the search for faults (1) / integration errors only appear when units are joined so must be tested at that stage (1) / the system must be checked against requirements before handover (1) / confidence/quality is built up at each stage (1) 2. Unit: each individual module/component is tested on its own (1) / in-house by the developer/programmer to check it works in all circumstances, using black-box or white-box methods (1). Integration: tested units are combined (1) / to check they work together and data passes correctly between them/at their interfaces (1) 3. Units may expect data in different forms, so faults appear at the interfaces between them (1) / each unit's own tests would not reveal this (1) / example, e.g. one unit stores dates as dd/mm/yyyy and another reads mm/dd/yyyy, so dates are misread when combined (1)

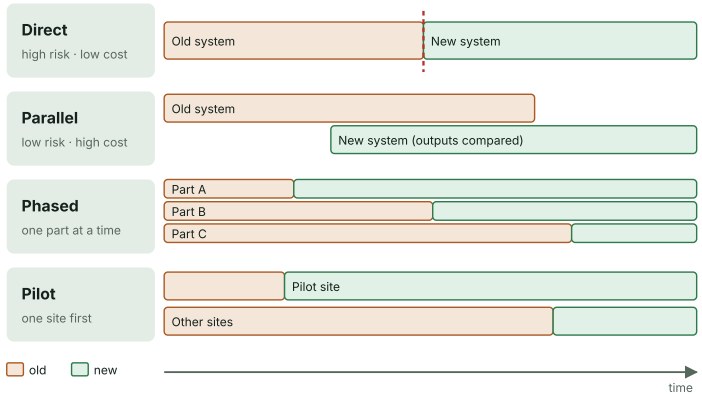
Implementation: changeover, data conversion and training

AP08 FACT FILE

Specification: students should be able to

- Describe the purpose of system implementation and the tasks it involves
- Describe and evaluate parallel, direct, pilot and phased changeover
- Recommend and justify a changeover method for a given scenario
- Explain what is meant by data conversion and describe what it involves
- Describe methods of training users and evaluate their suitability

Key words: implementation, changeover, parallel changeover, direct changeover, pilot changeover, phased changeover, big bang, data conversion, data migration, verification, installation, training, cascade training, e-learning, risk



Timelines comparing direct, parallel, phased and pilot changeover, showing when the old and new systems are in use

Overview: implementing a new system

Implementation is the stage at which a newly developed system is put into operation and replaces whatever was used before, whether an older computer system or a paper-based one. It has to be planned while the system is still being developed, because a lot has to be ready before the new system can take over completely: hardware and networks must be installed, software set up, staff trained and documentation provided. Existing data usually has to be converted into a form the new system can use. The organisation must then choose how to move from the old system to the new one. There are four changeover methods: parallel, where old and new run side by side; direct, where the old system stops and the new one starts on a set date; pilot, where the whole system is tried in one section first; and phased, where parts of the system are introduced one at a time. Each balances risk against cost and time differently, so the right choice depends on the organisation. After implementation, the system will need ongoing maintenance throughout its life.

Parallel and direct changeover

In parallel changeover the old and new systems operate together for a period. Every transaction is processed by both, and the two sets of results are checked against each other to confirm the new system gives correct output. When the new system has proved reliable, the old one is stopped. This is the safest method, because if the new system turns out to have flaws the old system is still there as a backup, and staff can become familiar with the new system gradually. However, it is expensive and time-consuming to keep two systems running, checking one against the other takes extra effort, and two separate copies of the data have to be kept, which may fall out of step. Direct changeover, sometimes called the big bang approach, switches off the old system on a particular date and introduces the new one immediately. There is no duplication of resources and everyone starts using the new system together. It is very risky, though: should the new system break down, going back to the old one may be difficult or impossible, and there may be a period when no system is available.

Pilot and phased changeover

Pilot changeover introduces the complete new system in one department, branch or section of the organisation first. The rest of the organisation keeps using the old system until the pilot has shown that the new one works, after which it is rolled out everywhere else. It suits organisations with several similar sections, such as a building society with many branches or a retailer with many shops. Only one section is affected if problems arise, and the staff in the pilot section gain experience that lets them help train others. Its drawbacks are that two systems run at the same time in different parts of the organisation, which needs extra resources, and it may be difficult to share data between sections using different systems. Phased changeover introduces the system one component at a time, for example payroll first, then invoicing, checking each part works before the next is switched on. Staff face change in stages, which can be less stressful, and a problem should affect only one part. However, the full changeover can take a long time, and requests to alter parts already introduced can delay the rest.

Choosing a changeover method

Because failure during changeover can be costly, organisations look for the lowest-risk method they can afford that is actually feasible. Where an organisation has several similar sites, pilot changeover is often a good balance, since it limits risk to one site without the full cost of parallel running. Where the consequences of failure would be severe, such as possible injury or major financial loss, parallel running may be justified despite its higher cost, because the old system remains as a safety net. Phased changeover suits a system made of distinct parts that can be introduced independently. Direct changeover carries the most risk and is used only when there is no realistic alternative, for example when the old and new systems cannot run at the same time, or for a small and non-critical system. A famous example of an unavoidable direct change was Sweden switching from driving on the left to driving on the right on a single morning in 1967: running both at once was impossible, so the change was planned in great detail months in advance. Direct changeover should be timed for a quiet period, with staff fully trained beforehand.

Data conversion

Data conversion is the task of transferring existing records into the structure and format the new system expects, so that it can be used fully from the start. When a computer system replaces a paper-based one, records such as forms, reports, orders and invoices must be turned into electronic data. This usually means creating the new database structure, employing staff to type in or scan the paper records, and then verifying and validating the data that has been entered. Verification might involve proof-reading entries against the original documents or keying data in twice, while validation rules reject impossible values. The volume of work can be large, particularly if scanned images of documents are to be stored. Where the new system replaces an older computer system, transferring the data, sometimes called data migration, is normally quicker, but fields may need reformatting, for example changing date formats or splitting a full name into first name and surname, and specialised or bespoke conversion software may be required. Poorly converted data causes errors and undermines users' confidence in the new system.

Training users

Users need training so that they can use the new system confidently and are more likely to accept it rather than resist it. Instructor-led classroom training allows questions and hands-on practice and gives everyone the same message, but it is expensive and takes staff away from their work. One-to-one or on-the-job training can be tailored to each person and uses real tasks, but it is slow and costly for large numbers. E-learning, video tutorials and online help let staff learn at their own pace and revisit material, and are cheap per person once produced, but offer no immediate help with problems and rely on staff being motivated. Printed manuals are cheap and useful for later reference but are often left unread. Cascade training, in which a few staff are trained and then train their colleagues, suits large organisations spread across many sites, although quality may fall at each level. Training must fit the changeover method: direct changeover needs everyone trained before the switch, whereas pilot staff can become the trainers for later sites.

Key terms from the mark schemes

Implementation
The stage at which a new system is put into operation, including installation, data conversion, training, documentation and changeover from the old system.

Changeover
The process of moving from an old system to a new system, using a planned method: parallel, direct, pilot or phased.

Parallel changeover
A changeover method in which the old and new systems run at the same time on the same transactions so that outputs can be compared until the new system is proven.
Credited: parallel running

Direct changeover
A changeover method in which the old system stops and the new one takes over at a single moment, with no overlap.
Credited: big bang / direct implementation

Pilot changeover
A changeover method in which the whole new system is introduced in one branch, department or site before being extended to every other part of the organisation.
Credited: pilot implementation

Phased changeover
A changeover method in which the new system is introduced one component or function at a time.
Credited: phased implementation

Data conversion
Transferring existing records into the structure and format the new system expects, including keying in or scanning paper records and verifying and validating the result.
Credited: data migration

Cascade training
A training method in which a small number of staff are trained and then train their colleagues.
Credited: train the trainer

Exam focus

Describe questions on changeover methods usually give a mark for what the method is and further marks for a benefit and a drawback, so learn at least one of each for all four methods. Evaluate or justify questions need you to apply advantages and disadvantages to the scenario and reach a reasoned recommendation; a general list with no link to the business earns limited credit. Phrase your answer around the organisation, for example explaining that parallel running is justified for payroll because an error would mean staff are paid wrongly. Classification questions describe a situation, often outside computing, and ask which method it shows. Look for the key feature: one location first means pilot, one part at a time means phased, both running together means parallel, and an overnight switch with no overlap means direct. The most common mistake is to confuse pilot and phased, so remember that pilot is the whole system in one place while phased is part of the system everywhere. For data conversion, refer to the actual records in the scenario, such as a diary or receipt book, and explain how they become tables in the new system.

? Check yourself

1. Describe the purpose of system implementation. [2]
2. Describe parallel changeover, giving one benefit and one drawback. [3]
3. Describe direct changeover, giving one benefit and one drawback. [3]

Answers: 1. To put the new system into operation/replace the old system (1) / ensuring resources such as trained staff, hardware and software are in place and data is converted so the system can become fully operational with minimum disruption (1) 2. Both systems operate at the same time, each transaction processed by both and outputs compared, until the new system is proven (1) / benefit: very safe, the old system is a backup if the new one fails / staff can get used to the new system (1) / drawback: expensive/time-consuming to run two systems / time needed to compare results / two sets of data may not be synchronised (1) 3. On a chosen date the old system stops and the new one takes over straight away, with no overlap (1) / benefit: no work is done twice / all staff switch at once / cheapest and quickest (1) / drawback: high risk, should the new system break down, going back may be hard/impossible / there may be a period with no system available (1)

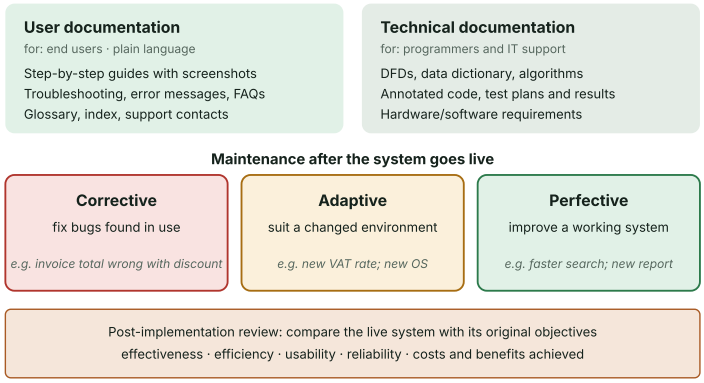
Documentation, maintenance and evaluation

AP09 FACT FILE

Specification: students should be able to

- Explain why documentation is produced at each stage of system development
- Describe user documentation and technical documentation and explain how each is used
- Describe the purpose of system maintenance and the reasons it is needed
- Describe and evaluate corrective, adaptive and perfective maintenance
- Describe the purpose and content of a post-implementation review

Key words: documentation, user documentation, technical documentation, user guide, troubleshooting, FAQ, system maintenance, corrective maintenance, software patch, adaptive maintenance, perfective maintenance, maintenance contract, annotated code, post-implementation review, evaluation criteria



User and technical documentation compared, the three types of maintenance with examples, and the post-implementation review

Overview: documenting and maintaining a system

Once a system is live, two activities keep it useful: good documentation and ongoing maintenance. Documentation is produced at every stage of development, not just at the end. Each stage's documents provide the starting point for the next, such as the requirements specification guiding the designers and the design document guiding the programmers; they record the work done; they support maintenance by keeping track of what was built and what has since changed; and they allow new staff to take over if members of the development team leave. For the finished system there are two main types. User documentation helps the people who use the system day to day, while technical documentation helps the administrators and programmers who look after it. Maintenance is any change made to a system during its working life. However carefully a system is designed and tested, it will need changing: to fix errors, to adapt to new circumstances, or to make improvements. These are known as corrective, adaptive and perfective maintenance. Finally, a post-implementation review evaluates whether the system has achieved the objectives set at the start of the project.

User documentation

User documentation is written for the end users of a system, so it uses plain language, avoids unnecessary jargon and is organised around the tasks people want to carry out. It typically begins with an overview of what the system does, followed by the hardware and software needed to run it and any installation instructions. The core is a user guide giving step-by-step instructions for common tasks, usually illustrated with screenshots. A troubleshooting section explains what to do when things go wrong, including the meaning of error messages, and a frequently asked questions section answers common queries quickly. Additional training material, such as tutorials or practice exercises, may be included, together with routine procedures such as how and when to back up data. User documentation can be supplied as a printed manual, an online help system, context-sensitive help that appears for the part of the screen being used, or short video tutorials. Good user documentation reduces calls to the help desk, speeds up training and helps users become confident and independent with the new system.

Technical documentation

Technical documentation is written for the system administrator, the maintenance staff and any users with strong technical knowledge. Its purpose is to let someone who did not build the system understand exactly how it works, so that it can be maintained, repaired and extended. It contains the system specification and user requirements; data models such as data flow diagrams, entity relationship diagrams and the data dictionary; the input and output designs, report specifications and query designs; the program code listings, which should be annotated with comments and use meaningful identifiers; the test plans together with the test data used and the results obtained; and detailed descriptions of the hardware and software configuration, such as server, network and database settings. Without it, every change would require programmers to work out the system from the code alone, which is slow, expensive and likely to introduce new errors. Technical documentation must itself be kept up to date whenever maintenance is carried out, otherwise it becomes misleading. It is especially important because the original developers may have left by the time changes are needed.

Why maintenance is needed

System maintenance covers every change made to a system after it goes live, for as long as it remains in use. Just as a car needs servicing, a computer system needs attention for as long as it is used, however rigorously it was designed and tested. Several things trigger maintenance. A software error may be uncovered during normal use that testing did not find. Some part of the organisation's business processes may change, so the system must be modified to match. A security vulnerability may be discovered that must be closed before it is exploited. Someone may identify a way of improving the system. The operating infrastructure, such as hardware or networks, may be upgraded, and the system can be changed to take advantage of it. These reasons fall into three broad categories: corrective, adaptive and perfective maintenance. Because maintenance continues for the whole life of a system, which may be many years, its total cost frequently exceeds the cost of the original development. Modular design, annotated code and accurate technical documentation all reduce that cost by making changes quicker and safer to carry out.

Corrective, adaptive and perfective maintenance

Corrective maintenance fixes flaws or weaknesses found once the system is in use, such as calculations that give the wrong result or a security hole that leaves the system open to attack. It is most urgent when the system is insecure or cannot carry out its basic tasks. Fixes are often supplied as software patches, small updates that are downloaded and frequently installed automatically. The original developers usually carry out the work, free of charge if the problem was caused by programmer error, or otherwise under a rolling maintenance contract, and the amount needed should fall over time as errors are removed. Adaptive maintenance changes the system because circumstances have changed, either externally, such as a new tax rate or new legislation, or internally, such as a revised discount policy. The system was correct when built but can no longer do its job without change. The client normally pays, since new functionality is being added, and a large amount soon after delivery may suggest the requirements were poorly analysed. Perfective maintenance improves a system that already works, for example by redesigning forms, improving help, adding shortcuts or archiving data to speed up searches.

Post-implementation review

A post-implementation review is a formal evaluation carried out once the system has been in use long enough for problems and benefits to show, often several months after go-live. It judges the system against the objectives and requirements agreed during analysis rather than against personal opinion. Evidence is gathered from user questionnaires and interviews, observation of the system in use, performance statistics such as response and processing times, help-desk and error logs, and financial records that show whether the costs and savings predicted in the feasibility study were achieved. The review considers effectiveness, meaning whether the system meets its requirements; efficiency, meaning whether it is fast and economical to run; usability, meaning whether staff find it easy to use; reliability, meaning how often it fails or loses data; and whether the expected benefits have been delivered. Its findings identify outstanding problems and recommend corrective, adaptive or perfective maintenance, and it records lessons learned so that future projects in the organisation can be managed better. Evaluation is different from testing: testing happens before handover, whereas evaluation measures success after implementation.

T Key terms from the mark schemes

User documentation
Non-technical documentation for end users, such as an overview, installation instructions, user guide, troubleshooting, FAQs and backup procedures.

Technical documentation
Documentation for administrators and maintenance staff, including specifications, DFDs and data models, I/O and report designs, annotated code, test plans and results, and hardware/software configuration.
`Credited: system documentation`

System maintenance
Any change applied to a system during its lifespan after it has been implemented.
`Credited: maintenance`

Corrective maintenance
Changes made to fix flaws, errors or weaknesses, including security flaws, discovered after the system is in use.

Adaptive maintenance
Changes made so that the system can still carry out its tasks after a change in circumstances, internal or external, such as new legislation, a new tax rate or a new business policy.

Perfective maintenance
Changes that improve the performance or usability of a system that already works, after initial delivery.

Software patch
A small update released to correct, change or enhance a program or its data, often downloaded and installed automatically.
`Credited: patch / update`

Annotated code
Program code that includes comments explaining its purpose and logic, so that others can understand and maintain it.
`Credited: commented code`

Post-implementation review
A formal evaluation, after a system has been in use for some time, of how well it meets its original objectives and requirements.
`Credited: evaluation`

C C# example

`Annotated code is technical documentation`

```
// CalculateFee: returns the monthly fee for a gym member
// Parameters: age in years, isStudent (true if full-time student)
// Rules agreed with client (requirements spec, section 4.2):
//   under 18 = £15, student = £20, everyone else = £32
decimal CalculateFee(int age, bool isStudent)
{
    if (age < 18) return 15m;
    if (isStudent) return 20m;
    return 32m;
}
Console.WriteLine(CalculateFee(16, false)); // 15
Console.WriteLine(CalculateFee(21, true));  // 20
```

Exam focus

Documentation questions often ask you to list components, usually one mark each. Give specific, different items, such as installation instructions, a troubleshooting section and FAQs for user documentation, or annotated code listings, data flow diagrams and test plans with results for technical documentation. Saying that user documentation ‘tells people how to use the system’ is too vague to earn more than one mark. If asked why documentation is produced at each stage, give distinct reasons: a starting point for the next stage, a record of work done, help with maintenance, and continuity when team members change. Maintenance questions frequently describe a series of changes and ask you to classify each. Ask yourself whether the system was wrong when delivered (corrective), whether something around the system has changed (adaptive), or whether a working system is being made better (perfective). A change caused by new law is adaptive, not corrective, because the system met its requirements when built. When defining each type, add a short example, as definitions with examples are commonly credited together. Do not confuse the post-implementation review with testing.

? Check yourself

1. A software house in Banbridge is building a records system for a veterinary practice. Explain why the team should write documentation throughout the project rather than only at the end. [4]
2. State three components of user documentation for an online appointment system used by patients of a Ballymena GP surgery. [3]
3. A Dungannon engineering firm's in-house IT technician has to maintain a stock system written by a contractor who has since left. State three components of technical documentation that the technician would need. [3]

Answers: 1. Any four from: each stage's documents give the next stage something to work from, e.g. programmers code from the design document (1) / they record what was done and decided at each stage (1) / they help later maintenance by showing how the records system was built and what has changed (1) / new staff can pick up the work if team members leave (1) / the finished system can be checked against what the practice agreed (1) 2. Any three from: overview/introduction to the system (1) / hardware/software requirements and installation instructions (1) / user guide with step-by-step instructions/screenshots, e.g. how to book or cancel an appointment (1) / troubleshooting section/error messages explained (1) / FAQ section (1) / training materials/tutorials (1) / backup/maintenance procedures (1) 3. Any three from: system specification/user requirements (1) / data models, DFDs, ER diagrams, data dictionary (1) / input/output design (1) / report specifications (1) / query designs (1) / program code listings/annotated code (1) / test plans with data used and results (1) / detailed hardware and software configuration (1)

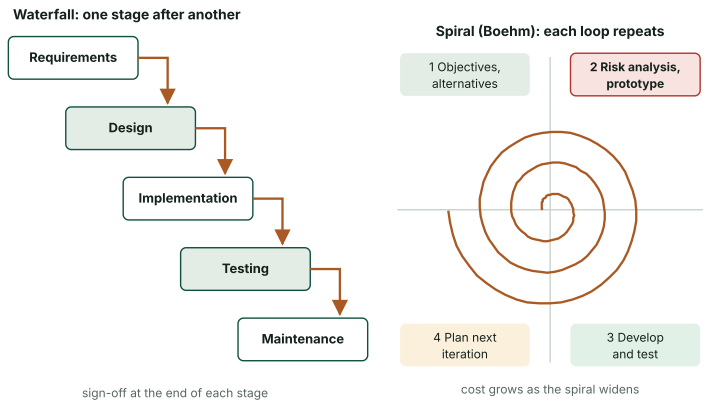
The waterfall model, iterative and spiral development

AP10 FACT FILE

Specification: students should be able to

- Describe the main features of the waterfall model of systems development
- Describe each stage of the waterfall model and the deliverable it produces
- Describe the involvement of users when a system is developed using the waterfall model
- Describe iterative development and the spiral model, including the role of risk analysis
- Evaluate the waterfall model, including its advantages and disadvantages, and its suitability for different projects

Key words: development methodology, waterfall model, sequential, deliverable, sign-off, system specification, design document, handover, maintenance, iterative development, iteration, increment, spiral model, risk analysis, prototype, stable requirements, change control



The waterfall model as a cascade of sequential stages beside the spiral model with its four quadrants and risk analysis in every loop

Overview: why development models matter

Turning an identified need into a working, installed system is a complex process, and organisations follow a development methodology to give it structure. There is no single perfect methodology. Each one makes different choices about how strictly the stages are ordered, how often users are involved and how easily requirements can change, so each suits some projects better than others. The waterfall model is the traditional approach: a fixed sequence of stages in which each must be completed and signed off before the next begins, producing a deliverable that feeds the following stage. Iterative approaches instead build the system in repeated cycles, so that users can see and comment on working versions early. The spiral model is an iterative approach that places risk analysis at the centre of every cycle. Understanding the features, strengths and weaknesses of each model allows you to recommend a suitable approach for a particular project, which is exactly what examination questions in this area ask you to do.

The stages of the waterfall model

The waterfall model is usually shown as five stages. In the requirements or analysis stage the systems analyst uses fact-finding methods (interviews, observation, questionnaires and inspecting documents) to understand any existing system and the needs of its users. The deliverable is the system specification, which must be clear, unambiguous and complete, because a requirement missed here is difficult and expensive to add later. In the design stage the specification is turned into a design document describing the processes, their inputs and outputs, the software units and how they interact, the user interface and the hardware needed. It must be precise because programmers code directly from it. In implementation a team of programmers, each given specific tasks, writes the code to the design, unit testing as they go. In the testing stage the complete system is verified against the specification using a test plan. Finally, maintenance covers installation, migrating data from any old system, ongoing support and corrections or improvements over the life of the system.

Advantages and disadvantages of the waterfall model

The waterfall model divides work into distinct stages with clearly defined deliverables. This makes progress easy to measure, allows each stage to be signed off formally, lets the developer invoice the client stage by stage and produces thorough documentation that helps later maintenance. Because each stage needs different skills, specialist staff can move from one project to the same stage of another project. When requirements are stable and well understood, costs and timescales can be estimated with some confidence. The disadvantages are serious, however. The model is inflexible: once a stage is signed off, changing it means going back up the waterfall. Each handover between stages is a point at which misunderstandings can be introduced, and such errors may travel through several stages before anyone notices, often not until testing, when correcting them is most expensive. Users see no working software until near the end. Staff who specialise in a single stage are also less flexible than the multi-skilled teams used in Agile development, so workloads are harder to balance across projects.

Users and the waterfall model

In a waterfall project the users are mainly involved at two points. At the start, the analyst gathers their requirements, which is especially important when there is no existing system to investigate, because the users' own descriptions are then the main source of information. Near the end, users take part in acceptance testing to confirm that the finished system works as they intended. Between these points they usually see nothing working, so a misunderstanding about what they wanted can remain hidden for months. Many well-documented real project failures share the same pattern: the people who would actually use the system were not consulted properly, deliverables were not formally signed off, changes were not tracked, quality assurance was skipped and no test plans were written. The lesson is that the waterfall model only works if it is followed with discipline. Involving all stakeholders, formal sign-off, change control, quality assurance during coding and a test plan written from the specification all reduce the risk that a misunderstanding reaches the final system.

Iterative development

Iterative development builds a system through repeated cycles rather than a single pass. Each iteration includes some planning, analysis, design, coding and testing, and produces a working version or increment that users can try. Their feedback is used to refine the requirements and to decide what the next iteration should include. Because users see real software early, misunderstandings come to light while they are still cheap to correct, and useful parts of the system can be delivered before the whole project is complete. Iterative development therefore suits projects where requirements are not fully known at the start or are likely to change. It does have costs: it depends on users being available to give regular feedback, it can be harder to agree a fixed price and deadline in advance, and the scope can keep growing if every iteration adds new requests without control. Agile methods such as Scrum, covered in the next lesson, are modern iterative approaches with defined roles and events that help to manage these problems.

The spiral model

The spiral model, proposed by Barry Boehm in the 1980s, is an iterative model driven by risk. The project is drawn as a spiral that starts small at the centre and widens as time and cost increase. Each loop passes through four quadrants. First, the objectives, alternative approaches and constraints for that loop are determined. Second, the alternatives are evaluated and the risks are identified and resolved, often by building a prototype to test an uncertain idea. Third, the next level of the product is developed and tested. Fourth, the client reviews progress and the next loop is planned. Because the biggest risks are tackled first, before large amounts of money are committed, the spiral model suits large, complex, high-risk or novel projects, such as safety-related control systems. Its weaknesses are that risk analysis requires specialist expertise and adds cost, the project can be hard to manage and to bring to a close, and the overhead is not justified for small, low-risk developments.

T Key terms from the mark schemes

Waterfall model
A sequential development methodology in which distinct stages are completed in order, each producing a deliverable and being signed off before the next stage begins.
`Credited: Linear sequential model`

Deliverable
The end product of a stage of development, such as a system specification or design document, which is passed on as the input to the next stage.
`Credited: End product`

System specification
The deliverable of the requirements stage: a clear, unambiguous and complete document of the user requirements for the new system.
`Credited: Requirements specification`

Handover
The point at which work passes from one waterfall stage, and often one group of staff, to the next; misunderstandings introduced here may travel undetected through later stages.
`Credited: Handover interface`

Spiral model
A risk-driven iterative model in which each loop determines objectives, analyses and resolves risks, develops and tests the next version, and plans the next loop.
`Credited: Boehm's spiral model`

Sign-off
Formal agreement by the client or manager that a stage's deliverable is complete and accepted, allowing the next stage to begin.

Design document
The deliverable of the design stage, describing processes, inputs and outputs, software units, user interfaces and hardware precisely enough for programmers to code from.

Iterative development
An approach in which a system is built through repeated cycles of planning, design, coding and testing, each producing a version that users review.

Risk analysis
Identifying possible threats to a project's success, assessing their likelihood and impact, and deciding how to reduce or resolve them.

Exam focus

Questions on this lesson often ask you to outline the key characteristics of the waterfall model, briefly describe each stage, describe user involvement, or list advantages and disadvantages, typically for 1–2 marks per point. For features, mark schemes credit: a sequence of distinct stages, each completed before the next begins, a deliverable at the end of each stage, and the possibility of returning to an earlier stage if an error is found. When describing a stage, name its deliverable (system specification, design document) because that is often the creditworthy detail. For user involvement, say precisely when users are involved – requirements and acceptance testing – not just that involvement is 'low'. Disadvantage answers earn more when they explain consequences, for example that a misunderstanding at a handover may go undetected until testing, when it is costly to fix. For the spiral model, examiners look for iteration and risk analysis. In evaluation questions, finish with a recommendation tied to the scenario: stability of requirements, size, risk and user availability.

? Check yourself

1. State two stages of the waterfall model and the deliverable produced by each. [2]
2. A software firm in Ballymena is using the waterfall model to build a payroll system for a haulage company. Describe the key characteristics of the way this project will be run. [4]
3. Describe what happens during the design stage of the waterfall model. [3]

Answers: 1. Any two stage + deliverable pairs: requirements/analysis → system specification (1) / design → design document (1) / implementation → program code (1) / testing → test results / tested system (1) / maintenance → updated, supported system (1). Both stage and deliverable needed for each mark. 2. Any four from: consists of a sequence of distinct stages (1) / e.g. analysis, design, implementation, testing, maintenance (1) / each stage must be completed/signed off before the next begins (1) / each stage produces a deliverable (1) / the deliverable is the input to the next stage (1) / the team may have to return to an earlier stage if a later stage reveals a mistake (1) / user mainly involved at analysis and testing (1). 3. Any three from: starts from the system specification (1) / identifies the processes and the inputs and outputs of each (1) / designs the software modules and how they communicate with one another (1) / designs the user interface, e.g. storyboards (1) / identifies hardware needs (1) / produces a precise, unambiguous design document (1).

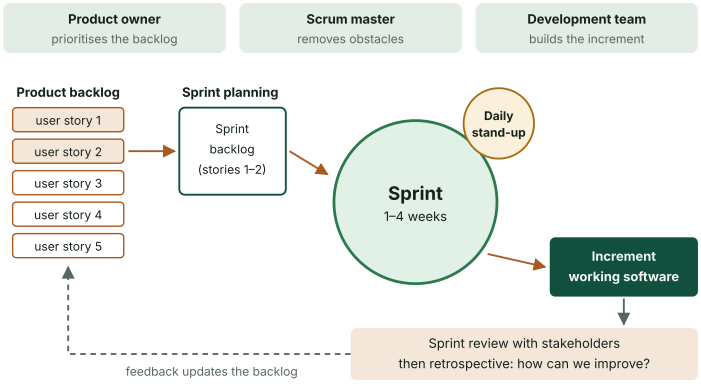
Prototyping, RAD and Agile methods

AP11 FACT FILE

Specification: students should be able to

- Describe prototyping, including throwaway and evolutionary prototyping
- Describe the main features and phases of Rapid Application Development (RAD), including user involvement
- Describe the main features of the Agile approach, including product and release backlogs, sprints, the Scrum team, daily Scrums and burn down charts
- Describe extreme programming (XP) practices such as pair programming and test-driven development
- Evaluate the waterfall model, RAD and Agile and recommend an approach for a given project

Key words: prototype, throwaway prototyping, evolutionary prototyping, rapid application development (RAD), user design, cutover, timeboxing, Agile, product backlog, release backlog, sprint, Scrum master, product owner, daily Scrum, burn down chart, sprint retrospective, user story, pair programming



The Scrum framework: a product backlog feeds sprint planning, the team works in a 1–4 week sprint with daily stand-ups, producing an increment that is reviewed

Overview: faster, more flexible approaches

The waterfall model delivers a system only at the end and involves users mainly at the start and during testing. Rapid Application Development and Agile methods were developed to overcome those weaknesses. Both rely on frequent, close contact with users and on producing working versions early, so that misunderstandings are found while they are still cheap to correct. RAD, developed by James Martin in the late 1980s and popularised by his 1991 book, delivers a system quickly through four phases, heavy use of prototypes, a fixed timescale and requirements ranked so that low-priority extras can be dropped. Agile, which takes its name from a manifesto published by a group of developers in 2001, delivers software in a series of short sprints, guided by a prioritised product backlog and organised by a multi-skilled Scrum team led by a Scrum master. Prototyping, in either its throwaway or evolutionary form, is a technique used within these approaches. None of these methods is always best, so you must be able to evaluate each one against the needs of a particular project.

Prototyping

A prototype is an early model of all or part of a system, built quickly so that users can see it and try it out. It is often a mock-up with little real processing behind it, but it shows the screens, menus and reports that users will work with. This helps users to identify and clarify requirements that they find difficult to describe in words, and it is particularly useful for testing alternative user interfaces. In throwaway prototyping the model is discarded once the requirements have been agreed, and the real system is then built properly using production tools and standards. In evolutionary prototyping the prototype is refined over and over in response to feedback until it becomes the final system, so none of the effort is wasted. The risks are different. A throwaway prototype can make users think the system is nearly finished, while an evolutionary prototype may carry poor structure from early, rushed decisions into the final product, making it harder to maintain and encouraging scope creep.

Rapid Application Development

RAD splits a project into a series of builds and sets a fixed time limit that must not be exceeded. Requirements are put in priority order, so if time runs short the desirable add-ons are sacrificed rather than the deadline. There are four phases. In requirements planning, everyone with an interest meets to agree the scope and key requirements; unlike waterfall, the requirements do not have to be fully fixed, because changes are expected. In user design, users and analysts work closely together, using prototypes to design the functions and to compare alternative interfaces. In construction, the programs are written but are continually refined as users comment on real screens and reports, and the modules are integrated and tested. In cutover, the new system replaces the old, which involves further testing, data conversion and training. RAD gets a system into use much sooner and keeps users involved throughout, so errors surface early. However, it needs skilled practitioners trained in the method and committed users, it can be expensive, and it suits large, safety-critical systems poorly.

Agile: backlogs, sprints and the Scrum team

An Agile project begins with a product backlog, an initial prioritised wish list of everything the product should do. At the start of each phase, release planning selects features from the product backlog and moves them into a release backlog. These are broken into work items, which may be new features, improvements or investigations, then estimated and ordered by priority. The work is carried out in sprints, fixed periods of a few weeks, each delivering a working increment, so the product grows incrementally. Scrum is the framework for running a project in sprints. The Scrum team is small and multi-skilled, typically including developers, testers and customer input, so it can analyse, design, build and test, and everyone contributes to discussion. The Scrum master makes sure the team works effectively, arranges meetings, monitors progress and removes obstacles, much like a project manager. The product owner is the voice of the client, setting priorities in the backlogs and preventing unrealistic items from being added. Each sprint ends with a sprint review of the product and a retrospective on the process.

Monitoring a sprint

Close monitoring keeps an Agile project on schedule. Each day begins with a daily Scrum, a short meeting of about fifteen minutes, often held standing up to keep it brief. Every member answers three questions: what they did yesterday, what they plan to do today and what obstacles are in their way. The Scrum master takes responsibility for resolving the obstacles raised, although everyone may contribute ideas. Progress is shown on a burn down chart, which plots the work remaining against the days of the sprint. The ideal line falls in a straight line from all the work at the start to none at the end. For example, with 60 story points in a 10-day sprint the ideal rate is 6 points per day. If the actual line sits above the ideal line, the team is behind schedule; if it is below, the team is ahead. The chart lets the Scrum master spot a sprint at risk early and take corrective action, such as moving a multi-skilled team member onto a task that is overrunning.

Evaluating Agile and extreme programming

Agile has clear advantages. A project can start quickly with the most important requirements and then evolve, working deliverables appear early, continuous contact with the customer makes misunderstandings less likely, and multi-skilled team members can be moved to whichever task is falling behind. Its weaknesses are that the time needed for tasks can be difficult to estimate, how good those estimates are relies heavily on how experienced the Scrum master is, documentation may be less detailed than in traditional methods, and the customer must commit time throughout, which makes a fixed price and scope hard to agree in advance. Extreme programming is an Agile method that concentrates on engineering practices to keep code quality high despite rapid change. In pair programming two developers share one workstation, one writing code and the other reviewing it, so errors are spotted immediately and knowledge is shared. Test-driven development writes automated tests before the code. Small, frequent releases, continuous integration, refactoring and an on-site customer complete the approach.

T Key terms from the mark schemes

Prototype
An early model of all or part of a system, built quickly so that users can evaluate it and clarify their requirements.
`Credited: Mock-up`

Throwaway prototyping
An approach in which a prototype is used to establish requirements and is then discarded before the real system is built.

Evolutionary prototyping
An approach in which a prototype is repeatedly refined in response to user feedback until it becomes the final system.

Rapid application development (RAD)
A methodology that delivers systems quickly through four phases (requirements, user design, construction, cutover), frequent prototypes, close user involvement and a strict timescale with prioritised requirements.
Credited: RAD

Timeboxing
Fixing the time allowed for a project or part of it so that the deadline cannot be overrun; lower-priority features are dropped instead.
Credited: Timebox

Product backlog
In Agile, the prioritised wish list of all the features the product should eventually provide.

Release backlog
The features chosen from the product backlog for the next release, broken down into work items.

Sprint
In Agile, a fixed period of a few weeks in which the team produces a working increment of the product.
Credited: Iteration

Scrum master
The team member who ensures the Scrum team works effectively, arranges meetings, monitors progress and removes obstacles.

Daily Scrum
A short daily meeting of about 15 minutes in which each team member says what they did yesterday, what they will do today and what obstacles they face.
Credited: Daily stand-up

Burn down chart
A graph of the work remaining in a sprint against time, compared with an ideal straight line, used to check whether work is on schedule.
Credited: Burndown chart

Pair programming
An extreme programming practice in which two programmers work at one computer, one writing code and the other reviewing it, swapping roles regularly.

C C# example

Worked example: a burn down chart

```
int total = 60, days = 10;
int[] actual = { 60, 53, 48, 44, 40, 34, 30, 22, 14, 7, 0 };

for (int day = 0; day <= days; day++)
{
    int ideal = total - (total / days) * day;
    string status = actual[day] > ideal ? "behind" : "on track";
    Console.WriteLine($"Day {day}: ideal {ideal}, actual {actual[day]} ({status})");
}
```

Exam focus

Typical questions ask you to describe the main features of RAD or Agile, briefly describe each RAD phase, describe user involvement, or give two advantages and two disadvantages, usually at one or two marks per point. For RAD, credit is given for builds, a strict timescale, requirements ranked in priority with desirable add-ons dropped, frequent prototypes and the four named phases: requirements, user design, construction and cutover. For Agile, name the product backlog, the release backlog, sprints and the Scrum master co-ordinating the team. User involvement should be stated precisely: continual, through prototypes in RAD or constant consultation in Agile. Distinguish throwaway from evolutionary prototyping by what happens to the prototype afterwards. Do not confuse the sprint review, which examines the product, with the retrospective, which examines the process. In extended evaluation questions, compare all three approaches against the scenario – stability of requirements, user availability, deadline, size and need for documentation – and finish with a justified recommendation. Generic lists of features without reference to the context score poorly.

? Check yourself

1. Explain the difference between throwaway and evolutionary prototyping. [2]
2. A Craigavon developer promises a hotel group a working room-booking system 'in twelve weeks, guaranteed' using Rapid Application Development (RAD). Describe the features of RAD that make such a promise possible. [4]
3. Name the four phases of RAD and briefly describe what happens in the user design phase. [4]

Answers: 1. Throwaway: the prototype is discarded once requirements are agreed and the real system is built separately (1) / Evolutionary: the prototype is refined repeatedly until it becomes the final system (1). 2. Any four from: project divided into a series of builds (1) / strict timescale/timebox that cannot be overrun (1) / requirements ranked in priority order (1) / desirable add-ons dropped if time runs out (1) / prototypes produced frequently and delivered to users for feedback (1) / much more user involvement than waterfall (1) / four phases: requirements, user design, construction, cutover (1) / reuse of components / GUI builders (1). 3. Requirements (planning), user design, construction, cutover (2 marks for all four in any order; 1 mark for two or three) / User design: users and analysts interact regularly (1) / prototypes/mock-ups show users what the system will look like and test alternative interfaces (1).

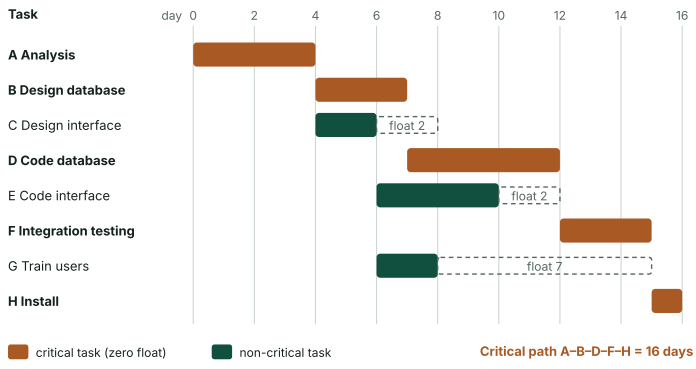
Organising and managing a software project

AP12 FACT FILE

Specification: students should be able to

- Describe the main elements in the organisation of a software project: resources, constraints and risks
- Explain the main features of project management for a software project, including the role of the project manager and SMART objectives
- Describe the roles of members of a software project team
- Construct and interpret Gantt charts and critical path analysis, identifying the critical path, earliest start times, latest finish times and float
- Evaluate the use of project management tools: Gantt charts and critical path analysis

Key words: resources, recurring resource, depleting resource, elastic resource, dedicated resource, constraints, scope, risk, requirements shift, staff turnover, project manager, SMART objectives, performance indicator, Gantt chart, critical path analysis (CPA), earliest start time (EST), latest finish time (LFT), float



Gantt chart for an eight-task project showing the critical path A, B, D, F, H of 16 days and the float on tasks C, E and G

Overview: resources, constraints and risks

Every software project, whichever methodology it follows, has the same central task: to achieve its objectives using the resources available, staying within its constraints and keeping risks under control. Resources are what the project can use: human resources such as programmers and database administrators, technical resources such as processing power and network bandwidth, financial resources such as the budget, and physical resources such as office space. Constraints are the limits placed on the project, usually concerning its scope, its deadline and the resources available. Risks are anything that could harm the project or cause it to fail, such as missing the deadline, shifting requirements, staff leaving or low productivity. The project manager is responsible for balancing all three. To do this they set clear, agreed objectives, plan and schedule the work, allocate staff and monitor progress. Tools such as Gantt charts and critical path analysis help them see how tasks depend on one another, which tasks can run in parallel and which tasks must not be delayed if the project is to finish on time.

Classifying and managing resources

As well as being human, technical, financial or physical, resources can be classified by how they can be used. By availability, a recurring resource such as a developer assigned to the project does not run down as it is used, whereas a depleting resource such as money or time is gradually consumed. By location, a fixed resource such as a server room cannot easily be moved, while a movable resource such as a member of staff can. By elasticity, an elastic resource such as the budget or the number of staff can in theory be increased or reduced, while an inelastic resource such as the time before a hard deadline cannot. By ownership, a dedicated resource works only on this one project, while a shared resource, such as a graphic designer, also works on others. Resource management has three activities. Planning estimates and schedules what will be needed. Deployment allocates resources to tasks and monitors progress, reallocating them if a stage slips. Release compares what was actually used with the plan, so that future estimates improve.

Constraints and risks

Constraints restrict what the developer can do. Scope constraints are characteristics fixed by the client, such as the number of simultaneous users, the response time required or the browsers and mobile platforms that must be supported. Time constraints set a deadline, so the project manager must check that the agreed scope can actually be delivered in the time available. Resource constraints limit budget, staff and equipment; even elastic resources have a ceiling in practice. Constraints must be stated explicitly, with an actual date and an actual maximum budget, because vague phrases such as ‘as quickly as possible’ are open to different interpretations. Risks must be identified and managed. Missing the deadline can cost the client business and, under a fixed-price contract, leave the developer paying for extra work. Requirements shift, often caused by users not being involved enough during analysis, leads to rework and extra cost. Staff turnover can remove vital knowledge, so collaboration and documentation should share it. Low productivity on long tasks is reduced by splitting them into smaller sections with their own deadlines.

Software project management and SMART objectives

Software project management emerged as a distinct role in the 1970s after a series of high-profile failures. It differs from managing a construction project in three ways: software is intangible, so progress is hard to see; most large systems are bespoke, so experience from one project transfers less well to the next; and technology changes so quickly that new tools may appear during the project. The project manager controls analysis, development and deployment, producing a realistic plan that sets out what will be built, the acceptance criteria and how the system will be maintained, then scheduling activities, allocating staff and managing risk. Success depends on clear objectives agreed by everyone at the start. Each objective needs a statement of what it should achieve, performance indicators that will measure it and the value of each indicator that counts as success. A single target value is better than a range, because a range lets each side claim a different result as success. The mnemonic SMART summarises this: objectives should be Specific, Measurable, Agreed, Realistic and Timed.

Gantt charts

Project management software helps the manager plan each task, delegate staff to tasks, monitor progress and analyse the schedule to spot problems before they happen. The Gantt chart is the most widely used of these tools. It is a horizontal bar chart in which each task is a bar placed against a timeline, so the position and length of the bar show the task’s start, duration and end. A column often records which tasks each task depends on. From a Gantt chart a manager can see what each task is, when it is scheduled to start and finish, how long it lasts, how far tasks overlap and which tasks depend on others. It is easy for clients and staff to understand, and it can show progress to date and the knock-on effect of a delay. In the worked example, the database coding can start only when the database design is finished, and integration testing must wait for both coding tasks. On a large chart, however, dependencies can be hard to follow and the critical path is not always obvious.

Critical path analysis

Critical path analysis models a project as a network. Nodes, drawn as circles, represent events such as the start or end of tasks, and arrows represent the tasks, labelled with their durations. Each node records its earliest start time and its latest finish time. In the forward pass, working from left to right, the earliest start time of each node is found from the tasks leading into it; where several arrows meet, the highest value is taken because every one of those tasks must be finished. The final node gives the project duration. In the backward pass, working from right to left, the latest finish time of the final node is set to the project duration and task durations are subtracted; where paths meet, the lowest value is taken. Nodes where the two times are equal are critical, and the tasks joining them form the critical path, the longest chain through the project. Any delay to a critical task delays the whole project. Non-critical tasks have float, the time they can slip without affecting the finish date.

T Key terms from the mark schemes

Resources
Anything a project can use to meet its objectives: human (staff), technical (e.g. processing power), financial (budget) or physical (e.g. accommodation).

Elastic resource
A resource whose amount can be increased or decreased, such as the budget or number of staff; an inelastic resource, such as time before a hard deadline, cannot.

Constraint
A restriction on a project that the developer cannot change, usually concerning scope, time or resources.

Risk
Anything that could have a harmful effect on a project or cause it to fail, such as missed deadlines, requirements shift, staff turnover or low productivity.

Requirements shift
Changes to what the client wants after the project has started, often caused by inadequate user involvement during analysis.
Credited: Scope creep

Project manager
The person responsible for planning, scheduling, resourcing, monitoring and managing risk so that a project is delivered on time, within budget and to the required quality.
Credited: Software project manager

SMART objectives
Objectives that are Specific, Measurable, Agreed, Realistic and Timed, so that all parties share the same view of success.

Gantt chart
A horizontal bar chart showing project tasks against a timeline, including their start and end, duration, overlaps, dependencies and milestones.

Critical path
The longest sequence of dependent tasks in a project, which determines the minimum time in which the project can be completed.

Earliest start time (EST)
In critical path analysis, the earliest time a task can start, once every task it depends on has finished.
Credited: EST

Latest finish time (LFT)
In critical path analysis, the latest time by which an activity must be complete if the project is not to be delayed.
Credited: LFT

Float (slack)
The amount of time a non-critical task can be delayed without delaying the completion of the whole project.
Credited: Slack

C C# example

Worked example: finding the critical path

```
string[] task = { "A", "B", "C", "D", "E", "F", "G", "H" };
int[] dur = { 3, 4, 2, 5, 6, 3, 2, 1 };
int[][] pre = { new int[0], new[] { 0 }, new[] { 0 }, new[] { 1 },
                new[] { 2 }, new[] { 3, 4 }, new[] { 2 }, new[] { 5, 6 } };
int[] est = new int[8];
for (int i = 0; i < 8; i++)          // forward pass
    foreach (int p in pre[i])
        est[i] = Math.Max(est[i], est[p] + dur[p]);
int end = est[7] + dur[7];            // 16 days
int[] lft = new int[8];
for (int i = 7; i >= 0; i--)          // backward pass
{
    lft[i] = end;
    for (int j = i + 1; j < 8; j++)
        if (Array.IndexOf(pre[j], i) >= 0)
            lft[i] = Math.Min(lft[i], lft[j] - dur[j]);
    Console.WriteLine($"{task[i]}: EST {est[i]}, float {lft[i] - dur[i] - est[i]}");
}
```

Exam focus

Expect to explain resources, constraints and risks with examples, expand SMART, draw a Gantt chart from a dependency table, complete a CPA diagram and identify the critical path, and explain the advantages of these tools. When explaining resources, constraints or risks, define the term and give a specific example; mark schemes credit the categories human, technical, financial and physical for resources, scope, time and resources for constraints, and time, requirements shift, staff turnover and productivity for risks. SMART should be expanded as Specific, Measurable, Agreed, Realistic and Timed, each with a brief explanation. On Gantt charts, check that no bar starts before every task it depends on has finished. On CPA diagrams, take the highest value when merging in the forward pass and the lowest in the backward pass, list the critical tasks in order and state the duration. When asked for advantages, link each feature to a benefit, for example identifying tasks that can run in parallel to minimise the project duration. A common error is giving the shortest path as the critical path; it is always the longest.

? Check yourself

1. State the four types of resource that may be available to a software project and give an example of each. [4]
2. Explain the difference between an elastic resource and an inelastic resource, using an example of each. [2]
3. Describe three constraints that may be placed on a software project. [3]

Answers: 1. Human, e.g. programmer / database administrator (1) / Technical, e.g. processing power / network bandwidth (1) / Financial, e.g. the budget (1) / Physical, e.g. office accommodation / transport links (1). Type and valid example needed for each mark. 2. Elastic: the amount can be increased or decreased, e.g. more money or staff can be assigned (1) / Inelastic: the amount cannot be changed, e.g. the time available before a fixed deadline (1). 3. Scope – characteristics of the product fixed by the client, e.g. number of simultaneous users, response time, browser compatibility (1) / Time – a fixed deadline by which the system must be delivered (1) / Resources – limits on budget, staff or equipment, e.g. a maximum budget (1).

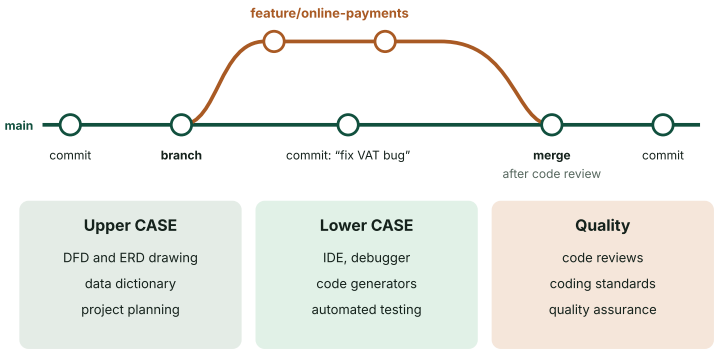
Version management and tools for software projects

AP13 FACT FILE

Specification: students should be able to

- Describe the importance of version management during systems development
- Describe how a version management system records changes and how versions are branched, merged and rolled back, including version management graphs
- Describe the conditions needed for version management to work successfully
- Describe the use of computer-aided software engineering (CASE) tools and collaboration tools
- Describe code reviews, quality assurance and the use of standards in software projects

Key words: version management, version control, repository, commit, revision, branch, merge, conflict, rollback, version management graph, CASE tool, upper CASE, lower CASE, code review, quality assurance, coding standards, collaboration tools



Version control: commits on the main branch, a feature branch that is merged back, and the CASE tools and quality techniques used in software projects

Overview: keeping control of shared code

On any project larger than a single programmer, many people change the same set of files over a long period. Without control, one person's work overwrites another's, nobody can tell who changed what, and a single bad change can break a system with no easy way back. Version management is how a development team keeps track of every change to its code files over time. All files are held in a repository, access is monitored, and each change is recorded with its author, the time it was made and a note explaining it. The team can work on different features in parallel, merge their work together, and return to an earlier working version if an error is discovered. Version management is supported by other tools and practices that help teams produce reliable software: CASE tools that assist with design and coding, collaboration tools that keep distributed teams co-ordinated, code reviews that catch defects early, and quality assurance processes and standards that build quality into every stage of the project.

How version management works

A version management system stores every file in a repository. When a developer finishes a change, they commit it, which saves a new version and records who made the change, when it was made and a message describing what was changed and any knock-on effects that others need to deal with. Each version is numbered, so any earlier version can be identified and retrieved. Developers can create a branch, a separate line of development, so that a new feature can be built without disturbing the main version that others rely on. When the feature is ready, the branch is merged back, combining both sets of changes. If two developers have changed the same lines, the system flags a conflict for a developer to resolve rather than silently losing work. Finally, the team can roll back to any previous version and carry on developing from there. Modern systems such as Git work in this way, and hosted services keep a remote copy of the repository, which also provides a backup.

Version management graphs

A project's history can be drawn as a version management graph. Each node is a version of a file, and an arrow joins each version to the version it was created from. A straight chain shows changes made one after another. When two arrows leave the same node, two developers have worked from the same version in parallel. When two arrows enter one node, those parallel versions have been merged. In the worked example, Ciara and Tomás both start from v2. Ciara's rewrite creates v3, Tomás's allergen labels create v4, and the system merges them into v5. Testing then reveals that v3 introduced a serious error, so v5 is also faulty. The team rolls back to v4, the last stable version, and develops it into v6. On the graph, v6 has an arrow from v4, not from v5, showing that the work from v1, v2 and v4 is kept while the faulty changes from v3 are left out. Drawing these graphs carefully, with every version labelled and every arrow pointing from the older to the newer version, is a common examination task.

Conditions and benefits

Version management only works if the whole team co-operates with it. There must be a consistent method of tracking changes and recording every version number; changes must be reversible; and every team member must be working from the same, up-to-date copy of each file when editing it, otherwise someone may build on out-of-date code and their work will clash with others'. In practice this also means committing regularly with clear messages, so that the history is useful and conflicts stay small. When these conditions are met the benefits are considerable. Several developers can work on the same system at once; branches keep experimental work away from the working version; merging and conflict detection stop changes overwriting each other; a broken change can be undone by rolling back; the full history provides an audit trail showing who changed what and why, which also helps a new team member understand the code; and a remote repository protects the team against losing work if a computer fails.

CASE and collaboration tools

Computer-aided software engineering tools are programs that support or automate parts of the development process. Upper CASE tools help with analysis and design, for example drawing data flow diagrams and entity relationship diagrams, maintaining a data dictionary and planning the project. Lower CASE tools support implementation and testing, such as integrated development environments, code generators, debuggers and automated testing tools. Integrated CASE tools combine both, keeping diagrams, data definitions and code consistent with one another. CASE tools enforce standard notation, produce documentation automatically and can speed up development, but they can be expensive and staff need training to use them well. Collaboration tools help a team, which may be spread across several offices or working remotely, to stay co-ordinated. They include shared repositories with issue trackers for recording bugs and tasks, messaging channels, video meetings, shared documents and wikis, task boards showing who is working on what, and continuous integration servers that automatically build and test the code every time a change is committed.

Code reviews, quality assurance and standards

In a code review, other developers examine code before it is accepted into the main version, often through a pull request on a shared repository. They look for errors, security weaknesses, poor readability and breaches of the team's coding standards. Reviews find defects early, when they are cheapest to fix, and spread knowledge of the code across the team, which reduces the damage if a developer leaves. Reviews can be formal inspection meetings or informal peer checks, and pair programming is a continuous form of review. Quality assurance is broader: it is the set of planned activities, including reviews, walkthroughs, inspections, testing and audits of documentation, that build quality into every stage of a project rather than relying on testing at the end. Standards support quality assurance. Coding standards set rules for naming, layout, commenting and error handling, so that any team member can read and maintain anyone else's code. Process and product standards, such as ISO 9001 for quality management and WCAG for accessibility, give the client confidence that the product has been built properly.

Key terms from the mark schemes

Version management
The process by which a development team keeps track of changes to code files over time, recording who made each change, when and why, and allowing return to earlier versions.
Credited: Version control / source control

Repository
The central store in which a version management system holds all of a project's files and their complete history of changes.

Commit
Saving a change to the repository as a new version, together with a message describing the change.
Credited: Check in

Branch
In version management, a separate line of development used to work on a feature or fix without affecting the main version.

Merge
Combining two parallel versions or branches into a single version that contains both sets of changes.

Rollback
Returning to an earlier, stable version of the code and continuing development from that point.
Credited: Revert

Version management graph
A diagram of a project's history in which nodes are versions and arrows show which version each was created from, showing branches, merges and rollbacks.

CASE tool
Computer-aided software engineering tool: software that supports or automates parts of development, such as diagramming (upper CASE) or code generation and debugging (lower CASE).

Code review
The examination of code by other developers to find errors, security weaknesses and breaches of coding standards before it is accepted.
Credited: Peer review

Quality assurance
Planned and systematic activities, such as reviews, inspections, testing and audits, that build quality into every stage of a project.
Credited: QA

Coding standards
Agreed rules for writing code, such as naming conventions, layout, commenting and error handling, that make code consistent and maintainable.

C C# example

Quality assurance and standards

```
// Coding standard example: meaningful names, a constant, a comment
// Returns the price including VAT, rounded to 2 decimal places
static decimal AddVat(decimal netPrice)
{
    const decimal VatRate = 0.20m;
    return Math.Round(netPrice * (1 + VatRate), 2);
}

Console.WriteLine(AddVat(12.50m));   // 15.00
```

Exam focus

Questions may ask you to explain why version management is important, describe what it records, state the conditions for it to work, or construct a version management graph from a description. Mark schemes credit: tracking every change to code files over time, being able to return to a previous working version, recording who made each change, when and a note describing it, and the three conditions – a consistent method of tracking changes and version numbers, the ability to reverse changes, and everyone working from the same version. Do not describe version management as simply 'saving files' or 'making backups'; link each feature to a team problem it solves, for example branches let a programmer develop a feature without breaking the version others are using. For graph questions, read the description slowly, label every version from v1 upwards, draw parallel versions from the same parent, show merges with two incoming arrows and show a rollback by drawing the new version from the earlier node. For CASE tools, give a specific example; for quality, remember that QA covers the whole process while testing is only one part.

? Check yourself

1. State three items of information a version management system records each time a change is made. [3]
2. Eight programmers at a Belfast software house are working on the same patient-records system at the same time. Explain why the company should use version management. [4]
3. Explain the terms branch and merge in version management. [2]

Answers: 1. Who made the change (1) / when the change was made (1) / a note/message describing the change and any knock-on effects (1). Also accept: the new version number (1). Max 3. 2. Any four from: many programmers work on a shared set of files (1) / without control, changes would overwrite each other / chaos (1) / every change is tracked with who, when and why (1) / the team can return to a previous working version if an error is found (1) / branches allow parallel development without affecting the working version (1) / merging combines work and flags conflicts (1) / provides an audit trail (1) / remote repository acts as a backup (1). 3. Branch: a separate line of development created from a version so that work, e.g. a new feature, does not affect the main version (1) / Merge: combining two lines of development/versions into a single version containing both sets of changes (1).

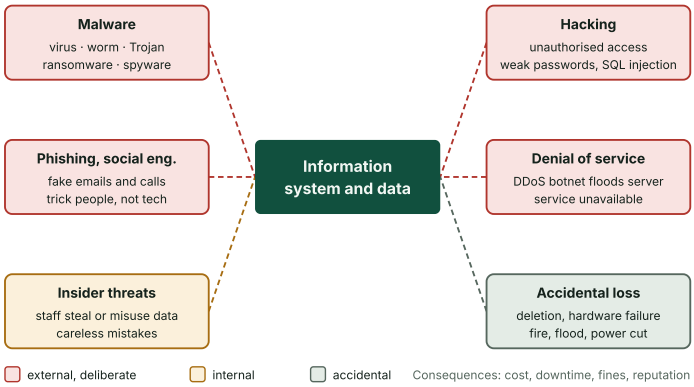
Threats to data, archiving and the need for backup

AP14 FACT FILE

Specification: students should be able to

- Describe the main deliberate threats to data: malware (viruses, worms, Trojans, ransomware and spyware), hacking, social engineering and denial of service
- Describe internal and accidental threats, including insider misuse, human error, hardware failure and natural disasters, and their consequences for organisations
- Explain why data is archived, including the advantages and disadvantages of archiving
- Explain the difference between archiving data and backing up data
- Explain the need for backup and recovery, including the use of restore points

Key words: malware, virus, worm, Trojan, ransomware, spyware, hacking, phishing, social engineering, distributed denial of service (DDoS), insider threat, active data, inactive data, data archiving, backup, data recovery, restore point, transaction log



Threats to an information system grouped as external deliberate attacks, insider threats and accidental loss

Overview: threats to data and why recovery matters

Every organisation, from a corner shop in Omagh to a health trust in Belfast, depends on its data. That data faces two broad kinds of threat. Deliberate threats include malicious software, hacking, social engineering, denial of service attacks and insiders who misuse the access they have been given. Accidental threats are at least as common: staff delete or overwrite files, hardware fails, power is lost, and buildings suffer fire or flooding. The consequences of losing data, or of having it stolen, include lost income, time when systems cannot be used, regulatory action by the Information Commission (formerly the ICO) under the UK GDPR and Data Protection Act 2018, and loss of customer trust. Because no organisation can remove every risk, good practice has two parts. The first is to reduce the likelihood of loss through security measures. The second is to make sure that, when loss happens anyway, data can be recovered quickly. This lesson covers the threats, explains how inactive data is archived so it is retained safely without slowing the live system, and explains why current data must be backed up and how a restore point is used during recovery.

Malware

Malware is any software written to damage, disrupt or gain unauthorised access to a system. The types are best told apart by how they spread and what they do. A virus attaches itself to a host file or program and spreads when that host is run or shared, so it needs some user action. A worm is a standalone program that copies itself across a network on its own, usually by exploiting a vulnerability that has not been patched, and can spread very quickly while consuming bandwidth. A Trojan pretends to be legitimate software, such as a free game or a fake invoice viewer, and the user installs it willingly; it does not replicate but delivers a hidden payload such as a back door that gives an attacker remote control. Ransomware encrypts the victim's files and demands payment, often in cryptocurrency, for the key; paying gives no guarantee of recovery, which is why recent, separate backups are the best defence. Spyware, including keyloggers, secretly records activity such as keystrokes, websites visited or passwords and sends the information to the attacker.

Hacking, social engineering and denial of service

Hacking means gaining unauthorised access to a computer system or the data on it. Under the Computer Misuse Act 1990, as amended, simply accessing material without permission is an offence, and further offences cover access with intent to commit other crimes and impairing the operation of a system. Hackers look for unpatched software, weak or reused passwords that can be cracked by brute-force or dictionary attacks, and web forms that do not validate input and so allow SQL injection. Social engineering attacks the person rather than the technology. Phishing emails imitate a bank, delivery firm or the organisation's own IT department to persuade users to enter credentials on a fake site; spear phishing is tailored to one individual, smishing uses text messages and vishing uses phone calls. These attacks exploit trust, urgency and fear, so awareness training is essential. A denial of service attack overwhelms a server with traffic so that genuine users cannot reach it. A distributed attack uses a botnet of compromised devices, making it much harder to filter out, and its main harm is lost availability rather than stolen data.

Internal and accidental threats

Many incidents start inside the organisation. A malicious insider, such as an employee who has been dismissed or is joining a competitor, may copy, alter or delete data using access they legitimately hold, which makes the activity harder to spot. Careless insiders cause even more breaches: sending a spreadsheet to the wrong email address, leaving an unencrypted laptop on a train, or clicking a phishing link. Accidental threats also include human error such as saving over a file or deleting a folder, hardware failure such as a disk crash, power cuts and surges, and environmental events such as fire or flooding. When personal data is involved, a breach that risks harm to individuals must normally be reported to the Information Commission within 72 hours of the organisation becoming aware of it. In an answer about consequences, always apply them to the scenario: a hospital might be unable to see test results, a council might miss benefit payments, and a sports club might lose its fixture and membership lists just before the season starts.

Archiving inactive data

Organisations hold active data, which is needed immediately for daily work, and inactive data, which is rarely used but must still be kept. A further education college needs instant access to current students' timetables, attendance and fees, but former students' records are needed only occasionally, for references, audits or legal requirements. Data archiving is the process of moving such inactive data out of the live system into separate, secure, long-term storage. The archive must be indexed so that records can be located again, and the data must be complete and suitably structured so that it is meaningful years later. The main advantages are that the live system holds far fewer records, so processing, reporting and backups are faster; older data is retained for reference and compliance; and cheaper, slower storage can be used. The disadvantages are that archived data is not instantly available, time must be spent searching for it, and access may be slow once it has been found. Archived personal data is still covered by the UK GDPR, so it should be kept only for a defined retention period and then securely deleted.

Backup, recovery and restore points

Backing up is a different activity from archiving. Archiving moves inactive data away for long-term retention, whereas a backup is a copy of current data, taken regularly and stored separately, so that the live system can be rebuilt if data is corrupted, deleted or destroyed. Because losing current data hits an organisation immediately, backups are a high priority and are normally automated, perhaps nightly or even several times an hour for critical systems such as online booking. Data recovery is the process of using backed-up data to return a system to an earlier correct state. A restore point captures the exact state of the system when a backup was taken, so recovery can return the system exactly to that state. Anything that changed after the restore point would be lost, unless the changes were also recorded elsewhere. Many database systems keep a transaction log; after restoring the latest backup, the logged transactions are re-applied in order, rolling the data forward to just before the failure. The gap between backups therefore determines how much work is at risk.

T Key terms from the mark schemes

Malware
Malicious software, such as viruses, worms, Trojans, ransomware and spyware, written to damage or disrupt systems or to gain unauthorised access to data.
`Credited: malicious software`

Worm
Standalone malware that replicates itself across a network without needing a host file or any user action, usually by exploiting unpatched vulnerabilities.

Trojan
Malware disguised as legitimate software that the user installs; it carries a hidden harmful payload such as a back door and does not self-replicate.
`Credited: Trojan horse`

Ransomware
Malware that encrypts a victim's files and demands payment in exchange for the decryption key.

Phishing
A social engineering attack using fraudulent emails, messages or websites that imitate trusted organisations to trick people into revealing personal details.

Social engineering
Manipulating people into giving away confidential information or access, rather than attacking the technology directly.

Distributed denial of service (DDoS)
An attack in which many compromised devices flood a server or network with requests so that legitimate users cannot access it.
`Credited: DDoS attack`

Insider threat
A risk to data posed by people inside an organisation, such as employees or contractors, who misuse their legitimate access deliberately or carelessly.

Data archiving
Moving inactive data that is no longer needed for daily use from the live system to separate, secure long-term storage, indexed so it can be retrieved later.
`Credited: archiving`

Inactive data
Data that is no longer needed for day-to-day operations but must be kept for reference, audit or legal reasons.
`Credited: historical data`

Data recovery
Using backed-up data to restore a system to a previously correct state after data has been lost or corrupted.

Restore point
A record of the state of a system at the precise moment it was backed up, allowing the system to be returned to that state.

Exam focus

Questions on this lesson tend to use State or List for one-mark points, Describe or Explain for two to four marks, and an extended question of six marks or more set in a scenario. For malware, make the distinguishing feature explicit: a worm spreads by itself across a network, a virus needs a host file and user action, and a Trojan is disguised and does not replicate. For archiving, a definition worth full marks says that inactive data is moved to separate secure storage and can still be retrieved later, ideally through an index. Advantages and disadvantages earn most credit when applied: say that reports on the supermarket's or club's live data would run faster, not just that the system improves. A common mistake is to treat archiving and backup as the same thing; remember that archiving moves inactive data out, while backing up copies current data that stays in the live system. When asked about a restore point, say what it records and that the system is restored to that state, then mention re-applying transactions logged since. In extended answers, cover a range of points and reach a judgement linked to the context.

? Check yourself

1. Describe the difference between a virus and a worm. [2]
2. A member of staff at a Lisburn engineering firm installs a free PDF tool that turns out to be a Trojan. Describe how the Trojan could harm the firm. [3]
3. Explain what is meant by phishing and describe two features that may help a user recognise a phishing email. [3]

Answers: 1. A virus attaches to a host file and needs the user to run or share it to spread (1) / a worm is standalone and self-replicates across a network without user action (1). 2. Any three from: it is disguised as legitimate software so the user installs it willingly (1) / it may open a back door giving an attacker remote access (1) / it may install spyware or a keylogger to steal passwords or designs (1) / it may download ransomware that encrypts the firm's files (1) / leading to loss of data, downtime or a data breach (1). 3. Phishing is a fraudulent email or message pretending to come from a trusted organisation (1) to trick the user into revealing personal details such as passwords or bank details (1). Any one feature for 1 mark, max 3 total: urgent or threatening tone / generic greeting / sender address or link that does not match the real organisation / spelling and grammar errors / request for login or payment details / unexpected attachment.

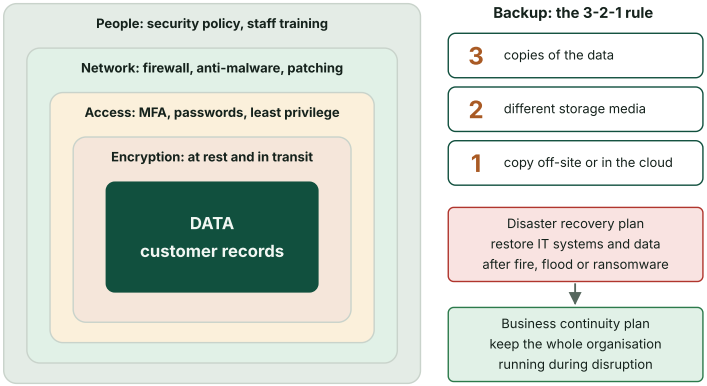
Protecting systems: security measures, backup methods and disaster recovery

AP15 FACT FILE

Specification: students should be able to

- Describe the purpose and content of a security policy
- Describe access control and authentication methods, encryption and firewalls
- Evaluate methods of backup: full backup, differential backup and incremental backup
- Describe how a system is restored using each method of backup
- Explain the purpose of a disaster recovery plan
- Describe the contents of a typical disaster recovery plan, including recovery point and recovery time objectives, and how plans are tested

Key words: security policy, authentication, multi-factor authentication, access rights, encryption, firewall, full backup, differential backup, incremental backup, off-site backup, downtime, disaster recovery, disaster recovery plan, recovery point objective (RPO), recovery time objective (RTO), business continuity plan



Layers of protection around data – policy and training, network defences, authentication and access rights, encryption – alongside the 3-2-1 backup rule and disaster recovery

Overview: layers of protection and a plan for recovery

No single measure keeps data safe, so organisations use layers. Some measures are procedural, such as a security policy that tells staff how to handle data, choose passwords and report incidents. Others are technical: authentication proves that users are who they claim to be, access rights limit what each user can do, encryption makes intercepted or stolen data unreadable, and firewalls filter network traffic. Physical measures, such as locked server rooms, protect the hardware itself. Even with all of these, data can still be lost through hardware failure, human error, ransomware or a flood, so every organisation also needs to be able to recover. That depends on a sensible backup strategy, using full, differential or incremental backups, and on knowing exactly how to restore from them. For serious incidents that knock out whole systems or buildings, a documented disaster recovery plan sets out what must be restored first, how quickly, by whom and from where. This lesson covers the protective measures, evaluates the three methods of backup, explains how systems are restored, and describes the purpose and contents of a disaster recovery plan.

Security policy, authentication and access rights

A security policy is a formal document that sets out the rules, procedures and responsibilities an organisation uses to protect its systems and data. It normally covers acceptable use of equipment and the internet, password requirements, who may access which data, the use of personal devices and removable media, remote working, how to report incidents, backup procedures and the training staff will receive, along with the consequences of breaking the rules. It must be communicated, enforced and reviewed. Access control then turns the policy into practice. Users first identify themselves and then authenticate using something they know, something they have or something they are. Multi-factor authentication combines at least two of these factor types, so an attacker who has phished a password still cannot log in without, for example, the user's phone. Once logged in, access rights determine what each user may do, such as read-only or read and write, and to which files. Following the principle of least privilege limits the damage caused by mistakes, malicious insiders or a compromised account, and audit logs help detect misuse.

Encryption and firewalls

Encryption applies an algorithm and a key to plaintext to produce ciphertext, which cannot be understood without the correct key. In symmetric encryption the same key encrypts and decrypts, which is fast but creates the problem of sharing the key safely. Asymmetric encryption uses a key pair: anyone can encrypt with the public key, but only the holder of the matching private key can decrypt. In practice, asymmetric encryption is used when a secure web connection is set up to agree a symmetric session key, which then encrypts the rest of the traffic. Encryption protects data in transit, such as HTTPS and virtual private networks, and data at rest, such as laptop drives, USB sticks and backups. It does not stop data being stolen or deleted; it stops stolen data being read. A firewall examines incoming and outgoing traffic against a set of rules, such as permitted IP addresses, ports and protocols, and blocks anything that does not match. Hardware firewalls guard the boundary of a network, while software firewalls protect individual devices. Anti-malware software and prompt patching complete the technical defences.

Full, differential and incremental backups

A full backup copies every file on the system. It is the simplest to restore, because the whole system comes back from a single backup, but it takes the longest to create and uses the most storage, and since only a small fraction of data usually changes each day, most of the copying is repeated work. Many organisations therefore take a full backup weekly and combine it with one of the other methods. A differential backup copies every file altered after the most recent full backup. It is quicker to create than a full backup, and a restore needs only two backups, but each differential is larger than the one before because changes keep accumulating until the next full backup. An incremental backup copies only the files modified after the most recent backup, whatever its type. It is the fastest to create and needs the least storage, but restoring requires the full backup plus every incremental since, applied in order, so it is the slowest and most complex restore. If any one incremental in the chain is corrupt or missing, later changes cannot be recovered reliably. The best choice depends on how much data changes, the storage available and how quickly the system must be back.

Restoring systems

A backup is only useful if the organisation can restore from it efficiently, so the restore process must be planned to keep downtime to a minimum. With full backups, the existing files are simply replaced with the most recent full copy, which is straightforward and usually the quickest option. With differential backups, the last full backup is restored first and then the most recent differential is applied; unchanged files come from the full backup and changed files from the differential, which always holds the newest backed-up version of each changed file. With incremental backups, the last full backup is restored and then every incremental made since is applied in date order. A file may have changed several times, so it may appear in several incrementals, and applying them in order ensures that the latest version ends up in place. In every case the system is then at the state of the last restore point. Any work done after that point is lost unless it was captured in a transaction log, in which case the logged transactions are re-applied to bring the data fully up to date. Restores should be practised regularly so staff know the procedure and can be confident the backups actually work.

Disaster recovery plans

Disaster recovery is the process by which an organisation recovers its IT systems after a catastrophic event, something much more serious than a single failed disk or deleted file. Disasters may be accidental, such as a fire or widespread data corruption, natural, such as flooding or storm damage, or deliberate, such as a major cyber attack or terrorism. They cannot be prevented, so a disaster recovery plan does not try to avoid them; its purpose is to minimise their effects and allow the organisation to resume operating quickly with as little disruption to customers as possible. The plan should be drawn up when a system is introduced and kept up to date. At a minimum it identifies the critical data, the key personnel and their responsibilities, the backup and recovery processes, and a fallback location where work can carry on. The organisation then lists the systems that must be recovered, ranked by importance, and sets a time limit for recovering each one. Techniques to meet those limits include off-site or cloud backups, private online storage, cloud-hosted servers that can be activated on demand and staff on standby. A wider business continuity plan covers keeping the whole organisation running, not just its IT.

Recovery objectives and testing

The CCEA textbook describes the recovery point objectives as the list of parts of the system that must be restored after a disaster, and the recovery time objectives as the time limit for recovering each of them. A retailer might set 12 hours to restore its online shop but five days for payroll, because the shop earns money every hour while staff are paid monthly. In wider industry use, the recovery point objective is usually expressed as the maximum amount of data, measured in time, that the organisation can afford to lose; an RPO of one hour means backups or replication must happen at least hourly. Both views lead to the same practical point: the most critical functions get the highest priority and the most expensive protection. A plan that has never been tested may fail when it is needed, so testing is essential. In a walk-through or theoretical test, participants step through the plan to check that everyone knows their role and that contact details and procedures are correct. In a simulation test, working systems are deliberately disabled and actually recovered, so weaknesses such as missing backups or unrealistic time limits are found and fixed before a real disaster.

T Key terms from the mark schemes

Security policy
A formal document setting out the rules, procedures and responsibilities an organisation uses to protect its systems and data.

Multi-factor authentication (MFA)
Authentication requiring two or more different types of evidence, such as a password and a code sent to a phone.
`Credited: two-factor authentication (2FA)`

Firewall
Hardware or software that monitors network traffic and blocks data that does not meet a set of security rules.

Differential backup
A backup that copies every file altered after the most recent full backup, so it grows until the next full backup.

Disaster recovery plan
A documented plan setting out how an organisation will restore its IT systems and data and resume operating after a catastrophic incident such as fire, flood or a cyber attack.
`Credited: DR plan`

Recovery time objective (RTO)
The time limit within which each part of the system must be recovered after a disaster.
`Credited: RTO`

Authentication
The process of proving that a user is who they claim to be, using something they know, have or are.

Encryption
Converting plaintext into ciphertext using an algorithm and a key so that it cannot be understood without the correct key.

Full backup
A backup that copies every file on the system, whether or not it has changed, each time it runs.

Incremental backup
A backup that copies only files modified after the most recent backup of either kind, full or incremental.

Recovery point objective (RPO)
In the CCEA textbook, the list of parts of the system that must be restored after a disaster; more widely, the maximum amount of data, measured in time, that can acceptably be lost.
`Credited: RPO`

Downtime
The period during which a system is not operational and cannot be used.

Exam focus

Backup questions often ask you to describe how each method works and give advantages and disadvantages, so learn a precise definition of each: incremental copies changes since the last backup of any type, while differential copies changes since the last full backup. Mixing these up is the most common error. For evaluate questions, compare backup time, storage needed, restore time and the risk of a broken chain, then make a justified recommendation for the scenario. Recovery questions in a context, such as a failure part-way through the week, reward a clear sequence: restore the last full backup, then the relevant differential or each incremental in order, state which point in time the system is now at, then re-apply logged transactions. For disaster recovery plans, the four contents usually credited are the critical data, the key staff, how backups are taken and restored, and a fallback site. When defining RPO and RTO, give the textbook meaning and support it with an example. For security measures, say how the measure works and which threat it counters, and remember that encryption makes stolen data unreadable rather than preventing theft.

? Check yourself

1. State three items that would be included in an organisation's security policy. [3]
2. A Northern Ireland bank asks customers to enter a password and then a code sent to their phone. Explain how this multi-factor authentication improves security. [2]
3. Describe the difference between symmetric and asymmetric encryption. [2]

Answers: 1. Any three from: acceptable use of equipment and internet (1) / password rules (1) / access levels for different staff (1) / use of personal devices and removable media (1) / remote working rules (1) / procedure for reporting incidents (1) / backup procedures (1) / staff training (1) / disciplinary consequences of breaches (1). 2. It requires two different types of evidence, something the customer knows and something they have (1); so a criminal who has stolen or phished the password still cannot log in without the customer's phone (1). 3. Symmetric encryption uses the same key to encrypt and decrypt (1); asymmetric uses a public key to encrypt and a different, matching private key to decrypt (1).

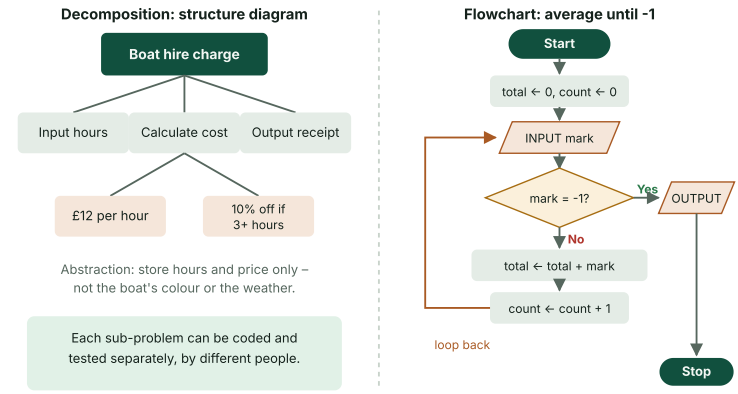
Programs, algorithms and their design

AP16 FACT FILE

Specification: students should be able to

- Describe the purpose of a computer program
- Explain the term algorithm and the properties a good algorithm must have
- Explain decomposition and abstraction as techniques for solving problems
- Describe how algorithms can be represented using pseudocode and flowcharts
- Use trace tables and dry runs to test an algorithm and find logic errors

Key words: computer program, application program, algorithm, finiteness, unambiguous, decomposition, abstraction, pseudocode, structured English, flowchart, input-process-output, sequence, selection, iteration, sentinel value, trace table, dry run, logic error



Decomposition of a boat-hire problem into a structure diagram, beside a flowchart of a sentinel-controlled averaging loop

From problem to program

A computer program is a set of instructions that a computer follows to carry out a task. Programs are software, which distinguishes them from the hardware they run on. Some software, such as the operating system, provides an interface so that people can use and control the computer; application programs, such as word processors, spreadsheets or a sports club's booking app, are designed to carry out a particular job for the user. Before any code is written, the programmer must work out exactly how the task will be done. That plan is an algorithm: a finite, ordered set of precise and unambiguous steps that solves a problem. Algorithms are not unique to computing – a recipe or a set of directions is an algorithm – but a computer is far less forgiving than a person, because it cannot use common sense to fill gaps or resolve ambiguity. Designing the algorithm first, using decomposition and abstraction, and expressing it in pseudocode or as a flowchart, allows the logic to be checked with a dry run before time is spent coding. Logic errors found at this stage are far cheaper and quicker to fix than errors found after the program has been written and released.

What makes a good algorithm

A useful algorithm has several properties. Its steps are in a specific order, and that order matters: outputting a total before it has been calculated gives the wrong answer. Each step must be precise, so 'wait a few minutes' is replaced by 'wait 5 minutes', and unambiguous, so that it can be interpreted in only one way. An algorithm must be finite: it must terminate after a limited number of steps, which means every loop needs a condition that will eventually be met. An instruction such as 'repeat until the cake rises' might never end. An algorithm takes zero or more inputs, may process them, and produces at least one output that is related to those inputs. It must be correct and effective, working for every valid case and dealing sensibly with invalid input, for example by rejecting a negative quantity with a clear message rather than crashing. Finally, it should be efficient, avoiding unnecessary steps or slow methods when a quicker one would do the job equally well. When asked to evaluate an algorithm, test it against each of these properties in turn.

Decomposition and abstraction

Computational thinking starts with decomposition: splitting a large problem into smaller, manageable sub-problems that can be designed, coded and tested separately, often by different programmers working at the same time. A structure diagram shows this top-down breakdown, with the whole problem at the top and progressively smaller tasks beneath it. Each sub-problem becomes a module, such as a procedure or function, which can be reused elsewhere and is easier to maintain. Abstraction removes detail that is not needed for the problem, producing a simpler model. For example, a cinema booking system stores seat numbers, prices and screening times but not the colour of the seats, and a bus route map shows stops and connections rather than every bend in the road. Abstraction reduces the amount of data to be stored and processed and lets designers concentrate on the parts that matter. Used together, the two techniques make problems easier to understand, reduce development time, allow work to be shared across a team and lead to modular code that is simpler to test and debug.

Representing algorithms

An algorithm can be written in plain English, which has the advantage that a client with no programming experience can read it and confirm that it describes the right process. Algorithms that are going to become programs are usually written in structured English or pseudocode instead, which use keywords and indentation that resemble program code. These are more helpful to the programmer, though slightly harder for a non-specialist to follow. Pseudocode has no single official syntax; what matters is that it is consistent and unambiguous and shows sequence, selection and iteration clearly. A flowchart represents the same logic as a diagram using standard symbols: a terminator for START and END, a rectangle for a process, a parallelogram for input or output, a diamond for a decision with two labelled exits, and arrows for the flow of control. A loop appears as an arrow returning to an earlier decision. Flowcharts are visual and easy to follow for small problems but become unwieldy for large ones, whereas pseudocode is quicker to write and translates more directly into code. Both are language-independent.

Building blocks and loops

Every algorithm is built from three constructs. Sequence means steps are carried out one after another in order. Selection means a decision is made and different steps are followed depending on whether a condition is true or false, as in IF...THEN...ELSE. Iteration means a set of steps is repeated. A count-controlled loop, such as FOR day ← 1 TO 4, is used when the number of repetitions is known before the loop starts. A condition-controlled loop, such as WHILE or REPEAT...UNTIL, is used when it is not, for example reading exam marks until the user enters a sentinel value of -1. A WHILE loop tests its condition at the start, so its body may run zero times; a REPEAT...UNTIL loop tests at the end, so its body always runs at least once. Variables used to accumulate a total or count occurrences must be initialised, usually to zero, before the loop begins, and must not be reset inside it. In a flowchart, both kinds of loop are drawn with a decision diamond and a flow line back to an earlier point.

Testing a design with a trace table

A dry run tests an algorithm by working through it by hand, one step at a time, without a computer. The results are recorded in a trace table, which has a column for each variable, often a column for any condition being tested, and a column for output. A new row is added each time values change, so every pass of a loop can be seen. Because the dry run follows the algorithm exactly as it is written, rather than as the designer intended it to work, it exposes logic errors: a total that is never initialised, a counter reset inside a loop, a loop that runs one time too many or too few because < has been used instead of <=, or a missing input that causes an infinite loop. Logic errors do not stop a program running, so they are not caught by the compiler; only careful testing reveals them. Good test data includes normal values, boundary values at the edge of a condition, such as exactly 10 mm of rain when the test is greater than 10, and erroneous values that should be rejected.

T Key terms from the mark schemes

Computer program
A set of instructions that a computer executes in order to perform a specific task.
`Credited: software that carries out a task`

Application program
Software designed to perform a specific task for the user, such as a word processor or spreadsheet.
`Credited: application software`

Algorithm
A finite, ordered sequence of precise, unambiguous steps designed to solve a problem or complete a task.
Credited: a step-by-step set of instructions to solve a problem

Finiteness
The property that an algorithm must terminate after a limited number of steps.
Credited: the algorithm must end

Decomposition
Breaking a complex problem into smaller sub-problems that can be solved independently.
Credited: top-down design

Abstraction
Removing unnecessary detail from a problem so that only the relevant details remain.

Pseudocode
A structured, language-independent, English-like way of describing an algorithm that resembles program code.
Credited: structured English

Flowchart
A diagram that represents an algorithm using standard symbols joined by flow lines showing the order of steps.

Trace table
A table used during a dry run to record the value of each variable and any output at each step of an algorithm.

Dry run
Working through an algorithm or program by hand, step by step, to check its logic.
Credited: desk check

Sentinel value
A special input value, such as -1, that signals the end of the data and stops a condition-controlled loop.
Credited: rogue value

Logic error
An error in the design of an algorithm that causes incorrect output although the program still runs.

C C# example

The rainfall algorithm in C#

```
int total = 0, wetDays = 0;
for (int day = 1; day <= 4; day++)
{
    Console.Write($"Rainfall for day {day} (mm): ");
    int rain = int.Parse(Console.ReadLine());
    total += rain;
    if (rain > 10)
    {
        wetDays++;
    }
}
Console.WriteLine($"Total rainfall: {total} mm");
Console.WriteLine($"Wet days: {wetDays}");
```

Exam focus

Short questions ask you to state or define: an algorithm is a sequence of precise, unambiguous steps to solve a problem, and a fully credited answer usually includes the idea of order and of solving a problem. For 'describe the purpose of a program' say it is a set of instructions that the computer carries out to perform a specific task. Properties questions usually award one mark per property named and one for an explanation or example, so pair each property with how it applies to the scenario given. When asked to write an algorithm, examiners credit logic rather than exact syntax: initialising totals and counters, a correctly controlled loop, inputs inside the loop, the right condition in the decision and an output after the loop. Flowcharts must use the correct symbols and label both exits of every decision. In trace table questions, follow the algorithm exactly as written, complete every row and do not skip a pass of the loop. Common mistakes include confusing decomposition with abstraction, forgetting the final output, using one exit from a decision and placing the output inside the loop so it prints every time.

? Check yourself

1. A Year 8 pupil says that a program and the computer it runs on are 'the same thing'. Describe the purpose of a computer program, explaining why it is different from the hardware. [2]
2. Before coding a lunch-ordering app for a Strabane primary school, a developer writes an algorithm. Explain what is meant by an algorithm. [2]
3. An algorithm is written to guide visitors around Castle Coole. Explain three properties that the algorithm must have to be useful. [6]

Answers: 1. A set of instructions (1) / executed by the computer to carry out a specific task or solve a problem; it is software, separate from the physical hardware that runs it (1) 2. A sequence of (step-by-step) instructions (1) / that are precise/unambiguous and in a specific order, designed to solve a problem/complete a task (1) 3. Any three properties, each identified (1) and explained in context (1), e.g.: steps in a specific order (1) so visitors see rooms in the correct route (1) / each step precise (1) e.g. 'walk 20 metres to the staircase' not 'walk a bit' (1) / unambiguous (1) e.g. naming which door when there are several (1) / finite (1) the tour must end, e.g. at the exit (1) / has inputs and at least one output (1) e.g. visitor's start point in, route out (1) / correct and effective (1) deals with a closed room by giving an alternative (1) / efficient (1) no unnecessary doubling back (1)

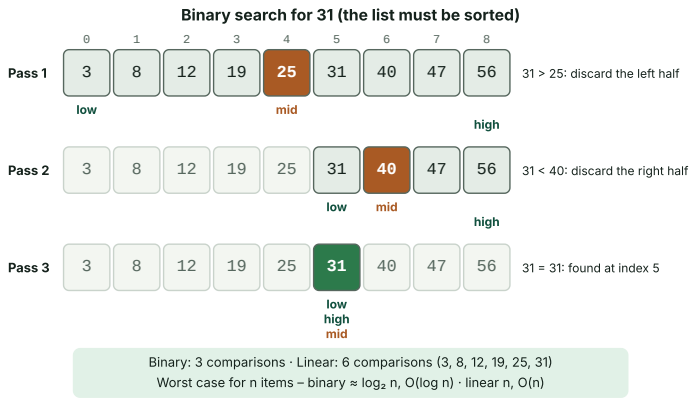
Searching algorithms

AP17 FACT FILE

Specification: students should be able to

- Describe how a linear search and a binary search work, using pseudocode, flowcharts and trace tables
- Implement linear search and binary search in C#
- Compare the efficiency of searching algorithms in terms of the number of comparisons in the best, average and worst cases
- Explain the conditions under which each search is appropriate

Key words: linear search, binary search, target, comparison, midpoint, sorted, integer division, divide and conquer, best case, average case, worst case, efficiency, time complexity, big-O notation, O(n), O(log n)



Binary search for 31 in a sorted array of nine items: each pass compares the middle item and discards half, finding it in 3 comparisons compared with 6 for a linear search

Searching: the big picture

Searching is one of the most common tasks a program carries out: checking a username at login, finding a product by its barcode or retrieving a patient's record in a hospital system. A search algorithm takes a collection of data and a target value and reports whether the target is present and, usually, its position. The two algorithms required are the linear search and the binary search. A linear search is simple and works on any list, sorted or not, but it may have to look at every item. A binary search is much faster on large lists because it discards half of the remaining data with each comparison, but it only works if the data is already sorted and can be accessed directly by position. Choosing between them is a classic trade-off between simplicity and speed. Efficiency is measured by counting comparisons, not by timing a program, because the count does not depend on the speed of a particular computer. The number of comparisons in the best, average and worst cases, and how that number grows as the list gets longer, is summarised using big-O notation.

Linear search

A linear, or sequential, search examines each item in turn from the start of the list, comparing it with the target. If they match, the search stops and returns the position; if the end of the list is reached without a match, the target is not present and a value such as -1 or a 'not found' message is returned. A well-written linear search stops as soon as the item is found, rather than carrying on to the end. Its strengths are simplicity and flexibility: it is easy to code correctly, works on unsorted data and can be used on data that can only be read in order, such as records in a serial file. Its weakness is speed. In the best case the target is the first item and only one comparison is needed, but in the worst case, when the target is last or missing, every item must be compared, so n comparisons are needed for a list of n items. On average about half the list is checked. The time grows in direct proportion to the size of the list, which is described as O(n), or linear time.

Binary search

A binary search works only on sorted data. Two pointers, low and high, mark the section of the list still being searched, starting at the first and last positions. The middle position is calculated as (low + high) DIV 2, using integer division so that it rounds down. The item at that position is compared with the target. If they are equal, the search ends. If the target is larger, it can only be in the upper half, so low is moved to mid + 1; if it is smaller, high is moved to mid − 1. The process repeats on the remaining half. If low becomes greater than high, no items are left to check and the target is not in the list. Because every comparison halves the search space, binary search is described as divide and conquer. Its best case is one comparison, when the target happens to be in the middle. Its worst case for n items is $\lfloor \log_2 n \rfloor + 1$ comparisons, so 1 000 items need at most 10 comparisons and a million need at most 20. This is O(log n), or logarithmic time.

Tracing a binary search

To trace a binary search, draw a table with columns for the pass number, low, high, mid, the item at mid and the action taken. Start with low at 0 and high at the last index. On each pass, calculate mid with integer division, look up the item, and decide whether it matches, whether low moves up to mid + 1, or whether high moves down to mid − 1. For the sorted list 2, 5, 9, 14, 21, 27, 33, 40, 46, 52, searching for 46 gives mid values of 4, 7 and 8, comparing with 21, 40 and 46, so the target is found at index 8 after three comparisons. Searching for 10 compares with 21, 5, 9 and 14; after the fourth comparison high becomes 2 while low is 3, so low is greater than high and the search reports that 10 is not present. Note that different textbooks round the midpoint differently or count positions from 1 rather than 0; in the exam, follow the algorithm you are given exactly and show your working so that the method can be credited.

Measuring efficiency

The efficiency of an algorithm is judged by how the number of steps it takes (time complexity) and the memory it uses (space complexity) grow as the size of the input increases, rather than by timing it on one computer, which would depend on the hardware. Analysts consider the best case, the average case and the worst case. Big-O notation expresses the order of growth, usually for the worst case, and ignores constant factors and smaller terms, because for large inputs only the dominant term matters; an algorithm that takes 3n + 2 steps is O(n). Common orders, from most to least efficient for large n, are O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n) and $O(n^2)$ quadratic. A logarithmic algorithm scales extremely well: doubling the size of a sorted list adds only one extra comparison to a binary search, whereas it doubles the worst-case work of a linear search. Both searches use only a few extra variables, so their space requirements are small and similar; the important difference between them is time.

Choosing a search

Binary search is not always the right choice. It requires the data to be sorted, and sorting has a cost: if a list is searched only once or twice, or if new items are added constantly in random order, the time spent sorting or keeping the list in order may outweigh the time saved by searching faster. It also needs direct access to any position in the list, so it suits arrays held in memory rather than data read one record at a time. For short lists, perhaps a few dozen items, the difference in speed is negligible and a linear search is simpler to write, test and maintain, with less risk of off-by-one errors in the low, high and mid calculations. Binary search is the better choice when a large data set is sorted, or can be kept sorted, and is searched frequently, such as a library catalogue, a dictionary or a product database searched thousands of times a day. In practice, large systems often use indexes or other structures, but the principle of reducing the number of comparisons is the same.

T Key terms from the mark schemes

Linear search
A search that checks each item in turn from the start until the target is found or the end of the list is reached.
`Credited: sequential search`

Binary search
A search of a sorted list that repeatedly compares the target with the middle item and discards the half that cannot contain it.

Divide and conquer
An approach that solves a problem by repeatedly reducing it to smaller sub-problems, as when binary search halves the list each time.

Time complexity
A measure of how the number of steps an algorithm takes grows as the size of its input grows.

Big-O notation
Notation that describes the order of growth of an algorithm's time or space, usually in the worst case, as input size increases, e.g. O(n).

Best case
The input that makes an algorithm do the least work, such as the target being the first item checked.

Logarithmic time
O(log n) growth, in which doubling the input size adds only a constant amount of extra work.
Credited: O(log n)

Worst case
The input that makes an algorithm do the most work, such as the target being last or absent in a linear search.

Integer division
Division that discards any remainder, so 13 DIV 2 = 6; used to calculate the midpoint in a binary search.
Credited: DIV

C C# example

Linear search in C#

```
static int LinearSearch(string[] list, string target)
{
    for (int i = 0; i < list.Length; i++)
    {
        if (list[i] == target) return i;
    }
    return -1;
}

string[] towns = { "Omagh", "Derry", "Belfast", "Enniskillen", "Newry" };
Console.WriteLine(LinearSearch(towns, "Enniskillen"));   // 3
Console.WriteLine(LinearSearch(towns, "Armagh"));        // -1
```

Exam focus

When asked to describe a binary search, examiners look for a sequence of clear points: the list must be sorted; find the middle item; compare it with the target; if it matches, stop; otherwise discard the half that cannot contain the target; repeat on the remaining half until the item is found or no items remain. Each point is typically worth one mark, so do not skip the stopping conditions. For a linear search, state that each item is checked in turn from the start until it is found or the end is reached. Trace questions may ask you to list the items compared, in order, or to complete a table of low, high and mid; show every pass and remember that integer division rounds down. Calculations of the maximum number of comparisons should show the halving or the power of two used. For 'compare' or 'evaluate' questions, make each point two-sided (linear needs no sorting, whereas binary does) and finish with a justified recommendation for the scenario. A common mistake is saying binary search is always better without mentioning the need for sorted data.

? Check yourself

1. State one condition that must be met before a binary search can be used. [1]
2. Describe how a linear search finds an item in a list. [3]
3. Describe how a binary search finds an item in a list. [5]

Answers: 1. The list/data must be sorted (into order) (1) 2. Start at the first item (1) / compare each item in turn with the target (1) / stop when the item is found or the end of the list is reached (target not present) (1) 3. List must be sorted (1) / find/compare the middle item with the target (1) / if equal, the item is found and the search stops (1) / if the target is less than the middle item, discard the upper half (or if greater, discard the lower half) (1) / repeat with the remaining half until found or no items remain (1)

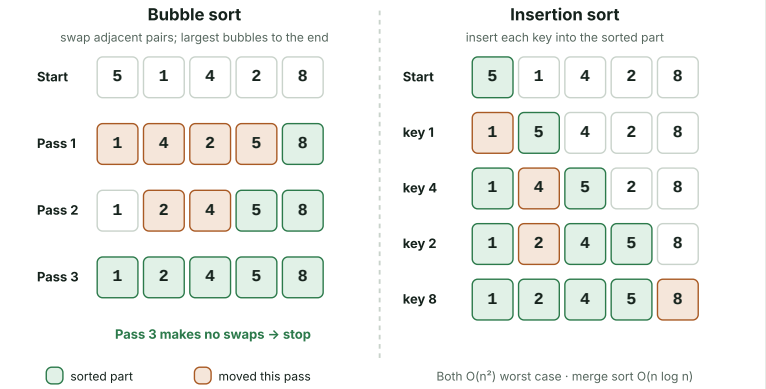
Sorting algorithms

AP18 FACT FILE

Specification: students should be able to

- Describe, trace and implement bubble sort and insertion sort using pseudocode and C#
- Describe the principles of merge sort and quick sort as divide-and-conquer algorithms
- Compare sorting algorithms in terms of time efficiency, memory use and suitability for a given data set
- Explain how design features such as a swapped flag improve the efficiency of an algorithm

Key words: bubble sort, insertion sort, merge sort, quick sort, pass, adjacent, swap, temporary variable, swapped flag, key, nested loop, divide and conquer, pivot, partition, in place, $O(n^2)$, $O(n \log n)$



Bubble sort and insertion sort of 5 1 4 2 8 shown pass by pass side by side

Sorting: the big picture

Sorting arranges data into ascending or descending order, numerically or alphabetically. It is one of the most common tasks in computing because sorted data is easier for people to read, as in a league table or class list, and easier for programs to process: a sorted list can be searched with a fast binary search, and duplicates, the largest value or the median can be found quickly. The AS course requires four sorting algorithms. Bubble sort and insertion sort are simple algorithms built from nested loops; they are easy to understand, trace and code, and sort the data in place, but the number of comparisons grows with the square of the number of items, so they become very slow for large lists. Merge sort and quick sort use a divide-and-conquer approach, repeatedly breaking the list into smaller parts and combining the results; they are harder to code, usually using recursion, but for large lists they are dramatically faster. Choosing between them means weighing time efficiency, memory use, the size and state of the data and how easy the code is to write and maintain.

Bubble sort

Bubble sort works through the list comparing each pair of adjacent items and swapping them if they are in the wrong order. One complete run through the list is called a pass. In an ascending sort the largest value is carried to the end of the list during the first pass, so it is in its final position, and each later pass can stop one item earlier. Swapping two array elements requires a temporary variable: the first value is copied to temp, the second is copied into the first position and temp is copied into the second; without temp one of the values would be overwritten and lost. A Boolean flag, often called swapped, is set to false at the start of each pass and to true whenever a swap is made. If a whole pass makes no swaps the list must be sorted, so the algorithm stops. This means an already sorted list needs only one pass of $n - 1$ comparisons, giving a best case of $O(n)$. In the worst case, a list in reverse order, about $n^2/2$ comparisons are needed, so the average and worst cases are $O(n^2)$.

Insertion sort

Insertion sort builds a sorted section at the start of the list, much as a card player arranges a hand of cards one at a time. Initially the first item on its own is treated as sorted. On each pass the next unsorted item is copied into a variable called the key. Working from right to left through the sorted section, every item larger than the key is shifted one place to the right, and when a smaller item or the start of the list is reached, the key is inserted into the gap. The sorted section therefore grows by one item per pass, and a list of n items needs n − 1 passes. Insertion sort sorts in place, needing only one extra variable. If the list is already sorted, each key is compared once and nothing moves, so the best case is O(n); if the list is in reverse order every key must move to the front, giving a worst case of $O(n^2)$. It usually makes fewer comparisons than bubble sort and is very effective for small lists, nearly sorted lists and adding a few new items to a list that is already in order.

Tracing a sort step by step

Sorting questions often ask you to show the state of a list after each pass, so careful tracing is essential. For a bubble sort, work along the list one adjacent pair at a time, making each swap before moving to the next pair; a common error is to rearrange several items at once. For the netball points 14, 9, 22, 5, 17, 11, the first pass swaps 14 and 9, leaves 14 and 22, then carries 22 past 5, 17 and 11 to give 9, 14, 5, 17, 11, 22. The second pass gives 9, 5, 14, 11, 17, 22 and the third gives 5, 9, 11, 14, 17, 22. A fourth pass makes no swaps, so the sort stops after 14 comparisons. For an insertion sort of the same list, record the key on each pass and how many items shift: 9 moves before 14, 22 stays put, 5 moves to the front past three items, 17 moves past 22, and 11 moves past 22, 17 and 14. Both traces end with the same sorted list, but the intermediate lists are different, which is how an examiner can tell which algorithm was applied.

Divide and conquer: merge sort

Merge sort repeatedly divides a list in half until every sub-list contains a single item, which is sorted by definition. It then merges the sub-lists back together in pairs. Merging two sorted lists is straightforward: compare the front items of each, move the smaller one into the new list, and repeat until one list is empty, then copy across whatever remains of the other. For example, 6, 2, 9, 4, 7, 1, 8, 3 is divided into eight single items, merged into the pairs [2 6], [4 9], [1 7] and [3 8], then into [2 4 6 9] and [1 3 7 8], and finally into 1, 2, 3, 4, 6, 7, 8, 9. Splitting a list of n items in half repeatedly produces about $\log_2 n$ levels, and merging at each level handles all n items, so merge sort takes O(n log n) time in the best, average and worst cases. This guaranteed performance makes it predictable for large data sets. Its main drawback is memory: the sub-lists must be stored while merging, so it needs extra space roughly equal to the size of the list.

Divide and conquer: quick sort

Quick sort chooses one item, the pivot, and partitions the rest of the list so that items smaller than the pivot are placed to its left and items larger than it to its right. The pivot is then in its final sorted position. The same process is applied recursively to the left and right sub-lists until every sub-list has zero or one item. Using the first item as the pivot, 6, 2, 9, 4, 7, 1, 8, 3 partitions around 6 into [2 4 1 3] and [9 7 8]; the left part partitions around 2 into [1] and [4 3], and so on until the whole list is sorted. When the pivot divides each list roughly in half, quick sort takes O(n log n) time and, because partitioning can be done within the original array, it needs little extra memory; in practice it is often the fastest general-purpose sort. However, if the pivot is consistently the smallest or largest item, for example choosing the first item of an already sorted list, each partition removes only one item and the time degrades to $O(n^2)$. Choosing the middle item, or a random item, as the pivot reduces this risk.

Key terms from the mark schemes

Bubble sort
A sort that repeatedly compares adjacent items and swaps them if they are in the wrong order until a pass makes no swaps.

Insertion sort
A sort that takes each item in turn and inserts it into its correct position within an already sorted section of the list.

Merge sort
A divide-and-conquer sort that splits a list into single items and then merges sub-lists back together in order.

Quick sort
A divide-and-conquer sort that partitions items around a pivot and then recursively sorts each partition.

Divide and conquer
An approach that solves a problem by repeatedly breaking it into smaller sub-problems of the same type and combining their solutions.

Pass
One complete traversal through the list (or the unsorted part of it) by a sorting algorithm.

Pivot
The item chosen in quick sort around which the other items are partitioned into smaller and larger groups.

Swapped flag
A Boolean variable in a bubble sort that records whether any swap was made during a pass, so the sort can stop early.
Credited: Boolean flag

Temporary variable
A variable used to hold one value during a swap so that it is not overwritten and lost.
Credited: temp

In place
Describes a sort that rearranges items within the original array, needing little or no extra memory.

C C# example

Bubble sort in C#

```
int[] points = { 14, 9, 22, 5, 17, 11 };
bool swapped;
int end = points.Length - 1;
do
{
    swapped = false;
    for (int i = 0; i < end; i++)
    {
        if (points[i] > points[i + 1])
        {
            int temp = points[i];
            points[i] = points[i + 1];
            points[i + 1] = temp;
            swapped = true;
        }
    }
    end--;
} while (swapped);
Console.WriteLine(string.Join(" ", points));   // 5 9 11 14 17 22
```

Exam focus

For trace questions, show the list after every pass, not just the final result, and follow the algorithm exactly as given; if asked for a specific pass, count carefully, since the first pass is often already shown. When describing bubble sort, credited points include comparing adjacent items, swapping them if they are in the wrong order, moving along to the end of the list, repeating passes and stopping when a pass makes no swaps. For insertion sort, mention taking each item in turn, comparing it with the sorted part, shifting larger items and inserting it in the correct place. Merge sort descriptions need both the splitting into single items and the merging of sorted sub-lists; quick sort descriptions need the pivot, the partition and recursion on each side. Explain questions about the temporary variable or the swapped flag carry two marks: what it does and why it matters. For 'compare' or 'evaluate' questions, use time efficiency for large n, memory use, behaviour on nearly sorted data and ease of coding, and finish with a justified choice for the scenario rather than a list of facts.

? Check yourself

1. Show the list 6, 3, 8, 2 after each pass of a bubble sort into ascending order. [3]
2. Show the list 6, 3, 8, 2 after each pass of an insertion sort into ascending order. [3]
3. Explain why a temporary variable is needed when swapping two items in an array. [2]

Answers: 1. Pass 1: 3 6 2 8 (1) / Pass 2: 3 2 6 8 (1) / Pass 3: 2 3 6 8 (1) 2. Pass 1: 3 6 8 2 (1) / Pass 2: 3 6 8 2 (1) / Pass 3: 2 3 6 8 (1) 3. When the first value is overwritten by the second, the original first value would be lost (1) / so it is copied to a temporary variable first and then copied into the second position (1)

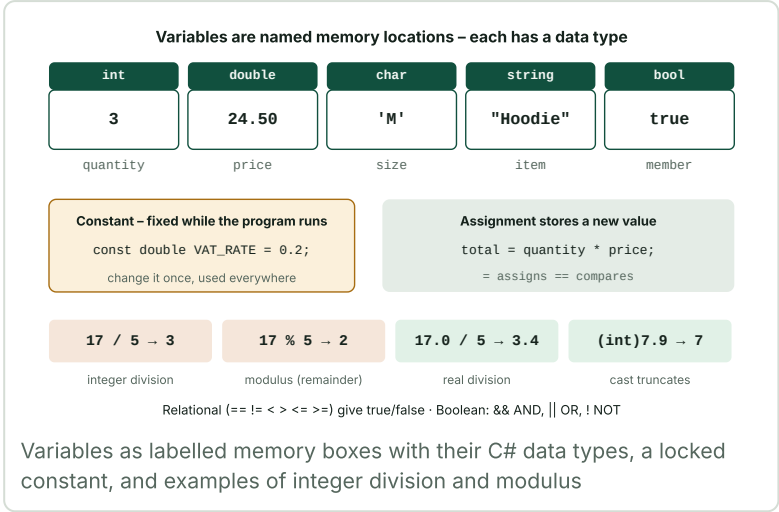
IDEs, translators, data types, variables and operators

AP19 FACT FILE

Specification: students should be able to

- Describe the main features of an integrated development environment (IDE), including the editor, preferences, compiler and debugging tools
- Explain the process of translation and the differences between assemblers, compilers and interpreters
- Explain the terms syntax, data type and variable, and use the data types integer, real, character, string and Boolean appropriately
- Explain the use of variables and constants, and declare, initialise and assign them in a high-level language
- Use arithmetic, relational and Boolean (logical) operators, including integer division and modulus, and evaluate conditions
- Write programs that take input from the user, convert it to a suitable type and produce formatted output

Key words: IDE, source code, object code, syntax, translator, assembler, compiler, interpreter, data type, variable, identifier, constant, assignment, casting, modulus, integer division, relational operator, Boolean operator



Variables as labelled memory boxes with their C# data types, a locked constant, and examples of integer division and modulus

Overview: from algorithm to running program

Once an algorithm has been designed, a programmer expresses it in a programming language such as C#. The result is source code: text that people can read and edit but that the processor cannot execute directly. Programmers usually work inside an integrated development environment (IDE), which combines an editor, a translator and debugging tools in one application so that code can be written, checked, run and corrected quickly. Before the program can run, its source code must be translated into machine code, the binary instructions the processor understands; this job is done by an assembler, a compiler or an interpreter, each working in a different way. Inside the program, data is held in variables. Each variable has an identifier and a data type, which decides what values it can hold, how much memory is reserved and which operations make sense. Values that must never change are declared as constants. Operators then do the work: arithmetic operators calculate, relational operators compare values to give true or false, and Boolean operators combine those conditions. Finally, programs communicate with users through input and output, converting text typed at the keyboard into numbers and formatting results neatly on screen. These ideas are the foundation for every program you will write in the AS course.

The integrated development environment

An IDE is a software application that gives programmers a complete set of tools for developing software. Its editor lets code be written, edited and saved, and most editors are smart: syntax highlighting colours keywords, strings and comments differently, automatic indentation shows which statements belong inside which blocks, and line numbers and code folding make long programs easier to navigate. A syntax checker underlines invalid code as it is typed, and code completion (called IntelliSense in Visual Studio) suggests keywords and identifiers after a few letters, improving speed and reducing spelling errors. Preferences allow a team to agree layout rules, such as whether an opening brace goes on its own line, and the IDE can apply them automatically so everyone's code looks the same when it is combined. The IDE also contains or connects to a compiler, so the program can be built and run with a single command, and any translation errors are listed with their line numbers. Finally, debugging tools allow the programmer to set breakpoints, step through code one line at a time and watch the values of variables, which makes locating errors far easier. Debugging techniques are covered in more detail in AP23.

Translators

Translation is the conversion of a program from the language it was written in into machine code that the processor can execute. An assembler translates assembly language, a low-level language in which each mnemonic, such as LDA or ADD, corresponds to exactly one machine code instruction. High-level languages are translated by compilers or interpreters. A compiler translates the whole program before any of it runs. If errors are found, it produces an error report and no executable is created until every error is corrected, after which the program must be compiled again. Because the compiler sees the complete program, it can optimise the object code, so the finished program runs quickly and can be distributed without its source code. An interpreter translates one statement, executes it, then moves to the next. It produces no object code, so the source code and interpreter are needed every time the program runs, and execution is slower. Errors are only discovered when the faulty line is reached, which may be after earlier statements have already changed files or data. However, interpreters give immediate feedback, which makes them useful during development and debugging. C# combines both ideas: source code is compiled to an intermediate language, which the .NET runtime compiles to machine code just in time.

Variables, identifiers and constants

A variable is a named location in memory that stores a single value which can change while the program is running. Its name is called its identifier. Most languages, including C#, require variables to be declared before use: the declaration states the data type and identifier so that the right amount of memory can be reserved, and it may also give an initial value. Assignment, using a single equals sign, replaces the current value with a new one. Identifiers must follow the language's rules: in C# they begin with a letter or underscore, contain only letters, digits and underscores, cannot contain spaces, cannot be reserved keywords and are case-sensitive, so total and Total are different variables. Conventions are not enforced by the compiler but should be followed so that code is readable: use meaningful names such as ticketPrice rather than tp, and write variables in camelCase. A constant is a named value fixed when the program is written, declared in C# with const. Using a constant for a value such as a VAT rate or a maximum group size means it cannot be changed accidentally while the program runs, its purpose is clear from its name, and if it ever needs updating it is changed in one place only.

Choosing data types

A data type defines what kind of value a variable can hold, how much memory it needs and which operations can be performed on it. C# uses int for whole numbers such as counts and ages, and double for real numbers such as measurements and averages. For money, decimal is better than double because it stores decimal fractions exactly, avoiding small binary rounding errors. A char holds exactly one character in single quotes, a string holds any sequence of characters in double quotes, and a bool holds only true or false. The choice should match how the data will be used, not just what it looks like. A telephone number or a membership code made of digits should be a string, because it may start with a zero, may contain spaces and will never be used in arithmetic. A yes/no fact, such as whether a fee has been paid, should be Boolean because there are only two possible states. Choosing the right type saves memory, prevents meaningless operations and lets the compiler detect mistakes, such as trying to store text in an integer. Converting between types is called casting: (int)7.9 truncates to 7, while (double)total forces real rather than integer division.

Operators, expressions and conditions

Arithmetic operators in C# are +, -, *, / and %. When both operands of / are integers, C# performs integer division and discards the remainder, so 17 / 5 gives 3; if either operand is real, the result is real, so 17.0 / 5 gives 3.4. The modulus operator % gives the remainder, so 17 % 5 gives 2. Together, DIV and MOD split a quantity into whole units and what is left, for example converting minutes into hours and minutes, and n % 2 == 0 tests whether a number is even. Shorthand operators include ++ to add one and += to add a value to a variable. Relational operators (==, !=, <, <=, >, >=) compare two values and always produce a Boolean result. Boolean operators combine conditions: && (AND) is true only when both sides are true, || (OR) is true when either or both are true, and ! (NOT) reverses a value. To evaluate a condition by hand, substitute the values, work out any brackets first, replace each comparison with true or false, then apply the Boolean operators. Brackets make the intended order clear, since && is evaluated before ||. Watch for the classic slip of writing = (assignment) when == (comparison) is meant.

T Key terms from the mark schemes

Data type
A classification of data that determines the values a variable can hold, the memory it needs and the operations that can be performed on it.

Variable
A named location in memory that stores a single value which can change while the program runs.
`Credited: named storage location`

Constant
A named value that is set when the program is written and cannot change while the program runs.

Identifier
The name given to a variable, constant or subroutine so that it can be referred to in a program.
`Credited: variable name`

Syntax
The set of rules that define how valid statements must be written in a programming language.
`Credited: grammar of the language`

Integrated development environment (IDE)
A software application that provides the tools needed to develop programs, such as an editor, translator and debugging tools, in one place.
`Credited: IDE`

Assembler
A translator that converts assembly language into machine code, each mnemonic usually becoming one machine code instruction.
`Credited: one-to-one translation`

Compiler
A translator that converts the whole of a high-level program into object code before it is executed, reporting any errors found.

Interpreter
A translator that translates and executes a high-level program one statement at a time, producing no object code.

Modulus
An operator (% in C#, MOD in pseudocode) that gives the remainder after integer division.
`Credited: MOD`

Integer division
Division that gives only the whole-number part of the result, discarding any remainder (DIV).
`Credited: DIV`

Casting
Converting a value from one data type to another, e.g. (double)total or (int)7.9.
`Credited: type conversion`

C C# example

`Declaring, assigning and constants`

```
const double VAT_RATE = 0.2;
string item = "Hoodie";
int quantity = 3;
double price = 24.50;
bool member = true;
char size = 'M';
double total = quantity * price;        // 73.5
if (member) total = total * 0.9;        // 10% member discount
total = total + total * VAT_RATE;
Console.WriteLine($"{quantity} x {item} ({size}): £{total:F2}");
// Output: 3 x Hoodie (M): £79.38
```

Exam focus

Questions on this lesson use the command words state, describe and explain. To describe IDE features, name the feature and say how it helps, for example 'syntax highlighting colours keywords so the structure is easier to read' rather than just listing tools; three features are typically worth one or two marks each. For translators, mark schemes credit clear contrasts: a compiler translates the whole program before execution and produces an executable, while an interpreter translates and executes one statement at a time and produces no object code; an assembler translates assembly language with a one-to-one mapping. Avoid saying an interpreter 'has no errors' or that a compiler 'runs the program line by line'. When asked to define a variable, include that it is a named memory location whose value can change during execution. For data type questions, give the type and, when asked, a reason tied to how the data is used: phone numbers are strings because of the leading zero and no arithmetic. When evaluating expressions, show the substituted values, take care with integer division and MOD, and give Boolean results as true or false. Common mistakes are using a real type for a count, and confusing = with ==.

? Check yourself

1. A Belfast software start-up is choosing an IDE for its new apprentices. Describe three features of an IDE that would help the apprentices write correct source code. [6]
2. An apprentice says the C# file they typed 'is the program the CPU runs'. State what is meant by translation, and why it is needed before the program can run. [2]
3. Explain two differences between a compiler and an interpreter. [4]

Answers: 1. Any three features, each with description of how it helps (2 marks each): syntax highlighting / colour coding (1) makes keywords and structure easier to read (1) / automatic indentation (1) shows which statements belong to which block (1) / syntax checking as you type (1) underlines errors such as a missing semicolon so they are fixed before compiling (1) / code completion or IntelliSense (1) suggests identifiers, reducing spelling mistakes (1) / preferences or auto-formatting (1) keep a team's layout consistent (1) / debugging tools such as breakpoints and stepping (1) let the programmer inspect variables to locate errors (1) 2. Converting a program from the language it is written in (source code, e.g. C#) (1) into machine code / object code that the processor can execute, because the CPU cannot run high-level source code directly (1) 3. Any two differences, two marks each: a compiler translates the whole program before execution (1) whereas an interpreter translates and executes one statement at a time (1) / a compiler produces an executable/object code (1) whereas an interpreter produces no object code so it is needed each time (1) / a compiler reports all errors after translation (1) whereas an interpreter stops at the first error encountered (1) / compiled code runs faster (1) because an interpreter retranslates each statement every time it runs (1)

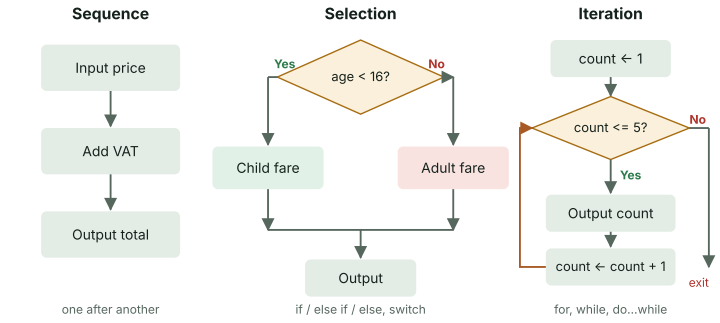
Program control: sequence, selection and iteration

AP20 FACT FILE

Specification: students should be able to

- Describe the fundamental programming concepts of sequence, selection and iteration
- Use if, if...else, else if, nested if and switch statements for selection, with conditions built from relational and Boolean operators
- Describe count-controlled iteration and use for loops, including loops that count up, down and in steps
- Describe condition-controlled iteration and use while (pre-condition) and do...while (post-condition) loops, including sentinel values
- Use nested loops and nested selection, and trace programs that combine all three constructs

Key words: sequence, selection, iteration, condition, if...else, else if, nested selection, switch, count-controlled, condition-controlled, for loop, loop counter, while loop, do...while loop, sentinel value, nested loop, infinite loop, trace table



The three control structures as flowcharts: sequence, selection with two branches, and iteration with a loop back

Overview: controlling the flow of a program

Every program, however large, is built from three control structures. Sequence means carrying out instructions one after another in the order they are written, so the position of each statement matters: a total must be calculated after the values it depends on have been input and before it is output. Selection lets a program choose between different paths depending on a condition that evaluates to true or false, for example charging a different fare for a child or adding a fee only for an overweight bag. Iteration repeats a block of statements, called the body of the loop, so the same code does not have to be written out many times. Iteration comes in two forms. Count-controlled loops repeat a number of times known before the loop starts, while condition-controlled loops repeat while, or until, a condition is met, so the number of repetitions does not need to be known in advance. These structures can be nested inside one another: a loop may contain decisions, and one loop may contain another. Being able to identify each structure in a given algorithm, choose the right one for a task, write it in C# and trace it by hand are all core skills tested in AS 1.

Selection with if, else if and nested if

An if statement evaluates a condition and runs its block only if the result is true; if it is false the block is skipped and execution continues after it. Adding else gives two mutually exclusive paths: exactly one of the two blocks will run, never both. When there are several possibilities, an else if chain tests conditions in order and runs only the first block whose condition is true, so the order of the tests matters. Arranging bands in order, lowest first or highest first, means each later condition needs only one bound, because earlier branches have already removed the other values. The final else catches anything not matched, which is useful for out-of-range or unexpected data. Nested selection puts one if statement inside another, so the inner decision is only made when the outer condition has a particular result; for example, a hospital fridge monitor might first ask whether a reading is within the safe range and only then decide whether an unsafe reading is too cold or too warm. Conditions are built using relational operators and combined with && and ||, as covered in AP19. Clear indentation shows which statements belong to which branch, and deep nesting should be avoided where a chain or combined condition reads more clearly.

The switch statement

A switch statement compares the value of one variable or expression with a list of constant case labels and jumps to the matching case. In C#, each case section must end with break (or another jump such as return) so that control does not fall through into the next case; several labels can be stacked above one section so that they share the same code, for example treating 6 and 7 as weekend days. The default section runs when no case matches, acting like the final else in a chain. A switch is clearer and less repetitive than a long else if chain when a single value is being matched against fixed options, such as menu choices, day numbers or category codes, and it makes the set of options easy to see at a glance. Its limitation is that the classic form tests only for equality with exact values, so it cannot directly express ranges such as age below 16; an if... else if chain is needed for those. Newer versions of C# do allow relational patterns in a switch, but the AS course expects the traditional form. In pseudocode, the equivalent is usually written as CASE ... OF with a list of options and an OTHERWISE clause.

Count-controlled loops

A count-controlled loop is used when the number of repetitions is known before the loop begins, either because it is fixed, such as five working days, or because it has already been entered, such as the number of people in a group. In C# this is a for loop, whose header has three parts: initialisation of an integer loop counter, a condition checked before each pass, and an update applied after each pass, usually i++ to add one. If the condition is true, the body runs again; once it is false, the loop ends and execution continues after it. The counter can be used inside the body, for example to number prompts, and it can count down (i--) or in steps (i += 3). Off-by-one errors are common, so check the boundaries carefully: for (int i = 1; i <= 5; i++) runs five times, as does for (int i = 0; i < 5; i++), but i < 5 starting from 1 runs only four times. In pseudocode, a count-controlled loop is written as FOR i ← 1 TO 5 ... NEXT i, or in plain English as REPEAT 5 TIMES. The foreach loop, which visits every element of an array or list, is also count-controlled and is covered with data structures in AP22.

Condition-controlled loops and sentinels

A condition-controlled loop repeats while, or until, a condition is met, so it is used when the number of repetitions is not known in advance. A while loop is a pre-condition loop: the condition is tested before each pass, so if it is false at the start the body never runs at all. This suits processing a list of values that ends with a sentinel, a dummy value such as -1 that marks the end of the data and is not itself processed. The standard pattern reads the first value before the loop and reads the next value as the last statement in the body. If the sentinel is entered first, the loop correctly processes nothing, but any later calculation such as an average must then guard against dividing by zero. A do...while loop is a post-condition loop: the body runs first and the condition is tested at the end, so it always runs at least once, which suits menus and input validation where the user must be asked at least once. The equivalent pseudocode REPEAT ... UNTIL stops when its condition becomes true, which is the opposite of do...while. If nothing inside the body can change the condition, the loop never ends: an infinite loop, which is a logic error.

Nested structures and tracing

Structures can be placed inside one another. With nested loops, the inner loop runs completely for every single pass of the outer loop, so the total number of times the inner body runs is the product of the two loop counts: an outer loop of 3 and an inner loop of 4 gives 12. Nested loops naturally process two-dimensional data such as seating plans, timetables, multiplication grids and the 2D arrays met in AP22. Real programs mix all three constructs, for example a validation loop to read a group size, then a for loop to process each member, with else if selection inside to place each person in a price band. To check such code, use a trace table: a column for each variable and condition, and a new row each time a value changes. Work through the code one statement at a time exactly as the computer would, rather than guessing the pattern. Trace tables reveal logic errors such as off-by-one boundaries, a sentinel being added to the total, or a counter that is never updated. Meaningful names (row, col, count) and consistent indentation make nested code far easier to read, trace and debug.

T Key terms from the mark schemes

Sequence
Executing instructions one after another in the order in which they are written.

Selection
A construct in which a condition determines which of two or more paths through the program is followed.
Credited: decision / branching

Iteration
Repeating a set of instructions, either a set number of times or while/until a condition is met.
`Credited: repetition / looping`

Condition-controlled loop
A loop that repeats while or until a condition is met, so the number of repetitions need not be known in advance.
`Credited: indefinite iteration`

Nested selection
An if statement placed inside another if or else block, so the inner decision is only made when the outer condition has a particular result.
`Credited: nested if`

Pre-condition loop
A loop that tests its condition before each pass, so the body may run zero times (while).

Infinite loop
A loop whose terminating condition is never met, usually because the control variable is not changed in the body.

Count-controlled loop
A loop that repeats a fixed number of times known before it starts, e.g. a for loop.
`Credited: definite iteration`

Nested loop
A loop placed inside the body of another loop; the inner loop completes fully for each pass of the outer loop.

Sentinel value
A dummy value entered to mark the end of the data, which is not processed as part of the data.
`Credited: rogue value / terminator`

Post-condition loop
A loop that tests its condition after each pass, so the body always runs at least once (do...while / REPEAT... UNTIL).

C# example

`Sequence: order matters`

```
Console.Write("Adult tickets: ");
int adults = int.Parse(Console.ReadLine());
Console.Write("Child tickets: ");
int children = int.Parse(Console.ReadLine());
double total = adults * 9.50 + children * 6.00;
total = total * 0.9;                        // 10% online discount
Console.WriteLine($"Total: £{total:F2}");
// Input 2 and 3  ->  Total: £33.30
```

Exam focus

Questions often give an algorithm and ask you to explain sequence, selection and iteration with reference to it; one mark is typically for the definition and one for identifying where it appears, so quote the relevant line, for example 'the IF total >= 500 statement only applies the discount when the condition is true'. For the difference between count-controlled and condition-controlled loops, state that a count-controlled loop repeats a number of times known before it starts, while a condition-controlled loop repeats until or while a condition is met, and give an example of each. For while versus do...while, state where the condition is tested and the consequence: zero runs are possible with while; at least one with do...while. When asked to justify a loop choice, link it to whether the number of repetitions is known. Code-writing questions credit correct logic, sensible variable names and correct boundaries, so check < versus <= carefully; mark schemes often accept equivalent code in any sensible form. In trace questions, write out every value in order rather than jumping to the answer. Common mistakes include processing the sentinel value, confusing UNTIL with WHILE conditions and forgetting that only the first true branch of an else if chain runs.

? Check yourself

1. State the three basic programming constructs. [3]
2. A Ballymena sports shop's loyalty app uses this algorithm: points ← 0; FOR purchase ← 1 TO 5: INPUT spend; points ← points + spend DIV 10; NEXT purchase; IF points >= 50 THEN OUTPUT "Voucher earned" ENDIF; OUTPUT points. By referring to the algorithm, explain the terms (a) sequence (b) selection (c) iteration. [6]
3. Describe the difference between a while loop and a do...while loop. [2]

Answers: 1. Sequence (1) / selection (1) / iteration or repetition (1) 2. (a) Sequence: instructions are executed one after another in the order written (1) e.g. points is set to 0 before the loop, and the voucher check and final OUTPUT points happen only after all five spends have been processed (1) (b) Selection: a condition decides which path is followed / whether a block runs (1) e.g. "Voucher earned" is output only IF points >= 50 (1) (c) Iteration: a set of instructions is repeated (1) e.g. INPUT spend and the points update are repeated once for each of the 5 purchases by the FOR loop (1) 3. A while loop tests the condition at the start so it may run zero times (1) / a do...while loop tests the condition at the end so it always runs at least once (1)

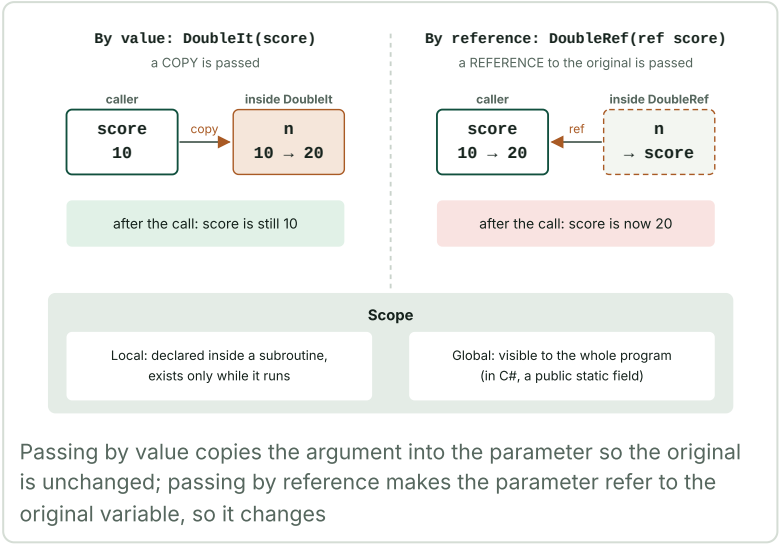
Subroutines: procedures, functions, parameters and scope

AP21 FACT FILE

Specification: students should be able to

- Explain what is meant by a subroutine and describe the difference between a procedure and a function
- Write and call procedures and functions in a high-level language, using parameters, arguments and return values
- Explain passing parameters by value and by reference, and predict the effect of each
- Explain the scope of local and global variables and the advantages of using local variables
- Explain the benefits of modular programming and decomposing a problem into subroutines

Key words: subroutine, procedure, function, method, call, parameter, argument, return value, by value, by reference, scope, local variable, global variable, side effect, modular programming, decomposition, library routine



Passing by value copies the argument into the parameter so the original is unchanged; passing by reference makes the parameter refer to the original variable, so it changes

Overview: building programs from subroutines

As programs grow, writing every instruction in one long block becomes unmanageable. A subroutine is a named, self-contained block of code that performs a specific task and can be called whenever that task is needed. When a subroutine is called, control passes to it; when it finishes, control returns to the statement after the call. There are two kinds. A procedure carries out a task, such as printing a receipt, without returning a value, while a function carries out a task and returns a single value, such as a calculated price, to the point where it was called. Data is passed into a subroutine through parameters, so the same code can work on different values each time. Parameters can be passed by value, where the subroutine receives a copy, or by reference, where it can change the caller's original variable. Each variable also has a scope, the part of the program where it can be used: local variables exist only inside their subroutine, whereas global variables can be accessed anywhere. Splitting a solution into well-designed subroutines is called modular programming, and it makes programs easier to write, test, maintain and share across a team. These ideas build directly on the top-down decomposition used when designing algorithms.

Procedures, functions and methods

In C#, every subroutine is written as a method with a header and a body. The header gives the return type, the name and the parameter list, for example static double AddVat(double price, double rate). A procedure has the return type void, meaning it returns nothing; it is called as a statement on its own, such as ShowMenu();. A function has a return type such as int, double, bool or string and must use a return statement on every path to send back a value of that type. Because it produces a value, a function call is used inside an expression, for example assigned to a variable, printed or tested in a condition: if (IsValidPin(pin)). The return statement also ends the function immediately, so any code after it on that path is not executed. Programs also make constant use of library routines, prewritten and tested subroutines supplied with the language, such as Console.WriteLine, Math.Round and int.Parse. In object-oriented languages, subroutines belonging to a class are called methods, which is why C# uses that term throughout. Choosing between a procedure and a function depends on whether the caller needs a result back: a calculation should normally be a function, while displaying output is usually a procedure.

Parameters, arguments and return values

A parameter is a variable listed in a subroutine's header, together with its data type, that receives a value when the subroutine is called. An argument is the actual value, variable or expression supplied in the call. Arguments are matched to parameters by position, so they must agree in number, order and data type; calling AddVat(40) when two parameters are expected, or passing text where a number is expected, is a compile-time error. Parameters make subroutines general: a single MonthlyCost function can work out the price for any member because the age and the off-peak flag are passed in, rather than being typed into the code. Inside the subroutine, parameters behave like local variables, and each call gets fresh copies of them. A function sends its result back with return, and the value of the whole call expression becomes that result. Designing the interface of a subroutine, meaning its name, parameters and return type, is an important step: if the interface is clear and documented, other programmers can use the subroutine correctly without needing to read the code inside it. Passing data through parameters and return values, rather than through global variables, keeps each subroutine independent and easy to test.

Passing by value and by reference

When an argument is passed by value, the subroutine receives a copy of the value in a new memory location. Any change made to the parameter inside the subroutine affects only the copy, which is discarded when the subroutine ends, so the caller's original variable is protected. This is the default in C# and in most languages, and it is the safer choice whenever the subroutine only needs to read the data. When an argument is passed by reference, the subroutine receives a reference to the original variable, effectively its memory address, so any change to the parameter changes the caller's variable directly. In C# this is requested with the ref keyword in both the header and the call, which makes the intention visible. Passing by reference is useful when a subroutine must update the caller's data, such as swapping two values in a sort, or must give back more than one result; it also avoids copying large amounts of data. The out keyword is a variant in which the subroutine must assign the value, as in int.TryParse. The disadvantage of passing by reference is the risk of side effects: a call can alter variables unexpectedly, making errors harder to find.

Scope: local and global variables

The scope of a variable is the part of a program in which it can be accessed. A local variable is declared inside a subroutine or a block, such as the body of a loop, and can only be used there. It is created when the subroutine is called and destroyed when it ends, so its memory is released and its value does not carry over between calls. Parameters are also local to their subroutine. Because local variables are invisible outside their subroutine, different subroutines can safely reuse common names like count or total without interfering with each other, and each subroutine can be understood, tested and reused on its own. A global variable is declared outside all subroutines and is accessible from anywhere in the program for the whole of its run. C# does not have true global variables, but a public static field of a class behaves in the same way. Globals are convenient for data that genuinely belongs to the whole program, but because any part of the code can change them, an unexpected value can be very hard to trace, modules become dependent on each other and are harder to reuse, and memory is held for the entire run. Good practice is to use local variables and parameters wherever possible.

Modular programming

Modular programming means dividing a program into separate, self-contained modules, such as subroutines, each responsible for one clearly defined task. It follows naturally from top-down design: the main program becomes a short sequence of calls that reads like an outline of the solution, and each call is developed separately. The benefits are significant. Each module can be tested on its own before being combined with others, so errors are easier to locate and fix. A module written once can be called many times and reused in other programs, reducing duplication and development time, and tested modules can be collected into libraries. Maintenance is simpler because a change, such as a new VAT rule, is made in one place. Large projects can be shared across a team, with programmers developing different modules at the same time, provided the interfaces are agreed. There are some costs: planning the modules and their interfaces takes time, and modules must be integrated and tested together as well as separately, because each can work correctly alone yet fail when combined. Overall, though, modular design produces code that is more reliable, readable and maintainable, which is why it is standard practice in industry.

Key terms from the mark schemes

Subroutine
A named, self-contained block of code that performs a specific task and can be called from elsewhere in a program.
Credited: subprogram / method / module

Procedure
A subroutine that performs a task but does not return a value (void in C#).

Function
A subroutine that performs a task and returns a single value to the point where it was called.

Parameter
A variable in a subroutine's header that receives a value (argument) passed in when the subroutine is called.

Argument
The actual value or variable supplied to a subroutine when it is called.
Credited: actual parameter

Passing by value
Passing a copy of an argument to a subroutine so that the original variable cannot be changed by it.
Credited: by val

Passing by reference
Passing the address of (a reference to) a variable so that changes made in the subroutine affect the original variable.
Credited: by ref

Scope
The region of a program in which a variable can be accessed.

Local variable
A variable declared inside a subroutine or block that can only be accessed there and exists only while it runs.

Global variable
A variable that can be accessed from any part of the program and exists for the whole run.

Modular programming
Dividing a program into separate, self-contained modules (subroutines), each performing one clearly defined task.

Library routine
A prewritten, tested subroutine supplied with a language or obtained from elsewhere that can be called in a program.

C C# example

Parameters and arguments

```
static void ShowLine(char symbol, int length)     // parameters
{
    for (int i = 0; i < length; i++) Console.Write(symbol);
    Console.WriteLine();
}

static double AddVat(double price, double rate)
{
    return price + price * rate;
}

ShowLine('=', 20);                                 // arguments
Console.WriteLine("Total: " + AddVat(40, 0.2));    // Total: 48
ShowLine('=', 20);
```

Exam focus

Definition questions reward the key distinguishing point: a function returns a value to the calling code, whereas a procedure does not; a local variable can only be accessed within the subroutine or block in which it is declared. When explaining by value versus by reference, say what is passed (a copy versus a reference or address of the original) and the consequence for the original variable (protected versus changed); one mark is often given for each half. Questions asking for advantages of local variables or disadvantages of globals expect specific points such as independent testing, reuse of identifiers and hard-to-trace changes. Benefits of modularity are best written as two-part points, giving the feature and its consequence, for example 'each module can be tested independently, so errors are easier to locate'. Avoid vague answers such as 'it is easier' or 'it is quicker' without saying why. In code questions, check that the header has the correct return type, parameter types and order, and that a function returns a value on every path. Common errors include confusing parameters with arguments, forgetting that a function call must be used in an expression, and claiming that subroutines make programs run faster.

? Check yourself

1. State the difference between a procedure and a function. [1]
2. Explain the difference between a parameter and an argument, using an example. [3]
3. Write the header of a C# function called CalculateFare that takes the number of zones (an integer) and whether the passenger is a student (Boolean) and returns the fare as a real number. [3]

Answers: 1. A function returns a value (to the calling code) whereas a procedure does not (1) 2. A parameter is a variable in the subroutine header that receives a value (1) / an argument is the actual value/variable passed in the call (1) / valid example identifying both, e.g. in AddVat(double price) price is the parameter and in AddVat(40) 40 is the argument (1) 3. static double CalculateFare(int zones, bool isStudent) – return type double (1) / name and int parameter (1) / bool parameter (1). Accept decimal/float for double; any sensible parameter names.

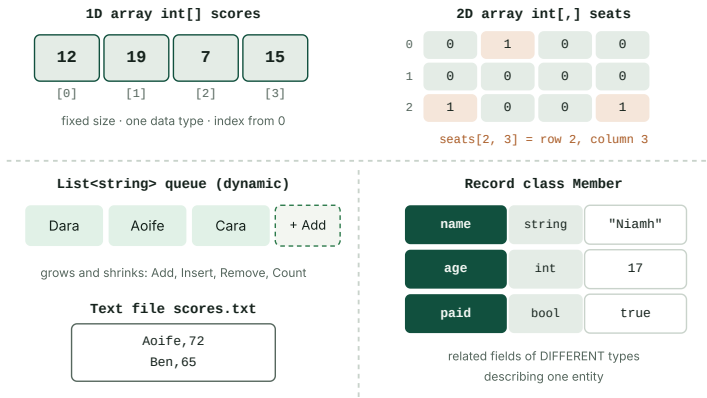
Data structures: arrays, lists, records and files

AP22 FACT FILE

Specification: students should be able to

- Describe the purpose of data structures and distinguish between static and dynamic structures
- Use one-dimensional and two-dimensional arrays, including traversing them with loops
- Use lists as dynamic data structures and justify the choice of a list or an array
- Describe and use records to group related fields of different data types
- Write to, append to and read from text files, splitting delimited lines into fields
- Explain why files are needed for persistent storage and how file errors can be avoided

Key words: data structure, array, index, element, two-dimensional array, static, dynamic, list, record, field, struct, traverse, text file, delimiter, CSV, persistent storage, append



Four data structures: a one-dimensional array with indexes, a two-dimensional array addressed by row and column, a growing list, and a record with fields of different types

Overview: storing collections of data

Most real programs handle collections of data rather than single values: the marks of a class, the seats in a theatre, the members of a club. A data structure groups these items under one identifier so that a loop can process them all with a few lines of code. This lesson covers four structures. An array stores a fixed number of elements of one data type, accessed by an index that starts at 0; a two-dimensional array arranges elements in rows and columns. A list is also a collection of items of one type, but it is dynamic, so items can be added and removed while the program runs. A record groups related fields of different types, such as a member's name, age and whether fees are paid, so that one item can be described completely. Finally, a text file stores data on secondary storage so that it survives after the program closes. Choosing well matters: the right structure makes code shorter, easier to read and less likely to contain errors, and exam questions often ask you to justify a choice for a given scenario.

One-dimensional arrays

An array is a static data structure: its size is fixed when it is created and every element has the same data type. In C# the declaration int[] goals = new int[5]; reserves space for five integers, each set to the default value 0, while an initialiser list such as { 4, 0, 12 } sets both the size and the values. Each element is reached directly through its index, so goals[0] is the first element and goals[goals.Length - 1] is the last. Direct access by index makes arrays fast, because the program does not need to look through earlier elements to reach a later one. A for loop with the counter running from 0 while it is less than Length is the standard way to traverse an array, for example to total the values, find the largest, count items that meet a condition or carry out a linear search. Trying to use an index outside 0 to Length - 1 causes an IndexOutOfRangeException at run time, which is one of the most common errors when working with arrays.

Two-dimensional arrays

A two-dimensional array stores data that naturally forms a grid or table, such as tickets sold for several shows over several nights, a seating plan, or the scores of each pupil in each test. Every element still has the same data type, and it is identified by two indexes, conventionally written [row, column]. In C# the declaration int[,] sales = new int[3, 4]; creates three rows of four columns, twelve elements in all. GetLength(0) returns the number of rows and GetLength(1) the number of columns, while Length returns the total number of elements. Processing a 2D array normally needs nested loops: the outer loop selects a row and the inner loop moves across each column of that row. To total each row, a running total is set to zero at the start of each pass of the outer loop and output or stored when the inner loop finishes; to total each column the loops are simply swapped. Keeping the order of the indexes consistent throughout a program is essential, because swapping them gives wrong results or an index out of range error.

Lists and choosing between a list and an array

A list is a dynamic data structure: it starts empty, or with some items, and grows or shrinks as the program runs. In C# a List<string> holds strings and a List<Member> holds Member records. Built-in methods make common tasks simple: Add puts an item at the end, Insert places it at a given index, Remove deletes the first matching item, RemoveAt deletes by position, Contains and IndexOf search, and Sort puts items in order. Count gives the current number of items. An array is the better choice when the number of items is known in advance and will not change, such as twelve monthly totals or a fixed board for a game, because it is simple and its memory is reserved once. A list is the better choice when the amount of data is unknown or varies, such as bookings taken during the day, because the program does not have to guess a maximum size, waste memory on unused elements or copy data into a bigger array. When justifying a choice, always link the property of the structure to the scenario.

Records

A record is a data structure made up of a fixed number of named fields that together describe one entity, such as a player, a pupil or a product. Unlike an array, the fields of a record may be of different data types: a player record might contain a string name, an integer squad number, a real height and a Boolean injured flag. In C# a record type can be defined with a struct or with a class that has public fields, and each field is accessed with dot notation, as in p.squadNumber. Grouping the fields keeps all the data about one item together, so it can be passed to a subroutine, stored or sorted as a single unit. The alternative, several parallel arrays such as names[] and ages[] that share an index, works but is error-prone because the arrays can easily fall out of step. An array or list of records behaves like a table in a database, with each record forming a row and each field a column, and it maps directly onto a line of a CSV file, which makes saving and loading data straightforward.

Text files and file handling

Data stored in variables, arrays and lists is held in RAM and is lost when the program ends or the computer is switched off. Text files on secondary storage provide persistent storage so that data can be reloaded the next time the program runs. A program opens a file in a particular mode. Opening for writing creates the file, or overwrites it completely if it already exists; opening for appending adds new lines to the end without disturbing existing data; opening for reading lets the program process the existing lines. A common layout is one record per line, with fields separated by a delimiter such as a comma, which is known as a CSV file. When a line is read, Split separates it into an array of strings, and numeric or Boolean fields must then be converted with methods such as int.Parse before they are used in calculations or conditions. Files must be closed after use so that buffered data is actually written to the disk and the file is released; a using block does this automatically. Robust programs also check that a file exists before reading it, or handle the exception that occurs if it does not.

Key terms from the mark schemes

Data structure
A way of organising and storing a collection of related data items under one identifier so that they can be processed efficiently.

Array
A static data structure that holds a fixed number of elements of the same data type under one identifier, each accessed by an index.
`Credited: a set of data items of the same type grouped under one name`

Index
The position number used to access an element of an array or list; in C# the first index is 0.
Credited: subscript

Two-dimensional array
An array whose elements are arranged in rows and columns and are accessed using two indexes, [row, column].
Credited: 2D array; table/matrix

Static data structure
A data structure whose size is fixed when it is created and cannot change while the program runs, such as an array.

Dynamic data structure
A data structure whose size can grow or shrink while the program is running, such as a list.

List
A dynamic data structure whose size can change at run time as items are added or removed, with methods such as Add and Remove.

Record
A data structure that groups a fixed set of related fields, which may be of different data types, to describe one item.

Field
One named item of data within a record, such as a member's name or age.
Credited: attribute

Text file
A file of characters, usually organised in lines, stored on secondary storage so that data persists between program runs.

Delimiter
A character, such as a comma, used to separate the fields of a record stored on one line of a text file.
Credited: separator

Persistent storage
Storage that keeps data after the program ends or power is removed, such as a file on secondary storage.
Credited: non-volatile storage

C# example

Declaring a one-dimensional array

```
int[] goals = new int[5];                 // five ints, all 0
string[] counties = { "Antrim", "Armagh", "Down",
                      "Fermanagh", "Londonderry", "Tyrone" };
double[] prices = new double[] { 2.50, 3.75, 1.20 };

goals[0] = 3;
goals[4] = goals[0] + 1;
Console.WriteLine(counties[3]);               // Fermanagh
Console.WriteLine(counties.Length);           // 6
Console.WriteLine(goals[4]);                  // 4
Console.WriteLine(prices[prices.Length - 1]); // 1.2
```

Exam focus

Definitions are usually worth one or two marks: for an array, mention a fixed number of elements, one data type, one identifier and access by index. For a record, stress related fields of different data types. Describe and explain questions about choosing a structure reward a property linked to the scenario, for example 'a list, because the number of bookings changes during the day' or 'a 2D array, because the data forms a grid of rows and seats'. Static versus dynamic questions need both sides: size fixed when created versus size can change while the program runs, ideally with an example of each. Code questions earn marks for a correct declaration, a loop with correct bounds, correct indexing and correct output, so write i < Length and start from 0. In file questions, credit is given for opening the file in the right mode, reading or writing a line at a time inside a loop, splitting on the delimiter, converting data types and closing the file. Common mistakes include starting indexes at 1, using <= Length, swapping row and column indexes, and forgetting that data read from a file is text.

? Check yourself

1. Define the term array. [2]
2. Explain the difference between a static and a dynamic data structure, giving a C# example of each. [4]
3. A youth theatre in Derry~Londonderry has 8 rows of 12 seats. Write a C# declaration of a suitable data structure to record whether each seat is booked, and a statement that marks the third seat in the first row as booked. [3]

Answers: 1. A data structure that stores a fixed number of elements/values (1) / of the same data type under one identifier, each accessed by an index (1). 2. Static: size is fixed when the structure is created and cannot change while the program runs (1) / e.g. int[] marks = new int[30]; (1) / Dynamic: size can grow or shrink at run time as items are added or removed (1) / e.g. List<string> names = new List<string>(); (1). 3. bool[,] seats = new bool[8, 12]; (accept int[,] or char[,] with explanation) - correct type and 2D syntax (1) / correct dimensions 8 and 12 (1) / seats[0, 2] = true; (1).

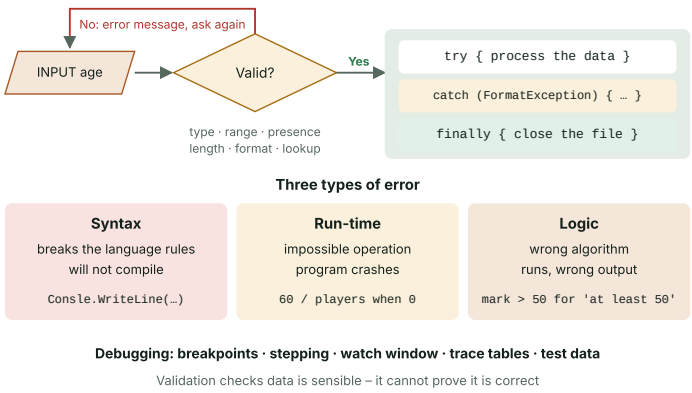
Robust programs: validation, exceptions, errors, testing and debugging

AP23 FACT FILE

Specification: students should be able to

- Describe and use validation checks, and distinguish validation from verification
- Use exception handling to deal with run-time errors
- Explain the differences between syntax, semantic, run-time and logic errors
- Select valid, extreme, invalid and null test data and use a test plan to find errors
- Use the debugging tools of an integrated development environment (IDE), such as breakpoints, stepping and watches, to locate errors (IDE features are introduced in AP19)
- Explain how the choice of translator (compiler or interpreter, covered in AP19) affects when syntax, semantic and run-time errors are detected

Key words: robust, validation, verification, exception, try...catch, syntax error, semantic error, run-time error, logic error, test plan, boundary data, IDE, breakpoint, watch, compiler, interpreter, stepping, call stack



A robust input process: validation loop rejects bad data, try and catch handle run-time errors, with the three error types summarised

Overview: building robust programs

A robust program keeps working sensibly when users make mistakes or when something unexpected happens, such as a missing file or an impossible calculation. Programmers achieve this in several ways. Validation checks that input data is reasonable before it is used, and verification checks that it was entered accurately. Exception handling catches run-time errors so that the program can display a helpful message and carry on instead of crashing. Errors still creep in while code is written, so it is important to recognise the four kinds of error: syntax, semantic, run-time and logic. Systematic testing with a test plan of valid, extreme, invalid and null data reveals many of these errors, and debugging tools in an integrated development environment (IDE), such as breakpoints, stepping and watch windows, help locate their causes. Finally, the lesson recaps the translators met in AP19, because whether a program is compiled or interpreted affects when each kind of error comes to light.

Validation and verification

Validation is an automatic check, carried out by the program, that input data is sensible and obeys set rules before it is processed or stored. A presence check makes sure something has been entered; a length check limits the number of characters; a range check ensures a value lies between limits; a type check confirms the data is of the expected type; a format check compares the data with a pattern, such as a postcode beginning with BT; and a lookup check compares it with a list of allowed values. In C#, int.TryParse performs a type check without crashing, and a do...while or while loop repeats the prompt until the data passes every check, giving a clear message each time. Validation improves data integrity, but it cannot guarantee that data is correct: a date of birth can be valid yet still be the wrong date. Verification addresses accuracy instead, by checking that the data matches what was intended. Double entry asks the user to type data such as a password twice and compares the two copies, while a visual check asks the user to read the data back and confirm it before it is saved.

Exception handling

When an operation fails while a program is running, for example dividing an integer by zero, converting the text 'ten' with int.Parse, reading past the end of an array or opening a file that does not exist, C# creates an exception object. If nothing handles it, the program stops with an error message, which may lose the user's data. Exception handling lets the programmer plan for these situations. Code that might fail is placed in a try block. One or more catch blocks follow, each handling a specific type of exception, such as FormatException, DivideByZeroException, IndexOutOfRangeException or FileNotFoundException, so that each problem can be given a suitable response. Specific catches must come before the general Exception type, which catches anything. A finally block contains code that runs whether or not an exception occurred, which makes it ideal for tidying up, such as closing a file. Exception handling and validation work together: validation prevents most bad input from being processed, while exception handling deals with problems that cannot easily be checked in advance.

Types of programming error

Syntax errors break the grammatical rules of the programming language, for example a missing semicolon, an unmatched bracket or a misspelt keyword. The compiler reports them, usually with a line number, and the program cannot run until they are corrected. Semantic errors occur when a statement is grammatically correct but has no valid meaning, much like the sentence 'the square circle sang loudly', which is grammatical but nonsense. Assigning text to an integer variable or using a variable before it has been given a value are examples; in C# the compiler catches many of these, although some only appear when the program runs. Run-time errors, also called execution errors, happen while the program is running when it attempts something impossible, such as dividing by zero, reading beyond the end of an array or converting non-numeric text to a number; without exception handling the program crashes. Logic errors are flaws in the algorithm itself: the program compiles and runs but produces the wrong result, for example using + instead of * or > instead of >=. They are the hardest to find because no error message is produced, so they are revealed only by testing.

Testing and test plans

Testing aims to show that a program does what it should and to uncover errors before the program is used for real. A test plan is prepared in advance and lists, for each test, what is being tested, the test data, the type of data and the expected outcome; when the test is run, the actual outcome is recorded and the test is marked pass or fail. Good test data covers four categories. Valid or normal data is typical input that should be accepted. Extreme or boundary data lies exactly at the edges of what is allowed, such as 0 and 120 for an age, and at any point where the processing changes, such as the age at which a concession price begins. Invalid or erroneous data should be rejected, for example values out of range or of the wrong type. Null data means nothing is entered at all. Boundary tests are especially valuable because logic errors often involve using the wrong relational operator. When a test fails, the programmer debugs and corrects the code, and then repeats all the tests to make sure the fix has not introduced a new error.

The IDE and debugging

The integrated development environment was introduced in AP19: its editor, syntax highlighting, syntax checking, code completion, preferences and compile-and-run facility all help programmers avoid syntax errors in the first place. This lesson concentrates on the debugging tools, which help locate run-time and logic errors once the program runs. A breakpoint pauses execution at a chosen line so that the program's state can be examined. Stepping then executes one line at a time: step into enters a called subroutine, while step over runs the whole call as a single step, so the programmer can follow exactly which path is taken through selections and loops. Watch and locals windows display the current values of chosen variables, and the call stack shows the chain of subroutine calls that led to the current line, which is invaluable when an exception is thrown inside a subroutine called from several places. Used together, these tools let the programmer find the first point at which the actual behaviour differs from the expected behaviour, rather than guessing. For example, pausing on the price selection in the leisure centre program and stepping with an age of 65 shows immediately that the concession branch is skipped.

Translators and error detection

Translators were covered in AP19, so only a recap is needed here, focused on robustness. Source code has to be translated into machine code before it can run: an assembler does this for assembly language, one mnemonic to one instruction, while high-level languages use a compiler or an interpreter. A compiler translates the whole program before any of it runs, so every syntax error, and in a strongly typed language such as C# most semantic errors, appear together in an error list and no executable is produced until they are fixed. An interpreter translates and runs one statement at a time, so an error is only discovered when execution reaches the faulty line, possibly after earlier statements have already written to a file or updated a database. Neither kind of translator can detect a logic error, because the faulty code is perfectly legal; only a well-designed test plan reveals it. Run-time errors appear while the program executes whichever translator is used, which is why validation and exception handling are needed even in a program that compiles cleanly.

T Key terms from the mark schemes

Robust program
A program that copes with invalid input and unexpected events without crashing or producing incorrect results.

Validation
An automatic check that input data is reasonable and follows set rules before it is processed.
`Credited: checks data is sensible/allowable`

Verification
A check that data has been entered or transferred accurately, for example by double entry or a visual check.
`Credited: checks data matches the original/source`

Exception
An object raised when a run-time error occurs, which stops the program unless it is handled in a try...catch structure.

Syntax error
An error that breaks the grammatical rules of the programming language, so the program cannot be compiled or run.

Semantic error
A statement that follows the syntax rules but has no valid meaning, such as assigning text to an integer variable.

Run-time error
An error that occurs while the program is running when an impossible operation is attempted, such as division by zero, often causing a crash.
`Credited: execution error`

Logic error
An error in the algorithm that causes the program to run but produce incorrect results.
`Credited: logical error`

Extreme data
Test data at the boundaries of the valid range, such as the highest and lowest values allowed.
`Credited: boundary data`

Breakpoint
A marker set in the IDE that pauses execution at a chosen line so the program's state can be examined.

Stepping
Executing a program one line at a time in the IDE, either entering called subroutines (step into) or running them as one step (step over).

Call stack
A debugger display of the chain of subroutine calls that led to the current line of execution.

C C# example

`A validation loop`

```
int tickets;
bool valid;
do
```

```
{
    Console.Write("Tickets (1-6): ");
    string input = Console.ReadLine() ?? "";
    valid = int.TryParse(input, out tickets) && tickets >= 1 && tickets <= 6;
    if (!valid) Console.WriteLine("Please enter a whole number from 1 to 6.");
} while (!valid);
Console.WriteLine("Accepted: " + tickets);
```

Exam focus

When classifying an error, describe its effect: it will not compile (syntax), it compiles but is meaningless (semantic), it crashes while running (run-time) or it runs but gives the wrong output (logic), and give a specific example. For validation questions, name the check and link it to the field in the scenario, for example 'a range check that the number of nights is between 1 and 14'; one generic list of checks gains little credit. Remember that validation does not ensure data is correct, which is the classic one-mark distinction from verification. For test data, give actual values with their category and the expected outcome; boundary values should be the limits themselves. Describing a debugging tool needs both what it does and how it helps find the error. Compiler and interpreter comparisons earn marks for matched pairs of points, such as whole program versus line by line, or executable produced versus no object code, so write both sides of each difference. In code-writing questions, a correct loop condition and a helpful error message are commonly credited.

? Check yourself

1. Describe the difference between validation and verification. [2]
2. A GP surgery in Ballymena is building an online booking form. State a suitable validation check for each of: (a) the patient's age (b) a 10-digit Health and Care Number (c) the appointment type, which must be 'Doctor', 'Nurse' or 'Phone'. [3]
3. Identify the type of error in each case: (a) int x = 10 / count; when count is 0 at run time (b) Console.WriteLine("Hi") with no semicolon (c) average = (a + b) * 2; when the average of a and b was required. [3]

Answers: 1. Validation checks that data is reasonable / sensible / follows rules (e.g. within a range) (1) / verification checks that data has been entered or copied accurately, matching the source (e.g. double entry) (1). 2. (a) Range check, e.g. 0 to 120 (accept type check) (1) / (b) Length check of exactly 10 characters (accept type check for digits only / format check) (1) / (c) Lookup check against the list of allowed values (1). 3. (a) Run-time (execution) error (1) / (b) Syntax error (1) / (c) Logic error (1).

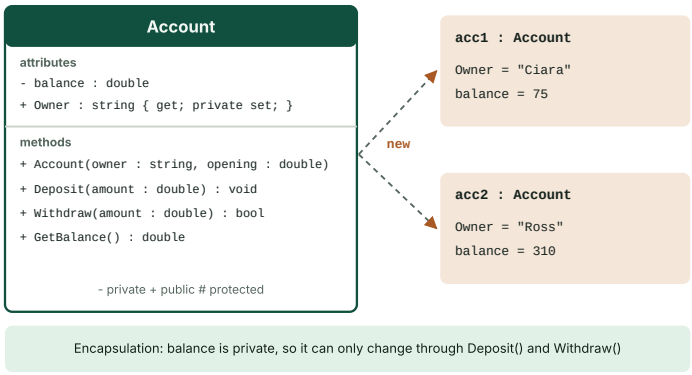
Classes, objects and encapsulation

AP24 FACT FILE

Specification: students should be able to

- Explain object-oriented programming terminology: class, object, attribute (property), method (behaviour), constructor and instantiation
- Explain the purpose of a method's parameter list and return type
- Explain encapsulation and its benefits, including private attributes accessed through get and set methods
- Write classes in C#, create objects from them and predict the output of object-oriented code
- Represent a class using a class diagram, including visibility symbols

Key words: object-oriented programming, class, object, instance, instantiation, attribute, state, method, parameter list, return type, constructor, encapsulation, private, public, get method, set method, property, class diagram



Class diagram of an Account class with private attribute and public methods, and two objects instantiated from it with their own state

Object-oriented programming: the big picture

Object-oriented programming (OOP) is a programming paradigm in which a program is organised as a collection of objects that work together. Each object bundles the data that describes it with the subroutines that act on that data, so a hire bike in a city bike-share system knows its own identifier, dock and battery level and also knows how to record a ride. This contrasts with a procedural program, where data is usually held in separate variables or records and passed into free-standing procedures and functions. The central ideas are the class, which acts as the template, and the object, which is a specific thing built from that template. Classes are made robust by encapsulation: their data is kept private and can only be reached through the methods the class chooses to make public. OOP became popular because large systems are easier to build, test and maintain when they are divided into self-contained classes that mirror real-world entities such as customers, accounts, tickets or buses. This lesson covers classes, objects, attributes, methods, parameter lists, constructors, encapsulation with get and set methods and properties, and class diagrams, using C# throughout.

Classes, objects and instantiation

A class is a template, or blueprint, that defines what every object of a particular type will contain: its attributes (the data it stores) and its methods (the behaviours it can perform). By convention a class name begins with a capital letter, for example Bike, Book or Patient. An object is one specific instance of a class. Two Bike objects share exactly the same list of attributes and the same method code, but each holds its own values, so one may be docked at City Hall with 80% charge while another sits at Titanic Quarter with 35%. The set of values held in an object's attributes at any moment is called its state. Creating an object is called instantiation, and in C# it is done with the new keyword, as in Bike b1 = new Bike("BK101", "City Hall", 80);. The variable b1 holds a reference to the new object, and its methods are then called with dot notation, such as b1.Ride(12, "Queen's University"). One class can be used to create any number of objects, which is why a well-designed class saves so much repeated code: a school library needs one Book class, not thousands of separate sets of variables.

Attributes, methods and parameter lists

Attributes, which some books call properties or fields, are variables declared inside a class. Each needs a suitable data type: a bike's ID is a string, its battery percentage is an integer, a hire price is a real number and whether a bike is reserved is a Boolean. Methods are subroutines declared inside a class that describe what an object can do or what can be done to it; the textbook also calls them behaviours. A method header shows its access modifier, return type, name and parameter list. The parameter list states what extra information the method needs to do its job, giving a type and a name for each item, so SetDock(string newDock) must be given a string, such as "Botanic", every time it is called. A method that needs no extra information has an empty parameter list, written as a pair of empty brackets, as in GetBattery(). The return type is void if the method simply carries out an action, or a data type such as int or string if it sends a value back to the calling code. The values actually supplied in a call are called arguments, and they must match the parameters in number, order and type.

Constructors

A constructor is a special method that runs automatically when an object is instantiated, and its job is to give the new object a sensible starting state. In C# a constructor has exactly the same name as its class and no return type at all, not even void, which is how it can be recognised in code and in class diagrams. Its parameters usually supply initial values that are copied into the private attributes, for example public Bike(string id, string startDock, int charge). If a parameter has the same name as an attribute, the keyword this identifies the attribute, as in this.name = name;. If a programmer writes no constructor at all, C# supplies a default one with no parameters, which leaves attributes at default values such as 0, false or null. A class may have more than one constructor provided their parameter lists differ, letting objects be created with different amounts of starting information. Constructors are also the natural place to set attributes that must never change afterwards, such as a membership number: the value is assigned once when the object is built and, because no set method is provided, it stays fixed for the life of the object.

Encapsulation with get and set methods

Encapsulation means combining an object's data and the methods that operate on that data into a single unit, the class, and hiding the internal details so that the data can only be read or changed through methods the class provides. In C# the attributes are declared private, which stops code outside the class accessing them directly. Public get methods, or getters, return the current value of an attribute, and public set methods, or setters, change it. This gives the class control over every change. A setter can validate the new value, for instance rejecting a negative age, so an object can never be put into an invalid state. An attribute that should never change once it has been allocated, such as a National Insurance number or a hospital number, is given a getter but no setter, which makes it read-only. A setter can also carry out related tasks, such as recording a change of address in an audit log, so programmers who use the class do not have to remember those side effects every time. Because other code depends only on the public methods, the internal storage can later be redesigned without breaking the rest of the program. C# properties provide the same controlled access in a more compact form.

Class diagrams

A class diagram is a design notation, taken from the Unified Modelling Language (UML), that shows a class as a rectangle divided into three compartments. The top compartment holds the class name, the middle compartment lists the attributes with their data types, and the bottom compartment lists the methods with their parameters and return types, for example + Ride(minutes : int, endDock : string) : void. Visibility is shown by a symbol in front of each member: a minus sign for private, a plus sign for public and a hash for protected. In a well-encapsulated class most attributes carry a minus sign, while the methods that form the public interface carry a plus sign. The constructor is listed among the methods, with the class's name and no return type. Class diagrams are drawn during design, before coding, so that a team can agree what each class will store and do and how the classes relate to one another, and they then serve as documentation for anyone maintaining the program later. Books and papers vary slightly in style, for instance writing 'return int' instead of ': int', so what matters is that names, data types and visibility are clear and consistent.

T Key terms from the mark schemes

Class
A template or blueprint that defines the attributes and methods (behaviours) shared by all objects of one type.
`Credited: Basic building block of OOP specifying properties and behaviours`

Object
A specific instance of a class, with its own values for the class's attributes.
`Credited: A member of a class`

Instantiation
Creating a new object from a class, in C# using the new keyword, which calls the constructor.

Attribute
A data item (field) declared in a class whose values describe the state of each object.
`Credited: Property`

Method
A subroutine defined in a class that describes an action an object can perform or that can be performed on it.
`Credited: Behaviour`

Constructor
A special method with the same name as the class and no return type that runs when an object is created to initialise its attributes.

Encapsulation
Combining data and the methods that act on it in a class, and hiding the data (private) so it can only be accessed through the class's methods.
`Credited: Information hiding`

Get method
A public method that returns the current value of a private attribute.
`Credited: Getter / accessor`

Set method
A public method that takes a new value as a parameter and uses it to change a private attribute, often after validating it.
`Credited: Setter / mutator`

Parameter list
The part of a method header, in brackets, that states the data items (with types) the method must be given when it is called.

Property (C#)
A class member with get and/or set blocks that gives controlled access to a private field.

Class diagram
A UML diagram showing a class as a box with its name, attributes and methods, using +, - and # for public, private and protected.

C C# example

`Worked example: a bike-share Bike class`

```
Bike b1 = new Bike("BK101", "City Hall", 80);
Bike b2 = new Bike("BK102", "Titanic Quarter", 35);
b1.Ride(12, "Queen's University");
b2.Ride(40, "City Hall");             // refused: not enough charge
Console.WriteLine(b1.Report());       // BK101 at Queen's University, battery 68%
Console.WriteLine(b2.Report());       // BK102 at Titanic Quarter, battery 35%
class Bike
{
    private string bikeId;
    private string dock;
    private int battery;
    public Bike(string id, string startDock, int charge) { bikeId = id; dock = startDock; battery
= charge; }
    public string GetBikeId() { return bikeId; }     // getter only: ID never changes
    public void Ride(int minutes, string endDock)
    {
        if (minutes > battery) return;              // 1% charge per minute
        battery -= minutes; dock = endDock;
    }
    public string Report() { return bikeId + " at " + dock + ", battery " + battery + "%"; }
}
```

Exam focus

Definitions are a reliable source of marks when both halves are present. A class is a template or blueprint that defines the attributes and methods (behaviours) of a type of object; an object is a specific instance of a class with its own attribute values. Questions that say 'by referring to the code' expect you to name the actual class, objects, attributes and methods from the listing, so quote identifiers such as Bike, b1, battery or Ride rather than giving only a general definition. For encapsulation, a full answer says that data and methods are held together in the class, that the attributes are made private, and that they can only be accessed or changed through get and set methods. When you write a class, examiners look for private attributes with sensible data types, a constructor that assigns every parameter to an attribute, getters that return values, setters that take a parameter and return nothing, and a display method if one is asked for. Watch for attributes that must not change, such as a student or membership number, and give them no setter. In output questions, work through each statement in order and copy the spaces, punctuation and line breaks exactly. A common slip is giving a constructor a return type.

? Check yourself

1. State the two things that a class definition specifies for each of its objects. [2]
2. A school library in Dungannon is building a loans system in C#. Using examples from this system, explain the relationship between a class and an object. [3]
3. State the purpose of a constructor and describe two ways in which a constructor differs from other methods in C#. [3]

Answers: 1. The attributes / properties (data) of the objects (1) / the methods / behaviours that can be performed on the objects (1) 2. A class is a template / blueprint defining attributes and methods (1) / an object is a specific instance / member of the class with its own attribute values (1) / suitable example, e.g. class Book; objects are individual books such as one copy with accession number A0041 (1) 3. Purpose: to initialise a new object / set the starting values of its attributes when it is created (1) / Any two from: it has the same name as the class (1) / it has no return type, not even void (1) / it runs automatically when new is used (1)

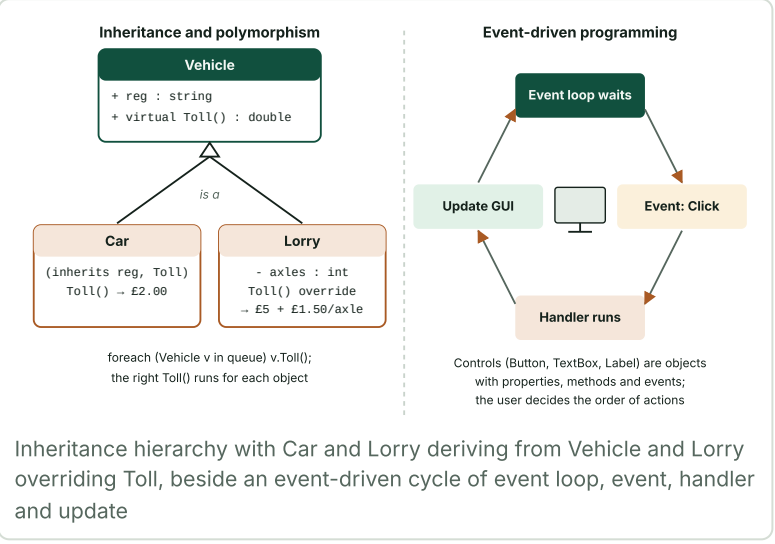
Inheritance, polymorphism, evaluating OOP and event-driven programming

AP25 FACT FILE

Specification: students should be able to

- Explain inheritance using the terms superclass (base class) and subclass (derived class), and represent a class hierarchy in a diagram
- Write C# subclasses that inherit from a base class, call the base constructor and add their own attributes and methods
- Explain polymorphism and implement it by overriding methods (virtual, abstract and override)
- Evaluate the object-oriented approach, discussing its advantages and disadvantages
- Describe event-driven programming and the use of events, event handlers and controls in a graphical user interface

Key words: inheritance, superclass, base class, subclass, derived class, class hierarchy, protected, base constructor, abstract class, polymorphism, virtual, override, overloading, code reuse, event-driven, event, event handler, control



Inheritance hierarchy with Car and Lorry deriving from Vehicle and Lorry overriding Toll, beside an event-driven cycle of event loop, event, handler and update

Building on classes: the big picture

Once a programmer can write classes and create objects, the next step is to make classes work together. Inheritance allows a new class to be built from an existing one, so a sports club system can define a general Member class once and then derive Junior and Adult classes that inherit everything a member has while adding their own details. Polymorphism builds on inheritance: the same method call, such as Fee(), can produce different results depending on which kind of object receives it, so a single loop can process a mixed list of members without checking each one's type. Together with encapsulation, these are the features that make object-oriented programming powerful for large systems. However, OOP is not automatically the best choice, so programmers must be able to evaluate it, weighing benefits such as realistic modelling, code reuse and reduced testing against extra complexity and a poor fit for some problems. Finally, this lesson looks at event-driven programming, in which the order of execution is decided by events such as button clicks. Graphical user interfaces are the most familiar example, and their buttons, text boxes and labels are themselves objects.

Inheritance

Inheritance is the mechanism by which a new, more specialised class takes on the attributes and methods of an existing, more general class. The general class is called the superclass, base class or parent class, and the specialised class is the subclass, derived class or child class. In C# the relationship is written with a colon, as in class Junior : Member. A subclass automatically has every attribute and method of its superclass and normally adds members of its own, so attributes accumulate as you move down a class hierarchy: if Member has two attributes and Junior adds one, a Junior object holds three. A subclass does not have to add anything, but usually does. Subclasses are independent of each other, so a method defined only in Junior cannot be used on an Adult object. Inheritance models an 'is a' relationship: a junior is a member, and a bus is a vehicle. If that sentence sounds wrong, inheritance is probably the wrong design. Hierarchies can have several levels, and a class hierarchy diagram shows the superclass at the top with lines and hollow arrowheads pointing up to it from each subclass.

Constructors, protected members and abstract classes

A subclass has its own constructor, but the superclass's attributes still need to be initialised. In C# the subclass constructor passes values up to the superclass constructor using : base(...), for example public Junior(int no, string n, string phone) : base(no, n), and then sets its own new attributes in its body. Private attributes of the superclass are inherited in the sense that every subclass object contains them, but code written in the subclass cannot refer to them directly; it must use the superclass's public get and set methods. Declaring a member protected, shown by # in a class diagram, makes it accessible inside the superclass and its subclasses while keeping it hidden from all other code. Sometimes a superclass describes an idea so general that no object should ever be created from it directly, such as Account in a credit union system where every account is in fact either a SavingsAccount or a LoanAccount. Declaring the class abstract prevents instantiation, so new Account(...) will not compile. An abstract class may contain abstract methods, which have a header but no body, and each non-abstract subclass is then forced to provide its own overriding version.

Polymorphism and overriding

Polymorphism, from the Greek for 'many forms', is the ability of objects of different classes to respond to the same method call in their own way. In C# it is achieved by overriding. The base class declares a method as virtual, meaning subclasses are allowed to replace it, or as abstract, meaning they must. A subclass then supplies a method with the same name, parameters and return type marked override. When a method is called through a reference of the base-class type, such as each item in a List<Vehicle>, the version that runs is chosen at run time from the actual class of the object, so a Car is charged the standard toll and a Lorry its axle-based toll. The calling code does not need an if statement to test each object's type, and a new subclass such as Coach can be added later without changing that loop. Overriding should not be confused with overloading, where several methods in the same class share a name but have different parameter lists. The compiler chooses the overloaded version from the arguments, at compile time, and no inheritance is involved.

Evaluating the object-oriented approach

OOP brings three big gains: realistic modelling, productivity and maintainability. Because classes correspond to real-world entities, the design is easier to discuss with users and maps naturally onto the problem. Inheritance and class libraries mean code is written once and reused, which improves productivity, and testing effort is reduced because a superclass that has already been tested does not need retesting; only the new or overridden methods in a subclass do. Encapsulation keeps each class's data safe from accidental change, and well-defined interfaces let a team develop and test classes separately and make maintenance easier. There are disadvantages. OOP suits some applications far better than others. It has been widely adopted for computer-aided design and manufacturing, engineering design, simulations, games and graphical interfaces, where objects are obvious, but programs that are very abstract or purely computational do not map neatly onto objects. For a small program or a straightforward data-processing job the extra design, code and memory overhead may bring little benefit. Deep hierarchies can become hard to understand, and a mistake in a base class spreads to every subclass. A good evaluation applies these points to the actual context and reaches a justified conclusion.

Event-driven programming and GUIs

In a sequential procedural program, the code decides the order in which steps happen. An event-driven program instead waits for events and responds to them, so the order of execution depends on what the user, the hardware or other software does. An event is an action or occurrence that the program can detect, such as a mouse click, a key press, a timer tick, a sensor reading or a message arriving. The program runs an event loop that listens for events and passes each one to the matching event handler, which is a subroutine attached to a particular control and event, such as btnCalculate_Click. Graphical user interfaces are the best-known example. Their controls, including buttons, text boxes, labels, check boxes, list boxes and timers, are objects with properties such as Text and Enabled, methods, and events they can raise, which shows how closely GUI programming and OOP are connected. The event-driven model suits interactive software because users choose what to do next, but programmers must make sure every handler works whatever order events occur in, for example by disabling a Save button until a file is open.

T Key terms from the mark schemes

Inheritance
The mechanism by which a new, more specialised class (subclass) takes on the attributes and methods of an existing, more general class (superclass) and can add or modify its own.

Derived class
A more specialised class that inherits from a base class.
Credited: Subclass / child class

Abstract class
A class that cannot be instantiated; objects must be created from its subclasses, which must implement any abstract methods it declares.

Overriding
Redefining an inherited method in a subclass with the same name and parameters, so the subclass's version runs for its objects.

Event-driven programming
A programming paradigm in which the flow of the program is determined by events such as user actions, sensor outputs or messages.

Control
An object on a graphical user interface, such as a button or text box, with properties, methods and events.
Credited: GUI component / widget

Base class
A general class whose attributes and methods are inherited by other classes.
Credited: Superclass / parent class

Protected
An access modifier that makes a member visible inside its own class and any subclasses but hidden from other code; shown as # in a class diagram.

Polymorphism
The ability of objects of different classes to respond to the same method call in different ways.
Credited: Many forms

Overloading
Defining several methods in the same class with the same name but different parameter lists.

Event handler
A subroutine that is run automatically when a particular event occurs on a control.

C C# example

Worked example: a GAA club membership system

```
Junior j = new Junior(101, "Orla", "07700 900123");
Console.WriteLine(j.GetName() + " " + j.GetMemberNo());   // Orla 101 (inherited)
Console.WriteLine(j.Fee());                                // 40 (overridden)
Console.WriteLine(j.Contact());                            // Orla, guardian 07700 900123
class Member
{
    private int memberNo;
    protected string name;
    public Member(int no, string n) { memberNo = no; name = n; }
    public int GetMemberNo() { return memberNo; }
    public string GetName() { return name; }
    public virtual double Fee() { return 120; }
}
class Junior : Member
{
    private string guardianPhone;
    public Junior(int no, string n, string phone) : base(no, n) { guardianPhone = phone; }
    public string Contact() { return name + ", guardian " + guardianPhone; }
    public override double Fee() { return 40; }
}
```

Exam focus

A complete definition of inheritance includes both ideas: a new, more specialised subclass takes on the attributes and methods (behaviours) of an existing superclass, and it can add its own or modify inherited ones. Use the terms superclass or base class and subclass or derived class correctly, and be ready to count how many attributes a subclass object has by adding inherited and new ones. For polymorphism, say that the same method call produces different behaviour for objects of different classes and support it with an example from the question, naming the overridden method. 'Evaluate' and 'discuss' questions on OOP are usually worth six marks or more and are marked in levels, so give developed points on both sides, such as more accurate modelling, reuse through inheritance and reduced testing, against unsuitability for abstract problems, extra complexity and overheads, then reach a conclusion that fits the scenario. Lists of single words earn little. When writing a subclass in C#, show the colon, call : base(...) in the constructor and use override where a method is replaced. For event-driven questions, use the terms event, event handler and control accurately and describe the sequence from event to output.

? Check yourself

1. A Fermanagh rugby club's system has a Member class and a new Junior class built from it. Explain what is meant by inheritance, using this example. [2]
2. A council's pitch-booking system has a superclass Booking with the attributes bookingRef, bookingDate and customerName. The subclass PitchBooking adds the attributes pitchNumber and floodlights. State how many attributes a PitchBooking object has and explain why. [2]
3. Explain what is meant by polymorphism, using an example. [3]

Answers: 1. A new, more specialised class (subclass / derived class, here Junior) takes on / inherits the attributes and methods of an existing, more general class (superclass / base class, here Member) (1) / and can add its own attributes and methods, e.g. a guardian's phone number, or modify inherited ones (1) 2. 5 (1) / it inherits all 3 attributes of Booking and adds 2 of its own (1) 3. The same method call / message (1) / produces different behaviour depending on the class of the object it is called on (1) / example, e.g. Fee() returns £120 for an Adult object but £40 for a Junior object because Junior overrides it (1)